CSS

# CHEAT SHEETS

*A Complete Reference for Modern CSS*

Cascade & Specificity · Selectors · Units & Functions · Custom Properties · Color · Box Model · Typography · Flexbox · Grid · Responsive & Print · Animation · Modern CSS

**77** REFERENCE SHEETS

CONTENTS


### FOUNDATIONS

| | |
|---|---|
| 01–04 | Introduction & Setup |
| 05–10 | Cascade & Inheritance |
| 11–18 | Selectors |
| 19–24 | Values, Units & Functions |
| 25–27 | Color |

### STYLING & LAYOUT

| | |
|---|---|
| 28–33 | Box Model & Sizing |
| 34–39 | Typography |
| 40–44 | Backgrounds & Visual Effects |
| 45–49 | Layout Fundamentals |
| 50–52 | Flexbox |
| 53–57 | Grid |

### RESPONSIVE & MOTION

| | |
|---|---|
| 58–62 | Responsive Design & Print |
| 63–68 | Transforms, Transitions & Animation |
| 69–72 | UI & Interaction |
| 73–77 | Organization, Best Practices & Quick Reference |


Covers modern CSS · cascade layers · :has() · container queries · grid · anchor positioning (2026)

Begin → Sheet 01

# Table of Contents




## INTRODUCTION & SETUP

01 Introduction & History
02 Adding CSS to a Page
03 Syntax & Rule Structure
04 Comments & Formatting

## CASCADE & INHERITANCE

05 The Cascade
06 Specificity
07 Inheritance & Global Keywords
08 Cascade Layers
09 Nesting & @scope
10 User-Agent Styles & Resets

## SELECTORS

11 Basic Selectors
12 Combinators
13 Attribute Selectors
14 Pseudo-Classes: User Action & Links
15 Pseudo-Classes: Structural
16 Pseudo-Classes: Form & State
17 Logical Pseudo-Classes
18 Pseudo-Elements

## VALUES, UNITS & FUNCTIONS

19 Values & Data Types
20 Length Units
21 Viewport & Container Units
22 Math Functions
23 Custom Properties
24 Custom Properties: Advanced

## COLOR

25 Color Syntax
26 Modern Color Spaces
27 Color Functions & Schemes

## BOX MODEL & SIZING

28 The Box Model
29 Margin & Padding
30 Sizing
31 Logical Properties
32 Borders & Outlines
33 Shadows

## TYPOGRAPHY

34 Font Properties
35 Web Fonts
36 Variable Fonts & Features
37 Text Styling
38 Spacing & Wrapping
39 Lists & Counters

## BACKGROUNDS & VISUAL EFFECTS

40 Backgrounds
41 Linear & Radial Gradients
42 Conic & Repeating Gradients
43 Filters & Blend Modes
44 Clipping, Masking & Images

## LAYOUT FUNDAMENTALS

45 Display & Formatting Contexts
46 Normal Flow & Floats
47 Positioning
48 Z-Index & Stacking Contexts
49 Overflow & Visibility

## FLEXBOX

50 Flexbox: Container
51 Flexbox: Items
52 Flexbox Patterns

## GRID

53 Grid: Tracks & Templates
54 Grid: Line-Based Placement
55 Grid: Areas & Auto-Placement
56 Grid: Alignment & Subgrid
57 Grid Patterns

## RESPONSIVE DESIGN & PRINT

58 Media Queries
59 Preference & Feature Queries
60 Container Queries
61 Print Styles
62 Responsive Patterns

## TRANSFORMS, TRANSITIONS & ANIMATION

63 2D Transforms
64 3D Transforms
65 Transitions
66 Keyframe Animations
67 Scroll-Driven Animations
68 View Transitions

## UI & INTERACTION

69 Cursors & Pointer Interaction
70 Styling Form Controls
71 Scrolling & Scroll Snap
72 Anchor Positioning & Top Layer

## ORGANIZATION, BEST PRACTICES & QUICK REFERENCE

73 Organizing Stylesheets
74 Naming Conventions
75 Accessible CSS
76 Debugging & Performance
77 Quick Reference

# Introduction & History



## WHAT CSS IS

**Cascading Style Sheets** describe how HTML is presented: layout, color, type, spacing and motion. HTML says *what* content is; CSS says *how it looks*. The language is developed by the **W3C CSS Working Group** as separate **modules** (Selectors, Color, Grid…) that each move through their own **levels**. There is no CSS4: "CSS3" just meant everything after CSS 2.1, and the yearly **CSS Snapshot** lists which modules are stable.

## NOTES

- A browser applies three style sources: its own UA stylesheet, the user's, and yours (the author)
- Three engines matter: **Blink** (Chrome, Edge, Opera), **WebKit** (Safari, every iOS browser) and **Gecko** (Firefox)
- Unknown properties and values are ignored, not errors, so new CSS degrades gracefully
- File extension is `.css` ; MIME type `text/css`

## ♿ ACCESSIBILITY

- Users can override author CSS with zoom, forced colors, custom fonts and user stylesheets
- Good CSS keeps content readable when styles fail; the HTML order is the reading order

## BROWSER SUPPORT

- **Baseline** labels a feature *Newly available* once it works in all major engines, and *Widely available* 30 months later
- Check caniuse.com or the Baseline badge on MDN before relying on a feature
- Use `@supports` to ship new features as progressive enhancement

## CODE EXPLAINED

→ `:root` : the `<html>` element; a home for site-wide variables

→ `body { … }` : a rule = selector + declaration block

→ `font-size: 2rem` : relative to the root font size

→ `a:hover` : pseudo-class, matches while the pointer is over a link

→ `@media` : an at-rule that applies its rules only when the condition is true

## A FIRST STYLESHEET

```css
/* styles.css */
:root {
  --brand: #5b4bc4;              /* custom property */
}

body {
  font-family: system-ui, sans-serif;
  line-height: 1.5;
  margin: 0;
}

h1 {
  color: var(--brand);
  font-size: 2rem;               /* 32px at default size */
}

a:hover {
  text-decoration-thickness: 2px;
}

.card {
  padding: 1rem;
  border: 1px solid #ddd;
  border-radius: 8px;
}

@media (min-width: 40rem) {     /* wider screens only */
  .card { padding: 2rem; }
}
```

## MILESTONES

| YEAR | EVENT |
|---|---|
| 1996 | CSS 1 published by the W3C |
| 1998 | CSS 2 adds positioning and media types |
| 2011 | CSS 2.1 finalized; work splits into modules |
| 2017 | Grid ships in all major browsers |
| 2022 | Cascade layers, `:has()` in Safari, container queries begin |
| 2023 | Nesting, `:has()` and container queries reach all engines |

## WHERE CSS RUNS

| ENGINE | BROWSERS |
|---|---|
| Blink | Chrome, Edge, Opera, Samsung Internet |
| WebKit | Safari (macOS, iOS); all iOS browsers |
| Gecko | Firefox |
| Baseline | Works in the current version of all three |

## EVERYDAY TOOLS

| TOOL | USE |
|---|---|
| DevTools | Inspect, edit and debug styles live |
| MDN | Reference and compat tables |
| caniuse | Browser support search |
| Stylelint | Lint CSS for errors and conventions |
| Prettier | Auto-format stylesheets |

## ⚠ COMMON MISTAKES

- Searching for "CSS3 feature" lists: modules version independently, so check each feature's support
- Treating an unsupported property as broken CSS: the browser skips just that declaration
- Testing only in Chrome: Safari is the only engine on iOS

## ✓ TIPS & BEST PRACTICES

- Keep MDN and DevTools open while you write; edit in DevTools, then copy the result back
- Check a feature's **Baseline** status before you use it without a fallback
- Write the plain version first, then layer newer features on top

## QUICK SUMMARY

- `selector { property: value; }` : the basic rule
- Cascade: UA, user and author styles combine by priority
- Modules + levels replace version numbers
- Baseline: Newly → Widely available after 30 months

# Adding CSS to a Page

link rel="stylesheet" · style element · style attribute · @import · media attribute · load order · render-blocking

## THREE WAYS IN

CSS reaches a page three ways: an **external stylesheet** linked with `<link rel="stylesheet">`, an **embedded** `<style>` element, or an **inline** `style` attribute. External files are the default: they are cached, shared across pages and keep markup clean. Stylesheets in the <head> are **render-blocking**: the browser waits for them before painting, which prevents a flash of unstyled content.

## NOTES

- Later sources win ties: a `<style>` after a `<link>` overrides it for equal specificity
- Inline `style` beats any selector unless the other declaration uses `!important`
- `@import` must come before all other rules except `@charset` and `@layer` statements
- A `media` attribute on `<link>` still downloads the file but stops it blocking render

## ♿ ACCESSIBILITY

- Inline styles can't respond to user preferences like `prefers-reduced-motion`; keep them out of the critical path
- Never put content in CSS alone; screen readers may skip or misread generated text

## BROWSER SUPPORT

- `<link>`, `<style>`, inline styles and `@import`: universal
- `@import … layer()`: Baseline 2022; `supports()` conditions on `@import` are newer, so check support
- `blocking="render"` on `<link>` / `<style>`: Chromium only

## CODE EXPLAINED

- → `rel="stylesheet"`: the only `rel` that applies CSS
- → `media="print"`: file loads but only applies when printing
- → `<style>`: rules for this page only; not cached separately
- → `@import … layer(theme)`: puts the file in a cascade layer
- → `@import … (width >= 60rem)`: a conditional import

## LINKING STYLESHEETS

```
<!-- in <head> -->
<link rel="stylesheet" href="/css/main.css">
<link rel="stylesheet" href="/css/print.css"
      media="print">
<link rel="preload" href="/css/late.css" as="style">

<style>
  /* embedded: page-specific rules */
  .hero { min-height: 60vh; }
</style>

<!-- inline: one element, avoid -->
<p style="color: tomato;">Warning</p>

<style>
  /* @import must open the stylesheet */
  @import url("reset.css");
  @import url("theme.css") layer(theme);
  @import url("wide.css") (width >= 60rem);
</style>
```

## METHODS COMPARED

| METHOD | CACHED | BEST FOR |
|---|---|---|
| `link` | Yes | All site styles |
| `style element` | With the page | Critical above-the-fold CSS |
| `style attribute` | No | One-off values set by JS |
| `@import` | Yes | Splitting files; adds a request chain |

## LINK ATTRIBUTES

| ATTRIBUTE | EFFECT |
|---|---|
| `href` | URL of the stylesheet |
| `media` | Only applies when the query matches |
| `disabled` | Stylesheet off until removed |
| `title` | Names an alternate stylesheet set |
| `crossorigin` | Needed to read cross-origin rules from JS |
| `integrity` | Subresource Integrity hash for CDN files |

## ⚠ COMMON MISTAKES

- Placing `@import` after other rules: the browser ignores it
- Chaining `@import`s: each waits for the last, slowing first paint
- Linking stylesheets at the end of `<body>`: content flashes unstyled first

## ✓ TIPS & BEST PRACTICES

- One or two linked files per page; let a bundler combine partials
- Inline only the small critical CSS for above-the-fold content
- Prefer `<link>` + `@layer` order over `@import` for performance

## QUICK SUMMARY

- `<link rel="stylesheet">`: external, cached, default
- `<style>`: per-page rules
- `style="…"`: inline, highest normal priority
- `@import`: top of file only; slower

# Syntax & Rule Structure

rule · selector · declaration block · property: value · at-rules · shorthand vs longhand · invalid declarations ignored

## ANATOMY OF A RULE

A stylesheet is a list of **rules**. A **style rule** is a **selector** followed by a **declaration block** in braces; each **declaration** is `property: value;`. **At-rules** start with `@` and either hold other rules (`@media`, `@layer`) or end with a semicolon (`@import`). When the browser can't parse a declaration it drops only that declaration and keeps going, which is what makes fallbacks work.

## NOTES

- Property names and keywords are case-insensitive; class names, IDs and URLs are not
- The last semicolon in a block is optional, but always write it
- A **shorthand** ( `margin` ) sets every longhand it covers, resetting the ones you omit
- An invalid selector in a selector list drops the whole rule (except inside `:is()` / `:where()` )

## ♿ ACCESSIBILITY

- Keep shorthand resets in mind: `font: 1rem serif` resets `line-height` to `normal` , which can cramp text
- Readable, consistently formatted CSS makes accessibility fixes easier to find

## BROWSER SUPPORT

- Core syntax: universal
- Native nesting inside rules: Baseline 2023
- Newer at-rules ( `@scope` , `@starting-style` ): check per feature

## CODE EXPLAINED

→ `h1, h2` : a selector list; both get the declarations

→ Second `background` overrides the first only where it parses

→ `margin: 1rem 2rem` : top/bottom then left/right

→ Statement at-rules end with `;` , block at-rules wrap rules

→ Comments can go anywhere whitespace is allowed

## RULES, DECLARATIONS, AT-RULES

```
/* style rule */
h1, h2 {                        /* selector list */
  color: #222;                  /* declaration */
  margin-block: 0 0.5em;
}

/* fallback: unknown value is dropped */
.box {
  background: #5b4bc4;          /* everyone */
  background: oklch(50% 0.2 280); /* newer */
}

/* shorthand vs longhands */
.a { margin: 1rem 2rem; }
.b {
  margin-top: 1rem;
  margin-right: 2rem;
  margin-bottom: 1rem;
  margin-left: 2rem;
}                               /* same result */

/* at-rules */
@import url("base.css");        /* statement */
@media (prefers-color-scheme: dark) {
  body { background: #111; }  /* block */
}
```

## PARTS OF A RULE

| PART | EXAMPLE |
|---|---|
| Selector | `.card > h2` |
| Declaration block | `{ … }` |
| Declaration | `color: red;` |
| Property | `color` |
| Value | `red`, `1px solid red` |
| `!important` | `color: red !important;` |

## COMMON AT-RULES

| AT-RULE | PURPOSE |
|---|---|
| `@media` | Conditional on viewport or device |
| `@supports` | Conditional on feature support |
| `@container` | Conditional on a container's size |
| `@layer` | Declare or fill a cascade layer |
| `@font-face` | Load a web font |
| `@keyframes` | Define an animation |
| `@import` | Pull in another stylesheet |

## ⚠ COMMON MISTAKES

- Missing semicolon: the next declaration becomes part of the previous value and both fail
- Using a shorthand after longhands: it silently resets them
- One bad selector in a list ( `a, ::-moz-x` ) kills the whole rule in other browsers

## ✓ TIPS & BEST PRACTICES

- Put the fallback first, the modern value second
- Use longhands when you only mean to change one side or part
- Validate with DevTools: dropped declarations show struck through with a warning icon

## QUICK SUMMARY

- `selector { prop: value; }` : style rule
- `@rule …;` or `@rule { … }` : at-rule
- Invalid declaration → skipped, rest still applies
- Shorthand resets omitted longhands

# Comments & Formatting

/* */ · no // comments · section banners · property order · Prettier · Stylelint · minification

## WRITING READABLE CSS

CSS has one comment syntax, `/* … */`, which can span lines but **cannot nest**. There is no `//` line comment: it silently breaks the next declaration or rule. Beyond comments, readability comes from consistent formatting (one declaration per line, a stable property order) and from tools like **Prettier** and **Stylelint** that enforce it. Comments and whitespace are stripped by minifiers, so explain freely.

## NOTES

- Nested `/* /* */ */` ends at the first `*/`, leaving stray text
- Comments are allowed between tokens, not inside them: `col/* */or` is invalid
- Use comments for *why* (a browser bug, a design decision), not *what*
- Section banners help in long files; split very long files instead

## ♿ ACCESSIBILITY

- Note accessibility decisions in comments (e.g. why an outline was replaced) so nobody removes them later

## BROWSER SUPPORT

- `/* */` comments: universal
- `//` comments: valid in Sass/Less source, invalid in plain CSS

## CODE EXPLAINED

→ Banner comments mark sections in long files

→ Grouping by Positioning → Box → Visual is one common order

→ `//` makes the parser treat the following text as a broken selector

→ A dated TODO explains why a prefix exists

→ Minifiers remove comments and whitespace for production

## COMMENTS AND FORMATTING

```
/* ========================================
   Components: Card
   ======================================== */

.card {
  /* Positioning */
  position: relative;
  z-index: 1;

  /* Box */
  display: grid;
  gap: 1rem;
  padding: 1.5rem;

  /* Visual */
  background: white;
  border-radius: 0.5rem;
}

// .old { color: red; }   ← NOT a comment:
//                          eats the next rule

/* TODO: remove after Safari 17 drops off */
.card { -webkit-backdrop-filter: blur(4px); }

/* minified: .card{position:relative;z-index:1} */
```

## PROPERTY ORDER STYLES

| STYLE | ORDER |
|---|---|
| Grouped | Positioning, box model, typography, visual, misc |
| Alphabetical | a → z; easy, no thinking |
| Concentric | Outside in: position, margin, border, padding, content |

## TOOLING

| TOOL | JOB |
|---|---|
| Prettier | Formats spacing, quotes, line breaks |
| Stylelint | Catches errors, enforces conventions |
| stylelint-order | Enforces property order |
| Lightning CSS | Minifies and adds prefixes |
| cssnano | PostCSS minifier |

## ⚠ COMMON MISTAKES

- Commenting out with `//`: breaks the next rule, not just the line
- Nesting comments to disable a block that already has comments
- Leaving dead commented-out code instead of deleting it (git keeps history)

## ✓ TIPS & BEST PRACTICES

- Format on save with Prettier; lint on commit with Stylelint
- Pick one property order and let a linter enforce it
- Keep comments that explain hacks next to the hack

## QUICK SUMMARY

- `/* … */`: the only CSS comment
- No `//`, no nesting
- Consistent order + formatter = readable diffs
- Minify for production

# The Cascade

origins · user-agent · author · !important · cascade order · source order · inline styles

## HOW CONFLICTS RESOLVE

When several declarations set the same property on the same element, the **cascade** picks one. It compares, in order: **origin and importance**, **context** (shadow DOM), **inline style**, **cascade layer**, **specificity**, and finally **order of appearance**. The first step that finds a difference decides; later steps never get a say. That's why a low-specificity rule in a later layer beats a high-specificity one in an earlier layer.

## NOTES

- Origins: user-agent (browser defaults), user (preferences), author (your CSS)
- `!important` flips origin priority: important user styles beat important author styles
- Inline `style=""` beats every selector of the same origin and importance
- Animations override normal declarations; transitions override everything else

## ♿ ACCESSIBILITY

- Important user styles beat yours by design; never fight them with `!important`
- Browser zoom and forced-colors mode act at the user-agent level, so test with them on

## BROWSER SUPPORT

- The cascade algorithm: universal
- Cascade layers step: Baseline 2022

## CODE EXPLAINED

→ `.note` beats `p` on specificity, the third tie-breaker step

→ Two `.note` rules tie on specificity, so source order decides

→ `!important` moves a declaration to a higher-priority stage

→ `@layer base, theme;` : later layers win over earlier ones

→ Unlayered styles beat all layered styles (for normal declarations)

## WHO WINS?

```
/* Same element: <p class="note" id="tip"> */

p        { color: black; }    /* (0,0,1) */
.note    { color: blue; }     /* (0,1,0) wins */

/* Equal specificity: last one wins */
.note    { color: green; }    /* now green */

/* !important jumps the queue */
p        { color: red !important; }  /* red */

/* Layers decide before specificity */
@layer base, theme;
@layer theme { p { color: purple; } }
@layer base  { #tip { color: gold; } }
/* unlayered .note above beats both layers */

/* Inline style beats any normal selector */
/* <p style="color: teal"> → teal
   unless another rule says !important */
```

## CASCADE ORDER

| STEP | HIGHER PRIORITY WINS |
|---|---|
| 1 Origin & importance | Transitions > important UA > important user > important author > animations > author > user > UA |
| 2 Context | Shadow DOM: outer context wins normal, inner wins important |
| 3 Element-attached | Inline `style` beats selectors |
| 4 Layers | Later layer > earlier; unlayered > layered (reversed for !important) |
| 5 Specificity | (id, class, type) compared left to right |
| 6 Order | Last declared wins |

## WHERE STYLES COME FROM

| ORIGIN | EXAMPLE |
|---|---|
| User-agent | `h1 { font-size: 2em }` built into the browser |
| User | Min font size, forced colors, user stylesheet |
| Author | Your `<link>`, `<style>`, `style=""` |
| Animation | Values from running `@keyframes` |
| Transition | Values mid-transition |

## ⚠ COMMON MISTAKES

- Raising specificity to beat a rule that actually wins on layer or origin
- Assuming the later file always wins: specificity is checked before order
- Sprinkling `!important` to win fights; it starts a new, harder fight

## ✓ TIPS & BEST PRACTICES

- In DevTools, the Styles pane lists winning declarations first and strikes through losers
- Use layers to set priority by intent instead of by specificity
- Keep specificity low and even so source order stays predictable

## QUICK SUMMARY

- Origin/importance → inline → layer → specificity → order
- First difference decides
- `!important` reverses origin and layer order
- Last declared wins only on a full tie

# Specificity

(a, b, c) weights · id · class · type · :is()/:not()/:has() weight · :where() zero · specificity wars

## SELECTOR WEIGHT

**Specificity** scores a selector as three numbers **(A, B, C)**: A counts **IDs**, B counts **classes, attributes and pseudo-classes**, C counts **type selectors and pseudo-elements**. Compare A first, then B, then C; the first column that differs decides. No number of classes beats one ID, because the columns never carry over. `:where()` always adds zero, which makes it the tool for writing easy-to-override defaults.

## NOTES

- `*` and combinators ( `>` , `+` , `~` , space) add nothing
- `:is()` , `:not()` and `:has()` take the weight of their *most specific* argument
- `:nth-child(2 of .x)` counts as one pseudo-class plus its argument
- Inline styles and `!important` aren't specificity; they're earlier cascade steps

## ♿ ACCESSIBILITY

- High-specificity component styles make it hard to add focus or contrast fixes later; keep weights low

## BROWSER SUPPORT

- Specificity rules: universal
- `:is()` / `:where()` : Baseline 2021; `:has()` : Baseline 2023

## CODE EXPLAINED

→ Columns are compared left to right: (1,0,0) beats (0,99,0)

→ Pseudo-elements count as types; pseudo-classes count as classes

→ `:is(#a, .b)` scores as `#a` even when `.b` is what matched

→ `:where()` lets any later class override a default

→ Repeating a class ( `.card.card` ) raises B without adding an ID

## SCORING SELECTORS

```
*                       /* (0,0,0) */
li                      /* (0,0,1) */
ul li                   /* (0,0,2) */
li::marker              /* (0,0,2) */
.item                   /* (0,1,0) */
[type="email"]          /* (0,1,0) */
a:hover                 /* (0,1,1) */
nav .link.active        /* (0,2,1) */
#main                   /* (1,0,0) */
#main .card h2          /* (1,1,1) */

/* functional pseudo-classes */
:is(#a, .b) p           /* (1,0,1)  most specific arg */
:where(#a, .b) p        /* (0,0,1)  always zero */
:not(.hidden)           /* (0,1,0) */
article:has(> img)      /* (0,0,2) */

/* easy-to-override defaults */
:where(.btn) { padding: .5em 1em; }
.btn-large   { padding: 1em 2em; } /* wins */

/* bumping without IDs */
.card.card { }          /* (0,2,0) */
```

## WHAT COUNTS

| SELECTOR PART | ADDS |
|---|---|
| `#id` | A (1,0,0) |
| `.class` | B (0,1,0) |
| `[attr]` | B |
| `:pseudo-class` | B |
| `type` | C (0,0,1) |
| `::pseudo-element` | C |
| `* and combinators` | nothing |
| `:where()` | nothing |

## COMPARE

| PAIR | WINNER |
|---|---|
| `#nav a vs .menu .item a` | `#nav a` (1,0,1) > (0,2,1) |
| `.a.b vs .c` | `.a.b` (0,2,0) |
| `div p vs p` | `div p` (0,0,2) |
| `:is(.a, #b) vs #c` | tie (1,0,0); order decides |
| `.x vs :where(#y) .x` | `.x` wins only on order; both (0,1,0) |

## ⚠ COMMON MISTAKES

- Styling with IDs: one ID outweighs any number of classes and forces more IDs to override
- Expecting 11 classes to beat an ID: columns don't overflow
- Forgetting `:is()` takes its heaviest argument, raising a whole rule's weight

## ✓ TIPS & BEST PRACTICES

- Style with classes; keep IDs for fragment links and JS hooks
- Wrap library and reset selectors in `:where()` so they're trivial to override
- Hover a selector in DevTools to see its specificity

## QUICK SUMMARY

- (IDs, classes/attrs/pseudo-classes, types/pseudo-elements)
- Compare columns left to right
- `:is/:not/:has` = heaviest argument
- `:where()` = 0

Sheet 07

# Inheritance & Global Keywords

inherited vs non-inherited · inherit · initial · unset · revert · revert-layer · all

## WHAT CHILDREN GET

If no declaration in the cascade sets a property on an element, **inherited properties** (mostly text: `color`, `font-*`, `line-height`, `visibility`, `cursor`) take the parent's computed value, while **non-inherited properties** (box and layout: `margin`, `border`, `background`, `display`) fall back to their **initial value**. Five **global keywords** work on every property to override that default, and `all` applies one of them to (almost) every property at once.

## NOTES

- Custom properties ( `--x` ) inherit by default
- Form controls don't inherit `font` in most UA stylesheets; set `font: inherit`
- `inherit` copies the parent's *computed* value: `em` sizes are already resolved
- `all` skips `direction` , `unicode-bidi` and custom properties

## ♿ ACCESSIBILITY

- `all: unset` on a button also removes its focus outline; add `:focus-visible` styles back
- Inherited `color` keeps text and icons ( `currentColor` ) in sync in high-contrast themes

## BROWSER SUPPORT

- `inherit` , `initial` , `unset` : universal
- `revert` : Baseline 2020; `revert-layer` : Baseline 2022

## CODE EXPLAINED

→ `color` and `font` flow down; `border` stops at `body`

→ `font: inherit` fixes controls that use the system font

→ `unset` acts as `inherit` or `initial` depending on the property

→ `revert` rolls back to the user-agent (or user) style

→ `revert-layer` rolls back to the value from earlier layers

## INHERITANCE IN ACTION

```
body {
  color: #222;            /* inherited by all text */
  font: 16px/1.5 system-ui;
  border: 1px solid red;  /* NOT inherited */
}

/* form controls ignore body font by default */
button, input, select, textarea {
  font: inherit;
  color: inherit;
}

a       { color: inherit; }   /* match parent */
.panel  { margin: initial; }  /* margin: 0 */
.reset  { color: unset; }     /* inherited → inherit */
.reset  { border: unset; }    /* non-inh. → initial */

/* back to browser default (UA style) */
h2.plain { font-size: revert; }

/* undo this layer, use the previous layer */
@layer theme {
  .card { color: revert-layer; }
}

/* blank slate component */
.widget { all: revert; }
```

## GLOBAL KEYWORDS

| KEYWORD | RESULT |
|---|---|
| `inherit` | Parent's computed value |
| `initial` | Spec's initial value (`display: initial` is `inline`) |
| `unset` | `inherit` if inherited, else `initial` |
| `revert` | Value from the previous origin (usually UA) |
| `revert-layer` | Value from the previous cascade layer |

## INHERITED?

| PROPERTY | INHERITS |
|---|---|
| `color, font-*, line-height` | Yes |
| `text-align, letter-spacing` | Yes |
| `visibility, cursor` | Yes |
| `list-style, quotes` | Yes |
| `margin, padding, border` | No |
| `background, display, width` | No |
| `opacity, transform` | No (but affects children visually) |

## ⚠ COMMON MISTAKES

- `display: initial` expecting the element's default: it gives `inline` ; use `revert`
- Expecting `opacity` to be inherited: children aren't set, they're painted with the parent
- `all: unset` wiping focus outlines and button cursor

## ✓ TIPS & BEST PRACTICES

- Set type on `body` once and let it inherit
- Use `color: inherit` on links inside colored components
- Prefer `revert` over `initial` to get browser defaults back

## QUICK SUMMARY

- Text props inherit; box props don't
- `inherit` / `initial` / `unset`
- `revert` → browser default
- `all: …` → every property at once

# Cascade Layers

@layer · layer order · anonymous layers · nested layers · @import layer() · unlayered styles · !important reversal

## PRIORITY BY INTENT

**Cascade layers** let you group styles and decide which group wins *before* specificity is compared. Declare the order once (`@layer reset, base, components, utilities;`) and later layers beat earlier ones no matter how specific their selectors are. **Unlayered** styles beat every layer. For `!important` declarations the order **reverses**, so an early layer like `reset` can protect critical rules.

## NOTES

- Order is fixed the first time a layer name appears; put the order statement first
- Adding rules to a layer later ( `@layer base { … }` ) keeps its original position
- Nested layers: `@layer framework.base` , ordered within their parent
- Anonymous `@layer { … }` blocks can't be added to later

## ♿ ACCESSIBILITY

- Put user-protective rules (focus rings, reduced motion) in a late layer or unlayered so components can't override them by accident

## BROWSER SUPPORT

- `@layer` , `revert-layer` , `@import layer()` : Baseline 2022
- Older browsers ignore layered rules entirely; test any fallback path

## CODE EXPLAINED

- → The first `@layer` statement fixes priority: utilities > components > base > reset
- → `.btn` (0,1,0) beats `#content a` (1,0,1) because its layer is later
- → `@import … layer(reset)` : library code can't beat your own layers
- → Unlayered `.legacy-banner a` wins over all layers
- → `components.card` : a sub-layer, ordered inside `components`

## LAYERING A SITE

```
/* 1. declare order: last wins */
@layer reset, base, components, utilities;

/* 2. third-party CSS into a low layer */
@import url("normalize.css") layer(reset);

@layer base {
  a { color: #5b4bc4; }
  #content a { color: navy; }   /* (1,0,1) */
}

@layer components {
  .btn { color: white; }        /* beats #content a */
}

@layer utilities {
  .text-red { color: red; }     /* beats .btn */
}

/* unlayered: beats every layer */
.legacy-banner a { color: black; }

/* nested layers */
@layer components {
  @layer card, modal;            /* components.card */
}
@layer components.card {
  .card { padding: 1rem; }
}
```

## NORMAL VS IMPORTANT

| SOURCE | NORMAL ORDER | !IMPORTANT ORDER |
|---|---|---|
| `reset layer` | Lowest | Highest |
| `base layer` | ↑ | ↓ |
| `utilities layer` | ↑ | ↓ |
| `unlayered` | Highest | Lowest |

## LAYER SYNTAX

| FORM | MEANING |
|---|---|
| `@layer a, b;` | Declare order only |
| `@layer a { }` | Add rules to layer a |
| `@layer { }` | Anonymous layer |
| `@layer a.b { }` | Nested sub-layer |
| `@import url(x) layer(a)` | Import into a layer |
| `revert-layer` | Fall back to earlier layers |

## ⚠ COMMON MISTAKES

- Forgetting the order statement: layers rank by first appearance, which can shift as files load
- Leaving framework CSS unlayered: it beats all your layered code
- Using `!important` in a late layer expecting it to win: important order is reversed

## ✓ TIPS & BEST PRACTICES

- Declare the full order in the first stylesheet that loads
- Import third-party CSS into the lowest layer
- Typical stack: `reset, base, layout, components, utilities`

## QUICK SUMMARY

- `@layer a, b;` : b beats a
- Unlayered beats layered
- `!important` reverses layer order
- Layer check happens before specificity

# Nesting & @scope

native nesting · & · nested at-rules · @scope · scope root · scope limit · proximity

## GROUPING RELATED RULES

**Native nesting** lets you write child rules inside a parent rule, like Sass. The & stands for the parent selector and is required when attaching to it (`&:hover`, `&.active`). Nested rules compile to `:is()` behavior, so they take the parent's highest specificity. **`@scope`** limits rules to a subtree, with an optional lower boundary (`to`), and when two scoped rules tie, the one with the **closer** scope root wins.

## NOTES

- `&` can go anywhere: `.theme-dark &` matches the parent inside `.theme-dark`
- You can nest `@media`, `@supports`, `@container` and `@layer` directly in a rule
- Unlike Sass, `&__title` concatenation doesn't work; nesting is selector-based
- Inside `@scope (.card)`, `:scope` refers to the scope root

## ♿ ACCESSIBILITY

- Deep nesting makes it easy to lose `:focus-visible` states; nest them next to `:hover`

## BROWSER SUPPORT

- Native nesting: Baseline 2023 (bare type selectors like `p {}` nested since late 2023)
- `@scope`: newer; check support and keep a plain-selector fallback

## CODE EXPLAINED

- → `& h2`: descendant of the parent; `&` is optional before a combinator
- → `&:hover`: no space, so it attaches to `.card` itself
- → `.theme-dark &`: parent inside an ancestor
- → Nested `@media` wraps declarations for the parent selector
- → `to (.content)`: the scope stops at this boundary

## NESTING AND SCOPE

```
.card {
  padding: 1rem;

  & h2 { margin-top: 0; }       /* .card h2 */
  > p  { color: #444; }         /* .card > p */

  &:hover,
  &:focus-within { outline: 2px solid; }
  &.is-featured  { border-color: gold; }

  .theme-dark & { background: #222; }

  @media (width >= 40rem) {
    padding: 2rem;              /* applies to .card */
  }
}

/* @scope: only inside .card, not .card .content */
@scope (.card) to (.content) {
  img     { border-radius: 8px; }
  :scope  { container-type: inline-size; }
}

/* proximity: nearer root wins on a tie */
@scope (.light) { p { color: black; } }
@scope (.dark)  { p { color: white; } }
```

## NESTING FORMS

| WRITE | MEANS |
|---|---|
| `& .x` | .parent .x |
| `&.x` | .parent.x |
| `&:hover` | .parent:hover |
| `> .x` | .parent > .x |
| `+ .x` | .parent + .x |
| `.x &` | .x .parent |
| `& &` | .parent .parent |

## @SCOPE PARTS

| PART | MEANING |
|---|---|
| `@scope (.a)` | Root: rules only match inside .a |
| `to (.b)` | Limit: stop at .b |
| `:scope` | The root element itself |
| `proximity` | Closer root wins ties |
| `@scope { } in <style>` | Root is the style element's parent |

## ⚠ COMMON MISTAKES

- Writing `&-title` or `&__el` expecting Sass concatenation
- Nesting more than three levels: selectors get long and specific
- Forgetting nested rules take the parent's specificity (`#id` parents make everything heavy)

## ✓ TIPS & BEST PRACTICES

- Nest pseudo-classes, states and media queries; avoid deep descendant chains
- Use `@scope … to (…)` for components that contain other components
- Keep a plain-selector fallback where `@scope` support matters

## QUICK SUMMARY

- `&`: the parent selector
- Nested `@media` works inside rules
- `@scope (root) to (limit)`
- Proximity breaks scope ties

# User-Agent Styles & Resets

UA stylesheet · normalize.css · modern reset · box-sizing reset · margin defaults · all: unset

## STARTING FROM DEFAULTS

Every browser ships a **user-agent stylesheet**: margins on `body` and headings, bullets on lists, blue underlined links, bold `th`. They differ slightly between engines. A **reset** clears or evens out these defaults so your styles start from a known base. Old "hard" resets removed everything; modern resets keep useful defaults and fix the annoying ones, like `content-box` sizing and images overflowing.

## NOTES

- **normalize.css** makes browsers consistent but keeps defaults
- A **modern reset** (Josh Comeau, Andy Bell) is ~30 lines of opinionated fixes
- Put the reset in the lowest cascade layer so everything overrides it
- Wrapping reset selectors in `:where()` keeps their specificity at zero

## ♿ ACCESSIBILITY

- Removing list styles with `list-style: none` drops list semantics in Safari/VoiceOver; add `role="list"` if the count matters
- Never reset `outline: none` globally; it hides keyboard focus

## BROWSER SUPPORT

- Every rule in a modern reset is Baseline Widely available
- `text-wrap: balance` (often added to headings): Baseline 2024

## CODE EXPLAINED

→ `box-sizing: border-box` : width includes padding and border

→ `* { margin: 0 }` : remove UA margins, add spacing back on purpose

→ `text-size-adjust` : stop iOS inflating text in landscape

→ `max-width: 100%` keeps media inside its container

→ Reduced-motion rule lives in the reset so nothing forgets it

## A MODERN RESET

```
@layer reset {
  *, *::before, *::after {
    box-sizing: border-box;
  }

  * { margin: 0; }

  html {
    -webkit-text-size-adjust: none;
    text-size-adjust: none;
  }

  body {
    min-height: 100dvh;
    line-height: 1.5;
  }

  img, picture, video, canvas, svg {
    display: block;
    max-width: 100%;
  }

  input, button, textarea, select {
    font: inherit;
  }

  p, h1, h2, h3 { overflow-wrap: break-word; }

  @media (prefers-reduced-motion: reduce) {
    * { animation-duration: 0.01ms !important; }
  }
}
```

## NOTABLE UA DEFAULTS

| ELEMENT | DEFAULT |
|---|---|
| `body` | `margin: 8px` |
| `h1–h6` | Bold, margins, `h1` 2em |
| `p, ul, ol` | Top and bottom margins |
| `ul, ol` | `padding-inline-start: 40px` |
| `a` | Blue, underlined, pointer |
| `button, input` | System font, not inherited |
| `img` | `display: inline` (gap below) |

## RESET APPROACHES

| APPROACH | IDEA |
|---|---|
| `Hard reset` | Zero everything (Eric Meyer 2011) |
| `normalize.css` | Keep defaults, fix differences |
| `Modern reset` | A few targeted fixes |
| `all: revert` | Undo author styles in one subtree |

## ⚠ COMMON MISTAKES

- `outline: none` in a reset: removes keyboard focus indicators site-wide
- Leaving the reset unlayered: it then beats component styles in layers
- Inline images leaving a gap under them; set `display: block`

## ✓ TIPS & BEST PRACTICES

- Keep the reset tiny and read every line of it
- Pair `* { margin: 0 }` with a spacing system ( `gap` , `margin-block` utilities)
- Test forms and lists after adding a reset

## QUICK SUMMARY

- UA styles differ by browser
- Reset in `@layer reset`
- `box-sizing: border-box` everywhere
- Keep focus outlines

# Basic Selectors

type · .class · #id · * · selector lists · compound selectors · case sensitivity

## PICKING ELEMENTS

A **simple selector** matches elements by one trait: **type** (`p`), **class** (`.note`), **ID** (`#intro`), **attribute** (`[href]`) or the **universal** `*`. Chain them with no spaces into a **compound selector** (`button.primary[disabled]`) to require *all* conditions on the same element. Separate selectors with commas to make a **selector list** that shares one declaration block.

## NOTES

- An element can have many classes: `class="btn btn-primary"`
- IDs must be unique per page; `#id` also has very high specificity
- Class and ID names are case-sensitive; HTML type selectors are not
- Names can't start with a digit unless escaped: `.\31 0col`

## ♿ ACCESSIBILITY

- Select by semantic element or ARIA state ( `[aria-expanded="true"]` ) so styles follow real meaning
- Avoid styling `div` s to look like buttons; style the `button`

## BROWSER SUPPORT

- All basic selectors: universal

## CODE EXPLAINED

→ `.note` : any element with class `note`

→ `button.primary` : a `<button>` that also has class `primary`

→ `.btn.btn--large` : both classes on one element

→ Commas make a list; each item is matched independently

→ A space between `.primary` and `button` means descendant, not compound

## SIMPLE AND COMPOUND

```
/* type */
p         { line-height: 1.6; }

/* class */
.note     { background: #fff8e1; }

/* id: unique, heavy */
#site-header { position: sticky; }

/* universal */
* { box-sizing: border-box; }

/* compound: all on the same element */
button.primary        { background: navy; }
a.external[target]    { padding-right: 1em; }
.btn.btn--large       { font-size: 1.25rem; }

/* selector list: shared block */
h1, h2, h3 {
  font-family: Georgia, serif;
  line-height: 1.2;
}

/* type must come first in a compound */
.primary button  { }   /* descendant! not same */
```

## SIMPLE SELECTORS

| SELECTOR | MATCHES |
|---|---|
| `p` | Every <p> |
| `.card` | Elements with class `card` |
| `#main` | The element with `id="main"` |
| `*` | Every element |
| `[href]` | Elements with an `href` attribute |
| `a.btn` | <a> with class `btn` |
| `ns\|p` | <p> in a namespace (SVG/XML) |

## NAMING RULES

| RULE | EXAMPLE |
|---|---|
| Case-sensitive | `.Card` ≠ `.card` |
| Hyphens, underscores OK | `.card__title--big` |
| No leading digit | `.3col` invalid; escape it |
| Escape specials | `.md\:flex` for `md:flex` |
| Unicode allowed | `.café` |

## ⚠ COMMON MISTAKES

- Adding a space in `button .primary` : that selects `.primary` *inside* a button
- Reusing an ID on several elements: CSS still matches all, but JS and anchors break
- Styling by ID and then needing another ID to override

## ✓ TIPS & BEST PRACTICES

- Use classes for styling, element selectors for base typography
- Keep compound selectors short; two parts is usually enough
- Group shared styles with selector lists instead of repeating blocks

## QUICK SUMMARY

- `p` · `.class` · `#id` · `*`
- No space = same element
- Comma = list
- Classes are case-sensitive

# Combinators

descendant · > child · + next-sibling · ~ subsequent-sibling · combinators in nesting

## RELATIONSHIPS BETWEEN ELEMENTS

**Combinators** join selectors by the relationship between elements in the tree. A **space** matches any descendant, > only direct children, + the element right after a sibling, and ~ any later sibling. Selectors are read right to left: the rightmost part is the element being styled, and everything to its left is a condition on its ancestors or earlier siblings.

## NOTES

- Sibling combinators only look forward; use `:has()` to style an earlier sibling
- `*` with a combinator ( `.grid > *` ) targets every child
- Long descendant chains are fragile when markup changes
- In nesting, a leading combinator ( `> li` ) is relative to the parent

## ♿ ACCESSIBILITY

- The "lobotomized owl" `* + *` spacing keeps rhythm without margins that collapse into focus outlines
- Don't rely on DOM order hacks ( `~` ) to show content that screen readers reach in a different order

## BROWSER SUPPORT

- All four combinators: universal
- Column combinator `||` : not supported anywhere

## CODE EXPLAINED

→ `article p` : paragraphs anywhere inside an article

→ `.menu > li` : not nested `li` s in submenus

→ `h2 + p` : only the first paragraph after each `h2`

→ `.flow > * + *` : every child except the first gets top margin

→ `li + li` : borders between items, none on the first

## FOUR COMBINATORS

```
/* descendant: any depth */
article p          { max-width: 65ch; }

/* child: direct children only */
.menu > li         { display: inline-block; }

/* next sibling: immediately after */
h2 + p             { margin-top: 0; font-size: 1.1em; }
label + input      { margin-left: .5rem; }

/* subsequent sibling: any later sibling */
h2 ~ p             { color: #444; }

/* checkbox hack: state → later sibling */
#nav-toggle:checked ~ .nav { display: block; }

/* flow spacing ("owl") */
.flow > * + *      { margin-top: 1em; }

/* every child */
.grid > *          { min-width: 0; }

/* in nesting */
.list {
  > li     { padding: .25rem; }   /* .list > li */
  li + li  { border-top: 1px solid #ddd; }
}
```

## COMBINATORS

| SYNTAX | MATCHES B WHEN |
|---|---|
| `A B` | B is inside A at any depth |
| `A > B` | B is a direct child of A |
| `A + B` | B immediately follows A (same parent) |
| `A ~ B` | B comes after A (same parent) |

## PATTERNS

| SELECTOR | USE |
|---|---|
| `li + li` | Divider between items |
| `* + *` | Consistent vertical rhythm |
| `.grid > *` | Style every grid item |
| `input:checked + label` | Style a label from input state |
| `h2 ~ *` | Everything after a heading |
| `ul ul` | Nested lists |

## ⚠ COMMON MISTAKES

- Expecting `+` or `~` to match an *earlier* sibling: they only look forward
- Using `div > div > div > p` : breaks as soon as markup changes
- Forgetting siblings must share the same parent

## ✓ TIPS & BEST PRACTICES

- Prefer `>` over a space when you only mean direct children
- Use `li + li` instead of `:not(:first-child)` for in-between styles
- Reach backward with `:has()` : `h2:has(+ p)`

## QUICK SUMMARY

- space = descendant
- `>` = child
- `+` = next sibling
- `~` = any later sibling

# Attribute Selectors

[attr] · [attr=v] · ~= · |= · ^= · $= · *= · i/s flags

## MATCHING ON ATTRIBUTES

**Attribute selectors** match elements by the presence or value of an attribute. `[attr]` checks that it exists; the operator forms compare the value exactly (=), by word (~=), by prefix (^=), suffix ($=), substring (*=) or language-style prefix (|=). Add `i` before the bracket closes to compare case-insensitively. They're ideal for styling by state, link type or `data-*` attributes.

## NOTES

- Values with spaces or special characters need quotes: `[title="Hi there"]`
- Each attribute selector adds (0,1,0) specificity, like a class
- `[class~="btn"]` equals `.btn` ; `[class*="btn"]` also matches `btn-group`
- `data-*` attributes make clean state hooks: `[data-state="open"]`

## ♿ ACCESSIBILITY

- Style from ARIA state so visuals can't drift from what assistive tech hears: `[aria-current="page"]` , `[aria-expanded="true"]`
- `[aria-disabled="true"]` elements stay focusable; style them as disabled anyway

## BROWSER SUPPORT

- All operators: universal
- `i` flag: universal in current browsers; `s` flag: Firefox only

## CODE EXPLAINED

→ `^=` starts with, `$=` ends with, `*=` contains

→ `~=` matches one whole word in a space-separated list

→ `|=` matches `en` exactly or `en-` followed by anything

→ `i` makes the value comparison case-insensitive

→ ARIA and `data-*` attributes double as styling state

## ATTRIBUTE OPERATORS

```
[disabled]                  { opacity: .5; }
[type="checkbox"]           { accent-color: #5b4bc4; }

/* link types */
a[href^="https://"]         { padding-right: 1em; }
a[href^="mailto:"]::before  { content: "✉ "; }
a[href$=".pdf"]::after      { content: " (PDF)"; }
a[href*="example.com"]      { font-weight: 600; }

/* word in a space-separated list */
[rel~="nofollow"]           { color: gray; }

/* lang prefix: en, en-US, en-GB */
[lang|="en"]                { quotes: "“" "”"; }

/* case-insensitive */
a[href$=".PDF" i]           { color: crimson; }

/* state hooks */
[data-state="open"]         { display: block; }
[aria-current="page"]       { font-weight: 700; }
[aria-expanded="true"] .icon { rotate: 180deg; }

/* combine with type */
input[type="email"]:invalid { border-color: red; }
```

## OPERATORS

| SELECTOR | MATCHES WHEN VALUE |
|---|---|
| `[a]` | Attribute present (any value) |
| `[a="v"]` | Is exactly v |
| `[a~="v"]` | Contains the word v |
| `[a\|="v"]` | Is v or starts with v– |
| `[a^="v"]` | Starts with v |
| `[a$="v"]` | Ends with v |
| `[a*="v"]` | Contains v anywhere |
| `[a="v" i]` | Case-insensitive match |

## USEFUL HOOKS

| SELECTOR | STYLES |
|---|---|
| `[href^="#"]` | In-page links |
| `[target="_blank"]` | Links that open a new tab |
| `[type="submit"]` | Submit inputs |
| `[hidden]` | Hidden elements (UA: `display: none`) |
| `[dir="rtl"]` | Right-to-left blocks |
| `[aria-invalid="true"]` | Fields flagged invalid |

## ⚠ COMMON MISTAKES

- Using `[class*="col"]` and accidentally matching `.collapse`
- Forgetting quotes around values containing spaces, dots or colons
- Setting `display` on `[hidden]` elements in your CSS and making them visible

## ✓ TIPS & BEST PRACTICES

- Prefer `~=` over `*=` for class-like lists
- Drive component state with `data-state` or ARIA attributes instead of extra classes
- Add `[hidden] { display: none !important; }` to your reset

## QUICK SUMMARY

- `[a]` exists · `[a=v]` exact
- `^=` start · `$=` end · `*=` contains
- `~=` word · `|=` lang prefix
- `i` flag ignores case

# Pseudo-Classes: User Action & Links

:hover · :active · :focus · :focus-visible · :focus-within · :link · :visited · :target

## REACTING TO THE USER

**User-action pseudo-classes** match elements in a temporary interaction state: `:hover` under the pointer, `:active` while pressed, `:focus` when focused. **`:focus-visible`** matches only when the browser decides a focus ring should show (keyboard use), so you can style keyboard focus without boxing every mouse click. **Link pseudo-classes** split links into `:link` (unvisited) and `:visited`, and `:target` matches the element named by the URL fragment.

## NOTES

- Order link states **LVHA**: `:link`, `:visited`, `:hover`, `:active` (equal specificity)
- `:visited` can only change colors, for privacy
- `:focus-within` matches a parent when anything inside it has focus
- `:any-link` matches both `:link` and `:visited`

## ♿ ACCESSIBILITY

- Never remove focus styles without a replacement; use `:focus-visible` with at least a 2px, 3:1-contrast outline
- Touch screens have no hover; don't hide essential content behind `:hover`

## BROWSER SUPPORT

- All on this sheet: Baseline Widely available
- `:focus-visible`: Baseline 2022

## CODE EXPLAINED

→ LVHA order stops `:visited` from overriding `:hover`

→ `:focus-visible` shows the ring for keyboard, not mouse clicks

→ `:focus:not(:focus-visible)`: an older pattern; usually unneeded now

→ `:focus-within` styles a container while a child is focused

→ `@media (hover: hover)` keeps hover effects off touch devices

## STATES IN ORDER

```
/* LVHA order */
a:link    { color: #5b4bc4; }
a:visited { color: #7b1fa2; }
a:hover   { text-decoration-thickness: 3px; }
a:active  { color: #311b92; }

/* keyboard focus only */
:focus-visible {
  outline: 3px solid #ff9800;
  outline-offset: 2px;
}

/* mouse focus: no ring (keyboard ring kept) */
button:focus:not(:focus-visible) {
  outline: none;
}

/* highlight a whole form row */
.field:focus-within { background: #f3f0ff; }

/* jump-to section */
:target {
  scroll-margin-top: 4rem;
  animation: flash 1s;
}

/* hover only on devices that can hover */
@media (hover: hover) {
  .card:hover { translate: 0 -2px; }
}
```

## PSEUDO-CLASSES

| SELECTOR | MATCHES |
|---|---|
| `:hover` | Pointer is over the element |
| `:active` | Being pressed or clicked |
| `:focus` | Has focus (any input method) |
| `:focus-visible` | Focused and a ring should show |
| `:focus-within` | Element or a descendant has focus |
| `:link` | Unvisited link with `href` |
| `:visited` | Visited link |
| `:target` | Element whose `id` is the URL `#fragment` |

## :VISITED LIMITS

| ALLOWED | NOT ALLOWED |
|---|---|
| `color`, `background-color` | display, size, position |
| `border-color`, `outline-color` | background-image |
| `fill`, `stroke` (SVG) | Read via `getComputedStyle` |

## ⚠ COMMON MISTAKES

- `:focus { outline: none; }` with no replacement: keyboard users get lost
- Writing `a:hover` before `a:visited`: visited links never show hover
- Showing menus on `:hover` only: touch and keyboard users can't open them

## ✓ TIPS & BEST PRACTICES

- Style `:focus-visible` globally once, then tweak per component
- Pair `:hover` with `:focus-visible` so keyboard users get the same feedback
- Use `:target` with `scroll-margin-top` under a sticky header

## QUICK SUMMARY

- `:hover` · `:active` · `:focus`
- `:focus-visible` = keyboard ring
- `:focus-within` = parent of focus
- Links: LVHA order

# Pseudo-Classes: Structural

:first-child · :last-child · :nth-child(An+B) · :nth-of-type · :only-child · :empty · :root · of S

## POSITION IN THE TREE

**Structural pseudo-classes** match elements by where they sit among their siblings, with no extra classes. `:nth-child(An+B)` is the workhorse: it counts siblings from 1, so `2n` is even, `2n+1` odd and `-n+3` the first three. The `-of-type` variants count only siblings of the same element type, and `:nth-child(An+B of S)` counts only siblings that match a selector.

## NOTES

- Counting is 1-based and includes all element siblings (not text nodes)
- `:nth-last-child()` counts from the end
- `:empty` matches elements with no children and no text, not even whitespace
- `:root` is `<html>` in HTML documents but has class-level specificity (0,1,0)

## ACCESSIBILITY

- Zebra stripes with `:nth-child(even)` help scanning wide tables; keep contrast between stripes and text
- Don't use position-based styling to convey meaning a screen reader won't announce

## BROWSER SUPPORT

- All structural pseudo-classes: universal
- `:nth-child(An+B of S)` : Baseline 2023

## CODE EXPLAINED

- → `even = 2n`, `odd = 2n+1`
- → `-n + 3` : n = 0, 1, 2 gives 3, 2, 1
- → `p:first-of-type` ignores earlier `h2` or `img` siblings
- → `of :not([hidden])` keeps stripes right when rows are filtered
- → Quantity query: the first item checks the total count from the end

## COUNTING SIBLINGS

```
li:first-child          { font-weight: bold; }
li:last-child           { border-bottom: none; }
li:only-child           { list-style: none; }

/* An+B */
tr:nth-child(even)      { background: #f6f6f6; }
tr:nth-child(2n + 1)    { }         /* odd */
li:nth-child(3n)        { }         /* 3, 6, 9… */
li:nth-child(-n + 3)    { }         /* first 3 */
li:nth-child(n + 4)     { }         /* 4 onward */
li:nth-last-child(2)    { }         /* 2nd from end */

/* of type: only <p> siblings counted */
article p:first-of-type { font-size: 1.2em; }
img:nth-of-type(2n)     { float: right; }

/* of S: count only visible rows */
tr:nth-child(even of :not([hidden])) {
  background: #f6f6f6;
}

/* quantity query: style when 4+ items */
li:first-child:nth-last-child(n + 4),
li:first-child:nth-last-child(n + 4) ~ li {
  flex-basis: 25%;
}

.alert:empty { display: none; }
```

## AN+B CHEATS

| EXPRESSION | MATCHES |
|---|---|
| `odd` / `2n+1` | 1, 3, 5… |
| `even` / `2n` | 2, 4, 6… |
| `3` | Only the 3rd |
| `3n` | 3, 6, 9… |
| `n+3` | 3rd onward |
| `-n+3` | First three |
| `3n+1` | 1, 4, 7… |

## STRUCTURAL SELECTORS

| SELECTOR | MATCHES |
|---|---|
| `:first-child` / `:last-child` | First / last sibling |
| `:only-child` | Has no siblings |
| `:nth-child(An+B)` | By position |
| `:nth-last-child()` | By position from the end |
| `:first-of-type` | First of its element type |
| `:only-of-type` | Only one of its type |
| `:empty` | No children or text |
| `:root` | The document root |

## ⚠ COMMON MISTAKES

- `p:first-child` when an `h2` comes first: nothing matches; use `:first-of-type`
- Expecting `:nth-of-type` to count classes: it counts element types only
- `:empty` failing because of a stray space or newline inside the element

## ✓ TIPS & BEST PRACTICES

- Use `:nth-child(… of …)` instead of re-indexing classes after filtering
- Prefer `li + li` over `li:not(:first-child)` for in-between borders
- Hide empty alert or badge containers with `:empty`

## QUICK SUMMARY

- `:nth-child(An+B)` 1-based
- `-of-type` counts same tag only
- `of S` counts matches only
- `:empty` = nothing, not even spaces

# Pseudo-Classes: Form & State

:checked · :disabled · :required · :valid · :user-invalid · :placeholder-shown · :open · :popover-open

## STYLING CONTROL STATES

Form controls expose their state through pseudo-classes: `:checked` for ticked boxes and selected radios, `:disabled`/`:enabled`, `:required`/`:optional`, and validity via `:valid` and `:invalid`. Because `:invalid` matches on page load (an empty required field is invalid), prefer **`:user-invalid`**, which waits until the user has interacted. Newer state pseudo-classes cover open `<details>`, dialogs and popovers.

## NOTES

- `:indeterminate` matches a checkbox set to the mixed state via JS
- `:placeholder-shown` is true while the field is empty and showing its placeholder
- `:default` matches the default button and initially-checked options
- `:read-only` matches non-editable inputs and ordinary text elements

## ♿ ACCESSIBILITY

- Don't signal errors by color alone; add an icon or message next to the field
- Disabled controls are skipped by Tab; consider `aria-disabled` with styling when users need to discover them

## BROWSER SUPPORT

- `:checked`, `:disabled`, `:valid`, `:required`: universal
- `:user-invalid` / `:user-valid`: Baseline 2023; `:popover-open`: Baseline 2024
- `:open` (details, dialog, select): newer; check support

## CODE EXPLAINED

- → `:checked + label`: style the label next to a ticked input
- → `:user-invalid` stays quiet until the user edits or submits
- → `:placeholder-shown`: drives floating-label effects
- → `form:has(:user-invalid)`: parent styling from a child's state
- → `details[open]` works everywhere; `:open` is the newer equivalent

## FORM STATES

```
input:checked + label        { font-weight: 600; }
:disabled          { opacity: .5; cursor: not-allowed; }
input:required     { border-left: 3px solid #5b4bc4; }

/* only after the user has touched the field */
input:user-invalid           { border-color: #c62828; }
input:user-invalid + .error  { display: block; }
input:user-valid             { border-color: #2e7d32; }

/* floating label */
.field input:placeholder-shown + label {
  translate: 0 1.5rem;
}

/* mixed-state checkbox */
input:indeterminate          { accent-color: gray; }

/* parent reacts to state with :has() */
form:has(:user-invalid) .submit { opacity: .6; }

/* open/closed components */
details[open] > summary      { color: #5b4bc4; }
details:open > summary       { }   /* newer */
[popover]:popover-open       { animation: pop .2s; }
dialog[open]::backdrop       { background: #0008; }
```

## INPUT STATE

| SELECTOR | MATCHES |
|---|---|
| `:checked` | Ticked checkbox, chosen radio or option |
| `:indeterminate` | Mixed checkbox or unset radio group |
| `:disabled / :enabled` | Disabled or usable controls |
| `:read-only / :read-write` | Editable or not |
| `:required / :optional` | Has or lacks `required` |
| `:placeholder-shown` | Showing its placeholder |
| `:default` | Default button or preselected choice |

## VALIDITY & OPEN

| SELECTOR | MATCHES |
|---|---|
| `:valid / :invalid` | Passes or fails constraints (immediately) |
| `:user-valid / :user-invalid` | Same, after interaction |
| `:in-range / :out-of-range` | `min`/`max` check |
| `:autofill` | Field filled by the browser |
| `:open` | Open details, dialog or select |
| `:popover-open` | Showing popover |
| `:modal` | Dialog opened with `showModal()` |

## ⚠ COMMON MISTAKES

- Styling `:invalid` red: every empty required field is red before the user types
- Hiding the native checkbox with `display: none`: it leaves the tab order and the accessibility tree
- Forgetting `[open]` attribute selectors still work where `:open` doesn't

## ✓ TIPS & BEST PRACTICES

- Use `:user-invalid` for error styling
- Style custom checkboxes with `appearance: none` on the real input
- Combine with `:has()` to style fieldsets, forms or labels from control state

## QUICK SUMMARY

- `:checked` · `:disabled` · `:required`
- `:user-invalid` for errors
- `:placeholder-shown` = empty field
- `:popover-open` · `:modal` · `[open]`

# Logical Pseudo-Classes

:is() · :where() · :not() · :has() · forgiving selector lists · parent-selector patterns

## SELECTOR LOGIC

Four functional pseudo-classes take selector lists. **`:is()`** matches any of them (OR) and takes the highest argument's specificity; **`:where()`** does the same with zero specificity. **`:not()`** excludes matches. **`:has()`** is the long-wanted **parent selector**: it matches an element that *contains* (or is followed by) something matching a relative selector, so a card can react to having an image or a form to holding an invalid field.

## NOTES

- `:is()` and `:where()` use **forgiving** lists: one invalid selector doesn't kill the rule
- `:not(.a, .b)` excludes both; it accepts complex selectors
- `:has()` arguments are relative: `:has(> img)`, `:has(+ p)`
- `:has()` can't be nested inside another `:has()`

## ♿ ACCESSIBILITY

- `:has(:focus-visible)` lets a whole card show focus when its link is focused
- `body:has(dialog[open])` can lock scroll while a modal is open

## BROWSER SUPPORT

- `:is()`, `:where()`, complex `:not()`: Baseline 2021
- `:has()`: Baseline 2023; use `@supports selector(:has(a))` for a fallback

## CODE EXPLAINED

- → `:is()` turns 3 × 2 combinations into one selector
- → `:where()` resets lists without raising specificity
- → `:not(a, b)` excludes internal and fragment links
- → `h2:has(+ p)`: style a heading based on what follows it
- → `:has(> :nth-child(5))`: a container that has at least five children

## IS, WHERE, NOT, HAS

```
/* :is() shortens long lists */
:is(article, section, aside) :is(h1, h2) {
  margin-top: 0;
}

/* :where() = zero specificity defaults */
:where(ul, ol)[class] { list-style: none; }

/* :not() */
a:not([href^="/"], [href^="#"]) {
  text-decoration-style: dotted;   /* external */
}
.btn:not(:disabled):hover { filter: brightness(1.1); }

/* :has(): parent and previous-sibling selector */
.card:has(img)           { grid-template-rows: auto 1fr; }
.card:has(> h2 + p)      { padding-top: 0; }
label:has(+ input:required)::after { content: " *"; }
h2:has(+ p)              { margin-bottom: .25rem; }

/* page-level state */
body:has(dialog[open])   { overflow: hidden; }
form:has(:user-invalid) button[type="submit"] {
  opacity: .6;
}

/* quantity: grid with 5+ items */
.grid:has(> :nth-child(5)) { --cols: 3; }
```

## COMPARED

| PSEUDO | LOGIC | SPECIFICITY |
|---|---|---|
| `:is()` | Any of (OR) | Most specific argument |
| `:where()` | Any of (OR) | Zero |
| `:not()` | None of | Most specific argument |
| `:has()` | Contains / followed by | Most specific argument |

## :HAS() PATTERNS

| SELECTOR | MEANING |
|---|---|
| `a:has(> img)` | Links that wrap an image |
| `li:has(> ul)` | Items with a submenu |
| `h2:has(+ p)` | Heading followed by a paragraph |
| `.field:has(:focus-visible)` | Field group in focus |
| `html:has(.dark-toggle:checked)` | Whole-page theme switch |
| `:has(:checked)` | Contains a checked control |

## ⚠ COMMON MISTAKES

- Using `:is(#id, .class)` and getting ID weight for every match
- Writing `:has(img)` when you mean direct children ( `:has(> img)` )
- Very broad `:has()` on `body` or `*` that the browser must re-check constantly

## ✓ TIPS & BEST PRACTICES

- Use `:where()` in resets and libraries
- Anchor `:has()` to a specific element or class to keep it fast
- Feature-test with `@supports selector(:has(*))`

## QUICK SUMMARY

- `:is()` OR, heaviest weight
- `:where()` OR, zero weight
- `:not()` exclude
- `:has()` parent / previous sibling

# Pseudo-Elements



## STYLING PARTS AND EXTRAS

**Pseudo-elements** style a part of an element (`::first-line`, `::marker`, `::placeholder`) or create a box that isn't in the HTML. `::before` and `::after` insert a child at the start or end of the element's content and only render when `content` is set (even to `""`). They're used for icons, decorative shapes, quotes, counters and overlays. Use two colons; the single-colon form works only for the four CSS 2 pseudo-elements.

## NOTES

- `::before` / `::after` don't work on replaced elements like `<img>` or `<input>`
- Generated boxes are `display: inline` by default
- Only one of each pseudo-element per element; nest wrappers for more
- `content` accepts strings, `attr()`, `counter()`, `url()` and `open-quote`

## ♿ ACCESSIBILITY

- Screen readers often announce `content` text; keep meaning in the HTML and use `content: "→" / ""` to give decorative text an empty alt
- `::selection` colors still need enough contrast

## BROWSER SUPPORT

- `::before`, `::after`, `::first-line`, `::first-letter`, `::selection`, `::placeholder`: universal
- `::marker`: Baseline 2020 (Safari supports only some properties); alt text in `content`: Baseline 2024

## CODE EXPLAINED

- → `content: "↗" / ""`: text after `/` is the alternative text (empty = decorative)
- → `attr(href)` prints an attribute value
- → `content: ""` + absolute positioning = a free extra layer
- → `::marker`: bullets and numbers of list items
- → `::backdrop`: the layer behind a modal dialog or popover

## GENERATED AND PART STYLES

```
/* decorative icon, hidden from screen readers */
.external::after {
  content: "↗" / "";
  margin-inline-start: .25em;
}

/* show the URL when printing */
@media print {
  a[href^="http"]::after { content: " (" attr(href) ")"; }
}

/* overlay layer */
.hero { position: relative; }
.hero::before {
  content: "";
  position: absolute;
  inset: 0;
  background: linear-gradient(#0000, #0009);
}

p::first-line        { font-variant: small-caps; }
.drop::first-letter  { float: left; font-size: 3em; }
li::marker           { color: #5b4bc4; content: "▸ "; }
::selection          { background: #ffe082; }
input::placeholder   { color: #767676; }
dialog::backdrop     { background: #000a; }
input[type=file]::file-selector-button {
  border-radius: 4px;
}
```

## PSEUDO-ELEMENTS

| SELECTOR | TARGETS |
|---|---|
| `::before / ::after` | Generated first / last child |
| `::first-line` | First rendered line |
| `::first-letter` | First letter of a block |
| `::marker` | List bullet or number |
| `::placeholder` | Input placeholder text |
| `::selection` | Highlighted text |
| `::backdrop` | Behind top-layer elements |
| `::file-selector-button` | File input button |

## CONTENT VALUES

| VALUE | OUTPUT |
|---|---|
| `"text"` | Literal string |
| `""` | Empty box (shapes, overlays) |
| `attr(data-x)` | Attribute value |
| `counter(n)` | Counter value |
| `url(icon.svg)` | An image |
| `open-quote` | Language-aware quote |
| `"x" / "alt"` | Content with alt text |

## ⚠ COMMON MISTAKES

- Forgetting `content`: the pseudo-element doesn't render at all
- Adding `::after` to `<img>` or `<input>`: replaced elements have no content box
- Putting meaningful text in `content` that copy, search and some screen readers miss

## ✓ TIPS & BEST PRACTICES

- Use `::before` with `inset: 0` for overlays instead of extra divs
- Give decorative generated text empty alt: `content: "★" / ""`
- Style placeholders with enough contrast (4.5:1) or don't rely on them

## QUICK SUMMARY

- `::before` / `::after` need `content`
- Two colons for pseudo-elements
- `::marker` · `::placeholder` · `::selection`
- No pseudo-elements on `<img>` / `<input>`

# Values & Data Types



## WHAT A VALUE CAN BE

Every property accepts specific **data types**, written in specs as `<length>`, `<color>`, `<percentage>` and so on. A value is built from **keywords** (`auto`, `none`), **numbers** with or without units, **strings**, and **functions** like `url()`, `calc()` or `rgb()`. On its way to the screen each value passes through stages: **specified** (from the cascade), **computed** (relative units resolved where possible), **used** (layout done) and **actual** (rounded to device pixels).

## NOTES

- `0` is the only length that may drop its unit
- Percentages resolve against something different per property (width of parent, font-size, own size…)
- Strings use single or double quotes; keywords never take quotes
- `getComputedStyle()` returns the *resolved* value, usually the used value for sizes

## ♿ ACCESSIBILITY

- Prefer relative units ( `rem` , `em` ) for text so user font-size settings scale it

## BROWSER SUPPORT

- All core data types: universal
- Typed `attr()` ( `attr(data-x type(<length>))` ): newer, Chromium first

## CODE EXPLAINED

→ `line-height: 1.5` (unitless) inherits as a ratio, unlike `1.5em`

→ `rebeccapurple` is one of 148 named colors

→ Font family names with spaces are quoted strings; generic families aren't

→ `16 / 9` is a ratio; spaces around `/` are optional

→ Computed values are what children inherit

## VALUE TYPES

```css
.box {
  width: 50%;                    /* <percentage> */
  max-width: 40rem;              /* <length> */
  line-height: 1.5;              /* <number>, unitless */
  z-index: 10;                   /* <integer> */
  opacity: 0.8;                  /* <alpha-value> */
  color: rebeccapurple;          /* <color> keyword */
  font-family: "Inter", sans-serif; /* <string>, keyword */
  background: url("bg.webp");    /* <url> */
  rotate: 15deg;                 /* <angle> */
  transition: opacity 200ms;     /* <time> */
  aspect-ratio: 16 / 9;          /* <ratio> */
  grid-template-columns: 1fr 2fr; /* <flex> */
  margin: 0;                     /* unitless zero */
  display: none;                 /* keyword */
}

/* value stages for font-size: 1.5em
   specified: 1.5em
   computed:  24px  (parent 16px)
   used:      24px
   actual:    24px  (rounded on device) */
```

## DATA TYPES

| TYPE | EXAMPLES |
|---|---|
| `<length>` | `12px`, `1.5rem`, `50vw`, `0` |
| `<percentage>` | `50%` |
| `<number>` / `<integer>` | `1.5`, `400` |
| `<color>` | `#333`, `red`, `oklch(…)` |
| `<angle>` | `45deg`, `0.5turn`, `1rad` |
| `<time>` | `200ms`, `1.5s` |
| `<string>` | `"Inter"`, `'→'` |
| `<url>` | `url(img.png)` |
| `<ratio>` | `16 / 9` |

## % RESOLVES AGAINST

| PROPERTY | BASIS |
|---|---|
| `width, margin, padding` | Containing block **width** (even top/bottom) |
| `height` | Containing block height (if definite) |
| `font-size` | Parent's font size |
| `line-height` | Element's font size |
| `translate` | The element's own size |
| `background-position` | Box minus image size |

## ⚠ COMMON MISTAKES

- Writing `margin-top: 10%` expecting a share of the height: vertical margins use the width
- Quoting keywords ( `display: "flex"` ): the declaration is invalid
- Using `line-height: 1.5em` on a parent: children inherit a fixed length, not a ratio

## ✓ TIPS & BEST PRACTICES

- Use unitless `line-height`
- Check the Computed tab in DevTools to see resolved values
- Learn what each property's percentage refers to before using `%`

## QUICK SUMMARY

- Keywords, numbers, strings, functions
- `0` needs no unit
- specified → computed → used → actual
- `%` basis varies by property

# Length Units

px · em · rem · % · ch · lh · rlh · print units

## ABSOLUTE AND RELATIVE

**Absolute** units are fixed: `px` is the CSS pixel (1/96 inch on screen, scaled by device), and `cm`, `mm`, `in`, `pt`, `pc` are tied to it (1in = 96px), so they mainly matter for print. **Relative** units scale with something: `em` with the element's font size, `rem` with the root's, `ch` with the width of `0`, and `lh` with the line height. Use `rem` for most sizes so text and spacing follow the user's preferred font size.

## NOTES

- `em` on `font-size` is relative to the *parent's* font size; elsewhere to the element's own
- Nested `em` font sizes compound: 1.2em inside 1.2em = 1.44×
- `ch` ≈ one character width; `65ch` is a comfortable line length
- `rlh` = root line height; useful for vertical rhythm

## ♿ ACCESSIBILITY

- `px` font sizes still zoom, but ignore the browser's default font-size setting; `rem` respects it
- Size media queries in `em` / `rem` so layouts adapt when users enlarge text

## BROWSER SUPPORT

- px, em, rem, %, ch, ex, cm/mm/in/pt/pc: universal
- `lh`, `rlh`: Baseline 2023
- `cap`, `ic`, `rex`, `rch`: Baseline 2023–2024; check before use

## CODE EXPLAINED

- → `font-size: 100%` keeps the user's choice instead of forcing 16px
- → `padding: .5em 1em` scales with each button's own font size
- → `1px` borders stay crisp; text sizes use rem
- → `1lh` sizes an icon to exactly one line of text
- → `pt` and `cm` make sense for print styles

## CHOOSING UNITS

```
html { font-size: 100%; }    /* user default, ~16px */

body {
  font-size: 1rem;               /* 16px */
  line-height: 1.5;
}

h1 {
  font-size: 2.5rem;             /* 40px */
  margin-block: 0 .5em;          /* .5 × 40 = 20px */
}

.btn {
  font-size: .875rem;            /* 14px */
  padding: .5em 1em;            /* scales with text */
  border: 1px solid;             /* hairlines in px */
}

.prose  { max-width: 65ch; }     /* line length */
.icon   { width: 1lh; height: 1lh; } /* match line */
.stack > * + * { margin-top: 1rlh; } /* rhythm */

/* compounding em */
.parent { font-size: 1.25em; }   /* 20px */
.parent .child { font-size: 1.25em; } /* 25px! */

@media print {
  body { font-size: 11pt; }
  .page { margin: 2cm; }
}
```

## RELATIVE UNITS

| UNIT | RELATIVE TO |
|---|---|
| `em` | Element's font size (parent's for font-size) |
| `rem` | Root `<html>` font size |
| `%` | Varies by property |
| `ch` | Width of `0` in the font |
| `ex` | x-height of the font |
| `lh` | Element's line height |
| `rlh` | Root line height |
| `cap` | Cap height of the font |

## ABSOLUTE UNITS

| UNIT | EQUALS |
|---|---|
| `px` | 1/96 in (reference pixel) |
| `in` | 96px |
| `cm` | 37.8px |
| `mm` | 3.78px |
| `pt` | 1/72 in = 1.333px |
| `pc` | 12pt = 16px |
| `Q` | quarter-millimeter |

## ⚠ COMMON MISTAKES

- Setting `html { font-size: 62.5% }` then forgetting to restore body text to `1.6rem`
- Nesting `em` font sizes and getting compounding growth
- Using `px` for all text, which ignores the user's default font size

## ✓ TIPS & BEST PRACTICES

- Default to `rem` for type and spacing, `em` for component-internal padding
- Use `ch` for readable measure, `px` only for hairlines and shadows
- Write breakpoints in `em` or `rem`, not `px`

## QUICK SUMMARY

- `rem` = root font size
- `em` = current font size
- `ch` = character width; `lh` = line height
- 1in = 96px; print units for print

# Viewport & Container Units

vw · vh · vmin · vmax · svh · lvh · dvh · cqi

## SIZING TO THE SCREEN OR CONTAINER

**Viewport units** are percentages of the viewport: `1vw` is 1% of its width and `1vh` 1% of its height. On mobile, browser toolbars grow and shrink, so `vh` was ambiguous. The **small** (`svh`), **large** (`lvh`) and **dynamic** (`dvh`) variants fix that. **Container query units** (`cqi`, `cqb`, `cqw`, `cqh`) are the same idea relative to the nearest **size container**, which makes components scale with their slot instead of the window.

## NOTES

- `100vw` includes the vertical scrollbar, so it can cause horizontal scroll
- `svh` = toolbars shown (smallest); `lvh` = toolbars hidden (largest); `dvh` follows them live
- `vi` / `vb` follow the writing mode's inline/block axes
- Without a container ancestor, `cq*` units fall back to small viewport units

## ♿ ACCESSIBILITY

- Pure `vw` font sizes don't grow with zoom; always mix with `rem` in `clamp()`
- `100vh` layouts can hide content behind mobile toolbars; use `100dvh` or `min-height`

## BROWSER SUPPORT

- `vw`, `vh`, `vmin`, `vmax`: universal
- `svh` / `lvh` / `dvh` and friends: Baseline 2022
- Container query units: Baseline 2023

## CODE EXPLAINED

→ Declare `vh` first as a fallback, then the newer unit

→ `100svh` fits even when the mobile toolbar is visible

→ `clamp()` with `rem + vw` scales but still zooms

→ `container-type: inline-size` makes the slot a size container

→ `5cqi` = 5% of the container's inline size

## VIEWPORT AND CONTAINER SIZING

```css
/* full-screen hero that respects mobile toolbars */
.hero {
  min-height: 100vh;        /* fallback */
  min-height: 100svh;       /* never hidden by toolbars */
}

/* app shell that tracks toolbars as they move */
.app { height: 100dvh; }

/* fluid type: never pure vw */
h1 { font-size: clamp(2rem, 1rem + 4vw, 4rem); }

/* square based on the smaller side */
.square { width: 50vmin; aspect-ratio: 1; }

/* container query units */
.card-slot { container-type: inline-size; }

.card h2 {
  font-size: clamp(1rem, 5cqi, 2rem); /* 5% of slot */
}
.card {
  padding: 4cqi;
  gap: 2cqi;
}

/* avoid: horizontal scroll from scrollbar */
.full-bleed { width: 100vw; }   /* use 100% */
```

## VIEWPORT UNITS

| UNIT | 1 UNIT = |
|---|---|
| `vw` / `vh` | 1% of viewport width / height |
| `vmin` / `vmax` | 1% of smaller / larger side |
| `vi` / `vb` | 1% of inline / block axis |
| `svh` / `svw` | Small viewport (toolbars shown) |
| `lvh` / `lvw` | Large viewport (toolbars hidden) |
| `dvh` / `dvw` | Dynamic: current size |

## CONTAINER UNITS

| UNIT | 1 UNIT = |
|---|---|
| `cqw` / `cqh` | 1% of container width / height |
| `cqi` / `cqb` | 1% of container inline / block size |
| `cqmin` / `cqmax` | 1% of smaller / larger |

## ⚠ COMMON MISTAKES

- `width: 100vw` on a page with a scrollbar: adds horizontal scroll
- `height: 100vh` on mobile: bottom content sits under the toolbar
- `font-size: 5vw`: text no longer responds to zoom (fails WCAG 1.4.4)

## ✓ TIPS & BEST PRACTICES

- Use `svh` for heroes and `dvh` for app shells
- Prefer `cqi` over `vw` inside components
- Combine viewport units with `rem` inside `clamp()`

## QUICK SUMMARY

- `vw` / `vh` = viewport %
- `svh` · `lvh` · `dvh` for mobile
- `cqi` = container inline %
- Never size text in `vw` alone

# Math Functions

calc() · min() · max() · clamp() · fluid type · round() · mod() · abs()

## COMPUTING VALUES

**`calc()`** does arithmetic with mixed units (`100% - 2rem`). **`min()`** and **`max()`** pick the smaller or larger of several values, and **`clamp(MIN, PREFERRED, MAX)`** keeps a fluid value between limits. That's the basis of **fluid typography** and spacing without media queries. Newer stepped functions (`round()`, `mod()`, `rem()`) and sign functions (`abs()`, `sign()`) round out the set.

## NOTES

- `+` and `-` need spaces around them: `calc(100% - 1rem)`
- Math functions nest and accept `var()`: `calc(var(--gap) * 2)`
- `min()` acts as a *maximum* size limit, `max()` as a *minimum*, which trips everyone up
- Trig functions (`sin()`, `cos()`, `atan2()`) also exist, for circular layouts

## ♿ ACCESSIBILITY

- Fluid type formulas must include a `rem` term so text still scales with zoom (WCAG 1.4.4)

## BROWSER SUPPORT

- `calc()`: universal; `min()`, `max()`, `clamp()`: Baseline 2020
- `round()`, `mod()`, `rem()`, trig functions: Baseline 2023–2024
- `abs()`, `sign()`: newer and less widely supported; check first

## CODE EXPLAINED

→ `calc()` mixes `%` and `rem` at layout time

→ `min(100% - 2rem, 70rem)`: a centered column with side gutters

→ `max(44px, …)`: never smaller than a touch target

→ `clamp()`: min, preferred (fluid), max

→ `round(up, value, 8px)` rounds to the next multiple of 8px

## CALC, MIN, MAX, CLAMP

```
.sidebar-layout {
  width: calc(100% - 2 * var(--gutter));
}

/* responsive width without media queries */
.container {
  width: min(100% - 2rem, 70rem);  /* ≤ 70rem */
}

/* at least 44px tall, more if needed */
.tap { min-height: max(44px, 2.75rem); }

/* fluid type: 1.25rem → 2rem between widths */
h2 {
  font-size: clamp(1.25rem, 0.9rem + 1.5vw, 2rem);
}

/* fluid spacing */
.section {
  padding-block: clamp(2rem, 6vw, 6rem);
}

/* snap to an 8px grid */
.box { height: round(up, 100% / 3, 8px); }

/* 0 or 1 from an index variable */
.item { --odd: mod(var(--i), 2); }

/* full-bleed inside a centered column */
.bleed {
  margin-inline: calc(50% - 50vw);
}
```

## FUNCTIONS

| FUNCTION | RETURNS |
|---|---|
| `calc(a + b)` | Result of the expression |
| `min(a, b, …)` | Smallest value |
| `max(a, b, …)` | Largest value |
| `clamp(min, val, max)` | `val` limited to the range |
| `round(strategy, a, step)` | `a` rounded to `step` |
| `mod(a, b) / rem(a, b)` | Modulus / remainder |
| `abs(a) / sign(a)` | Absolute value / -1, 0, 1 |
| `sin() cos() tan() atan2()` | Trigonometry |

## FLUID TYPE RECIPE

| STEP | VALUE |
|---|---|
| `Min size` | 1.25rem at 20rem viewport |
| `Max size` | 2rem at 80rem viewport |
| `Slope` | (2 - 1.25) / (80 - 20) = 0.0125 → 1.25vw |
| `Intercept` | 1.25 - 20 × 0.0125 = 1rem |
| `Result` | `clamp(1.25rem, 1rem + 1.25vw, 2rem)` |

## ⚠ COMMON MISTAKES

- `calc(100%-1rem)` without spaces: invalid, because `-1rem` parses as a negative number
- Using `min()` to set a minimum: `min()` picks the smaller value, so it caps
- Fluid type with only `vw`: no zoom scaling

## ✓ TIPS & BEST PRACTICES

- Generate fluid `clamp()` values with utopia.fyi
- Use `min(100% - gutter, max)` instead of width + max-width + padding
- Store results in custom properties to reuse them

## QUICK SUMMARY

- `calc()` mixes units
- `min()` caps, `max()` floors
- `clamp(min, fluid, max)`
- Spaces around `+` and `-`

# Custom Properties

--name · var() · fallback · :root · inheritance & scope · invalid at computed-value time

## CSS VARIABLES

**Custom properties** are author-defined properties whose names start with --. Read them with **`var(--name, fallback)`**. Unlike preprocessor variables they're live: they **cascade and inherit**, can change per element, in media queries or from JavaScript, and every `var()` updates. Define global **design tokens** on `:root`, then override them in a narrower scope to theme a component.

## NOTES

- Names are case-sensitive: `--Brand` ≠ `--brand`
- Values are stored as raw tokens and only checked when used
- A `var()` that resolves to an invalid value makes the property **invalid at computed-value time**: it becomes `unset`, not the earlier declaration
- `var()` can't build property names or selectors; only values

## ♿ ACCESSIBILITY

- Keep color tokens paired ( `--text` / `--surface` ) so every theme keeps contrast
- Expose `--focus-ring` as a token so components can't forget focus styles

## BROWSER SUPPORT

- Custom properties and `var()` : Baseline Widely available (since 2017)
- Typed, animatable custom properties via `@property` : see sheet 24

## CODE EXPLAINED

→ `:root` makes tokens available everywhere by inheritance

→ `var(--btn-text, white)` : fallback when the variable isn't set

→ `.btn--danger` changes the variable, not the rule

→ Changing a token in a media query updates every use

→ An invalid `var()` result doesn't fall back to the earlier `width`

## TOKENS AND THEMES

```
:root {
  --brand: #5b4bc4;
  --text: #1d1b26;
  --surface: #ffffff;
  --space: 1rem;
  --radius: 8px;
}

.btn {
  background: var(--brand);
  padding: calc(var(--space) / 2) var(--space);
  border-radius: var(--radius);
  color: var(--btn-text, white);       /* fallback */
}

/* component-level override */
.btn--danger { --brand: #c62828; }

/* theme switch */
[data-theme="dark"] {
  --text: #ece9f6;
  --surface: #17151f;
}

/* responsive token */
@media (width >= 60rem) {
  :root { --space: 1.5rem; }
}

/* invalid at computed-value time */
.box { --size: red; width: 100px; width: var(--size); }
/* width → auto, NOT 100px */
```

## SYNTAX

| FORM | MEANING |
|---|---|
| `--name: value` | Declare |
| `var(--name)` | Use |
| `var(--a, 10px)` | With fallback |
| `var(--a, var(--b, 0))` | Fallback chain |
| `--empty: ;` | Empty (valid) |
| `calc(var(--n) * 1px)` | Number to length |

## FROM JAVASCRIPT

| CODE | DOES |
|---|---|
| `el.style.setProperty('--x', '10px')` | Set on an element |
| `getComputedStyle(el).getPropertyValue('--x')` | Read |
| `document.documentElement.style…` | Set globally |
| `el.style.removeProperty('--x')` | Remove |

## ⚠ COMMON MISTAKES

- Writing `var(--space)px` : produces `16 px` tokens, not `16px` ; use `calc(var(--n) * 1px)`
- Expecting an invalid `var()` to fall back to the previous declaration
- Defining tokens on `body` and using them in `html` -level rules

## ✓ TIPS & BEST PRACTICES

- Keep raw tokens ( `--purple-600` ) and semantic tokens ( `--brand` ) separate
- Override tokens per component instead of rewriting rules
- Store unitless numbers and multiply in `calc()` when JS updates them

## QUICK SUMMARY

- `--name: value` / `var(--name)`
- Cascades and inherits
- Fallback: `var(--x, default)`
- Invalid result → `unset`

# Custom Properties: Advanced

@property · syntax · inherits · initial-value · animating variables · theming · setProperty()

## TYPED CUSTOM PROPERTIES

**@property** registers a custom property with a **type** (`syntax`), an `initial-value` and whether it `inherits`. Registered properties are validated, so an invalid value falls back to the initial value instead of breaking the property, and because the browser knows the type it can **interpolate** them. That makes gradients, angles and counters animatable, which raw `--vars` (treated as strings) can't do.

## NOTES

- `syntax` uses type names: `'<length>'`, `'<color>'`, `'<number>'`, `'<angle>'`, `'*'`
- Combine: `'<length> | <percentage>'`; lists: `'<length>+'`
- `inherits: false` keeps a value local to the element that sets it
- JS equivalent: `CSS.registerProperty({ name, syntax, inherits, initialValue })`

## ♿ ACCESSIBILITY

- Wrap property-driven animations in `prefers-reduced-motion: no-preference`

## BROWSER SUPPORT

- `@property`: Baseline 2024
- Unregistered variables still work everywhere; animations just jump instead of easing

## CODE EXPLAINED

→ `syntax: '<angle>'` lets the browser interpolate `0deg → 360deg`

→ Keyframes animate the variable; the gradient re-renders each frame

→ `transition: --stop` works only because `--stop` is registered

→ An invalid value ( `red` for a length) uses `initial-value`

→ `inherits: false` stops the value leaking into children

## REGISTER, THEN ANIMATE

```
@property --angle {
  syntax: '<angle>';
  inherits: false;
  initial-value: 0deg;
}

/* spinning conic border */
.ring {
  background: conic-gradient(
    from var(--angle), #5b4bc4, #00bcd4, #5b4bc4);
  animation: spin 3s linear infinite;
}
@keyframes spin { to { --angle: 360deg; } }

@property --stop {
  syntax: '<percentage>';
  inherits: false;
  initial-value: 0%;
}

/* animated gradient fill on hover */
.bar {
  background: linear-gradient(90deg,
    #5b4bc4 var(--stop), #ddd var(--stop));
  transition: --stop .4s;
}
.bar:hover { --stop: 100%; }

/* type safety */
@property --gap {
  syntax: '<length>';
  inherits: true;
  initial-value: 1rem;
}
.grid { --gap: red; gap: var(--gap); } /* → 1rem */
```

## @PROPERTY DESCRIPTORS

| DESCRIPTOR | PURPOSE |
|---|---|
| `syntax` | Allowed type(s); `'*'` = any |
| `inherits` | `true` or `false` (required) |
| `initial-value` | Default; required unless syntax is `'*'` |

## SYNTAX TYPES

| TYPE | EXAMPLE VALUE |
|---|---|
| `<length>` | `10px`, `2rem` |
| `<percentage>` | `50%` |
| `<length-percentage>` | `10px` or `50%` |
| `<number>` / `<integer>` | `1.5` / `3` |
| `<color>` | `#5b4bc4` |
| `<angle>` | `45deg` |
| `<time>` | `200ms` |
| `<custom-ident>` | `left` |

## ⚠ COMMON MISTAKES

- Forgetting `inherits` or `initial-value`: the whole `@property` rule is ignored
- Transitioning an unregistered variable: it switches instantly at the midpoint
- Registering a property with `syntax: '*'` and expecting animation

## ✓ TIPS & BEST PRACTICES

- Register any variable you plan to animate or transition
- Use `inherits: false` for per-element animation state
- Put `@property` rules at the top level, near your tokens

## QUICK SUMMARY

- `@property --x { syntax; inherits; initial-value }`
- Registered → typed, validated
- Typed → animatable
- JS: `CSS.registerProperty()`

# Color Syntax

named colors · hex · rgb() · hsl() · alpha · opacity · currentColor · transparent

## WRITING COLORS

CSS colors can be **named** (`tomato`), **hex** (`#ff6347`), **`rgb()`** or **`hsl()`**, plus the modern spaces on the next sheet. Modern syntax separates channels with **spaces** and alpha with a **slash**: `rgb(255 99 71 / 50%)`. **`currentColor`** reuses the element's `color` value, which keeps borders, icons and SVG fills in sync. Color alpha only affects that one color; `opacity` fades the whole element and its children.

## NOTES

- Hex shorthands: `#f60` = `#ff6600` ; with alpha: `#ff660080` or `#f608`
- `rgba()` / `hsla()` are legacy aliases; `rgb()` takes alpha now
- `hsl(h s l)` : hue in degrees, saturation and lightness in %
- `transparent` = `rgb(0 0 0 / 0)`

## ♿ ACCESSIBILITY

- Text needs **4.5:1** contrast (3:1 for large text and UI parts) against its background (WCAG 1.4.3, 1.4.11)
- Don't use color alone to convey meaning (WCAG 1.4.1)

## BROWSER SUPPORT

- Named, hex, `rgb()` , `hsl()` , `currentColor` : universal
- Space-separated syntax and 8-digit hex: Baseline Widely available

## CODE EXPLAINED

→ Modern `rgb()` : spaces between channels, `/` before alpha

→ Alpha can be `0–1` or a percentage

→ `currentColor` makes the border match the text automatically

→ `opacity` fades everything, including child text

→ Changing HSL lightness is a quick hover shade (but see oklch)

## COLOR FORMATS

```
.a { color: rebeccapurple; }          /* named */
.b { color: #5b4bc4; }                /* hex */
.c { color: #5b4bc480; }              /* hex + alpha */
.d { color: rgb(91 75 196); }         /* modern rgb */
.e { color: rgb(91 75 196 / 50%); }   /* + alpha */
.f { color: hsl(248 51% 53%); }       /* hue sat light */
.g { color: hsl(248 51% 53% / .5); }

/* legacy comma syntax still valid */
.h { color: rgba(91, 75, 196, 0.5); }

/* currentColor: follow the text color */
.icon-btn {
  color: #5b4bc4;
  border: 2px solid currentColor;
}
.icon-btn svg { fill: currentColor; }

/* alpha color vs opacity */
.overlay-a { background: rgb(0 0 0 / 60%); } /* text ok */
.overlay-b { background: black; opacity: .6; } /* fades */

/* tint via hsl lightness */
.btn       { background: hsl(248 51% 53%); }
.btn:hover { background: hsl(248 51% 45%); }
```

## FORMATS

| SYNTAX | EXAMPLE |
|---|---|
| Named | `tomato`, `rebeccapurple` |
| Hex | `#5b4bc4`, `#5b4bc4cc` |
| `rgb()` | `rgb(91 75 196 / .8)` |
| `hsl()` | `hsl(248 51% 53%)` |
| `hwb()` | `hwb(248 29% 23%)` |
| `currentColor` | Element's `color` |
| `transparent` | Fully clear |

## HUE WHEEL (HSL)

| HUE | COLOR |
|---|---|
| `0 / 360` | Red |
| `60` | Yellow |
| `120` | Green |
| `180` | Cyan |
| `240` | Blue |
| `300` | Magenta |

## ⚠ COMMON MISTAKES

- Using `opacity` to fade a background and fading the text with it
- Mixing comma and space syntax in one function ( `rgb(0, 0 0)` )
- Assuming equal HSL lightness looks equally light: yellow and blue at 50% differ a lot

## ✓ TIPS & BEST PRACTICES

- Use `currentColor` for borders, icons and SVG fills
- Check contrast in DevTools' color picker
- Prefer oklch for palettes; HSL lightness isn't perceptual

## QUICK SUMMARY

- `rgb(r g b / a)` · `hsl(h s l / a)`
- `#rrggbbaa` hex with alpha
- `currentColor` = text color
- Alpha color ≠ `opacity`

# Modern Color Spaces

oklch() · oklab() · lch() · lab() · color() · display-p3 · gamut · @media (color-gamut)

## PERCEPTUAL AND WIDE-GAMUT COLOR

**`oklch(L C H)`** describes color by **lightness** (0–100%), **chroma** (colorfulness, about 0–0.4) and **hue** (degrees). It's **perceptually uniform**: the same L looks equally light across hues, so palettes and hover shades come out even. It also reaches **wide-gamut** colors (Display P3) that `rgb()` can't. **`color()`** names a space explicitly, like `color(display-p3 1 0 0)`.

## NOTES

- oklch hue numbers differ from hsl: oklch 0 is pinkish red, 145 green, 265 blue
- Colors outside a screen's gamut are mapped to the nearest displayable color
- `lab()` / `lch()` (CIE) are older perceptual spaces; `oklab` / `oklch` fix their blue hue shift
- `none` can stand in for a missing channel: `oklch(70% 0 none)`

## ♿ ACCESSIBILITY

- Equal oklch lightness gives similar contrast across hues, which makes accessible palettes easier
- Wide-gamut colors still need contrast checked in sRGB fallback too

## BROWSER SUPPORT

- `oklch()`, `oklab()`, `lab()`, `lch()`, `color()`: Baseline 2023
- Add an sRGB hex fallback for very old browsers

## CODE EXPLAINED

→ Only L changes in the brand scale, so steps look even

→ Success and danger share L 60%, so they look equally strong

→ The hex line is the fallback for browsers that skip oklch

→ `color(display-p3 …)`: channels 0–1 in the P3 space

→ `@media (color-gamut: p3)` targets wide-gamut displays

## USING OKLCH

```
:root {
  /* one hue, even steps of lightness */
  --brand-300: oklch(80% 0.10 285);
  --brand-500: oklch(60% 0.18 285);
  --brand-700: oklch(42% 0.16 285);

  /* same lightness, different hue */
  --success:   oklch(60% 0.15 145);
  --danger:    oklch(60% 0.19 25);
}

.btn       { background: var(--brand-500); }
.btn:hover { background: oklch(52% 0.18 285); }

/* fallback first, then wide gamut */
.hot {
  background: #ff2e00;
  background: oklch(65% 0.30 35);
}

/* explicit color spaces */
.p3   { color: color(display-p3 1 0.2 0.1); }
.srgb { color: color(srgb 0.36 0.29 0.77); }

/* only on wide-gamut screens */
@media (color-gamut: p3) {
  .hero { background: oklch(70% 0.28 330); }
}
```

## FUNCTIONS

| FUNCTION | CHANNELS |
|---|---|
| `oklch()` | L 0–100%, C ~0–0.4, H deg |
| `oklab()` | L, a (green↔red), b (blue↔yellow) |
| `lch()` | L 0–100, C ~0–150, H deg |
| `lab()` | L, a, b (CIE) |
| `color()` | Space name + channels 0–1 |
| `hwb()` | Hue, whiteness, blackness |

## COLOR() SPACES

| SPACE | NOTES |
|---|---|
| `srgb` | Standard web gamut |
| `srgb-linear` | Linear light sRGB |
| `display-p3` | ~50% more colors; modern Apple, many phones |
| `rec2020` | Even wider; HDR content |
| `xyz` | Device-independent reference |

## ⚠ COMMON MISTAKES

- Copying hsl hue numbers into oklch: the hue scales differ
- Very high chroma at low or high lightness: falls outside the gamut and gets clipped
- No fallback in projects that support old browsers

## ✓ TIPS & BEST PRACTICES

- Build palettes by stepping L in oklch with fixed C and H
- Use oklch.com to pick values and see gamut limits
- Keep chroma under ~0.2 for UI colors that must work in sRGB

## QUICK SUMMARY

- `oklch(L C H)` perceptual
- Even lightness across hues
- `color(display-p3 …)` wide gamut
- Fallback hex first

# Color Functions & Schemes

color-mix() · relative color (from) · light-dark() · color-scheme · accent-color · system colors

## DERIVING COLORS

**`color-mix(in oklch, A 30%, B)`** blends two colors in a chosen space. **Relative color syntax** builds a new color from an existing one: `oklch(from var(--brand) calc(l - 0.1) c h)` darkens a token without a second hard-coded value. **`color-scheme`** tells the browser which themes a page supports, so form controls and scrollbars match, and **`light-dark(A, B)`** picks a value based on the active scheme.

## NOTES

- Mix percentages default to 50/50 and normalize if they don't add to 100%
- In relative syntax, channel names ( `l` , `c` , `h` , `r` , `g` , `b` , `alpha` ) are numbers you can use in `calc()`
- `light-dark()` only works when `color-scheme` includes both `light` and `dark`
- `accent-color` tints checkboxes, radios, range and progress

## ♿ ACCESSIBILITY

- `color-scheme: light dark` makes native controls readable in dark mode
- Use system colors ( `Canvas` , `CanvasText` , `LinkText` ) inside `forced-colors` styles

## BROWSER SUPPORT

- `color-mix()` : Baseline 2023; `color-scheme` , `accent-color` : Baseline 2022
- Relative color syntax, `light-dark()` : Baseline 2024

## CODE EXPLAINED

→ `color-scheme: light dark` enables `light-dark()` and dark native UI

→ `color-mix(…, black 15%)` : a 15% darker shade

→ Mixing with `transparent` makes a translucent tint

→ `from var(--brand)` exposes `l c h` as numbers for `calc()`

→ `accent-color` restyles checkboxes and radios in one line

## MIXING AND THEMING

```
:root {
  color-scheme: light dark;            /* both themes */
  --brand: oklch(58% 0.18 285);

  --surface: light-dark(#ffffff, #16141d);
  --text:    light-dark(#1d1b26, #ece9f6);
}

body { background: var(--surface); color: var(--text); }

/* mix: tints and shades */
.btn         { background: var(--brand); }
.btn:hover {
  background: color-mix(in oklch, var(--brand), black 15%);
}
.btn-soft {
  background: color-mix(in oklch, var(--brand) 15%,
                        transparent);
}

/* relative color syntax */
.btn:active {
  background: oklch(from var(--brand) calc(l - .12) c h);
}
.muted       { color: rgb(from var(--text) r g b / 60%); }
.complement {
  color: oklch(from var(--brand) l c calc(h + 180));
}

/* native controls */
form { accent-color: var(--brand); }

/* user forced a palette (Windows contrast) */
@media (forced-colors: active) {
  .btn { border: 1px solid ButtonText; }
}
```

## FUNCTIONS

| FUNCTION | DOES |
|---|---|
| `color-mix(in s, a p%, b)` | Blend two colors in space s |
| `oklch(from c l c h)` | Modify channels of c |
| `rgb(from c r g b / a)` | Same in rgb |
| `light-dark(a, b)` | a in light, b in dark |
| `contrast-color(c)` | Black or white for contrast (newer) |

## SYSTEM COLORS

| KEYWORD | USED FOR |
|---|---|
| `Canvas / CanvasText` | Page background / text |
| `LinkText / VisitedText` | Links |
| `ButtonFace / ButtonText` | Buttons |
| `Field / FieldText` | Inputs |
| `Highlight / HighlightText` | Selection |
| `GrayText` | Disabled text |
| `AccentColor` | OS accent (limited) |

## ⚠ COMMON MISTAKES

- Using `light-dark()` without setting `color-scheme` : always returns the light value
- Mixing in `srgb` and getting muddy midpoints; mix in `oklch` or `oklab`
- Hard-coding hover shades that drift from the token when the brand changes

## ✓ TIPS & BEST PRACTICES

- Derive hover, active and soft variants from one token
- Set `color-scheme` on `:root` in every project
- Test with Windows contrast themes via DevTools' forced-colors emulation

## QUICK SUMMARY

- `color-mix(in oklch, a, b)`
- `oklch(from x calc(l - .1) c h)`
- `light-dark()` + `color-scheme`
- `accent-color` for controls

# The Box Model

content · padding · border · margin · box-sizing · content-box vs border-box · DevTools box view

## FOUR NESTED BOXES

Every element renders as a rectangle made of four areas: **content**, **padding** (inside the border, takes the background), **border**, and **margin** (outside, always transparent). By default (`box-sizing: content-box`) `width` sets only the content, so padding and border are *added* on top. **`border-box`** makes `width` include padding and border, which is what almost everyone wants, and is why resets set it globally.

## NOTES

- Margins are transparent and can be negative; padding can't
- Backgrounds paint under padding and border by default ( `background-clip: border-box` )
- Inline elements ignore `width` , `height` and vertical margins
- `outline` and `box-shadow` don't take up space in the layout

## ♿ ACCESSIBILITY

- Padding enlarges the clickable area of links and buttons; aim for at least 24×24px targets (WCAG 2.5.8)

## BROWSER SUPPORT

- Box model and `box-sizing` : universal

## CODE EXPLAINED

- → `content-box` : rendered width = width + padding + border
- → `border-box` : rendered width = width
- → Margins never count toward `width`
- → `inherit` version lets a component switch back to `content-box`
- → `inline-block` gives inline elements full box behavior

## CONTENT-BOX VS BORDER-BOX

```
/* width: 200 + 2×20 padding + 2×5 border = 250px */
.content-box {
  box-sizing: content-box;     /* default */
  width: 200px;
  padding: 20px;
  border: 5px solid;
  margin: 10px;                /* outside, not in width */
}

/* width stays 200px; content shrinks to 150px */
.border-box {
  box-sizing: border-box;
  width: 200px;
  padding: 20px;
  border: 5px solid;
}

/* global reset (inheritable version) */
html { box-sizing: border-box; }
*, *::before, *::after { box-sizing: inherit; }

/* inline elements: vertical margins ignored */
span.tag {
  margin-block: 20px;          /* no effect */
  padding-block: 4px;          /* paints, no layout push */
  display: inline-block;       /* now both apply */
}
```

## THE AREAS

| AREA | BACKGROUND? | IN WIDTH? |
|---|---|---|
| `Content` | Yes | Always |
| `Padding` | Yes | border-box only |
| `Border` | Yes (under it) | border-box only |
| `Margin` | No | Never |

## BOX BEHAVIORS

| DISPLAY | WIDTH/HEIGHT | VERTICAL MARGIN |
|---|---|---|
| `block` | Yes | Yes (can collapse) |
| `inline` | No | No |
| `inline-block` | Yes | Yes |
| `flex / grid item` | Yes | Yes (no collapse) |

## ⚠ COMMON MISTAKES

- Setting `width: 100%` plus padding with `content-box` : the box overflows its parent
- Trying to size `<span>` s without changing `display`
- Using margin for the inner space of a clickable element (margin isn't clickable)

## ✓ TIPS & BEST PRACTICES

- Set `border-box` globally in your reset
- Use the DevTools box-model diagram to see every area
- Use padding for internal spacing, `gap` or margin for spacing between boxes

## QUICK SUMMARY

- content → padding → border → margin
- `border-box` : width includes padding + border
- Margins are outside and transparent
- Inline boxes ignore width/height

# Margin & Padding

1–4 value shorthand · auto margins · negative margins · margin collapsing · percentage padding

## SPACE AROUND AND INSIDE

`margin` and `padding` take **1 to 4 values** going clockwise from the top. `margin: auto` absorbs free space, which centers block elements horizontally and pushes flex items apart. Vertical margins between blocks in normal flow **collapse**: adjacent margins merge into the larger one instead of adding up. Collapsing never happens in flex or grid containers, and is blocked by padding, borders or a new formatting context.

## NOTES

- Clockwise order: **T**op, **R**ight, **B**ottom, **L**eft ("TRouBLe")
- Percentages for margin and padding use the containing block's **width**, even vertically
- Negative margins pull elements over neighbors or reduce space
- A parent's top margin collapses with its first child's when nothing separates them

## ♿ ACCESSIBILITY

- Don't use large negative margins that push text off-screen or under other content

## BROWSER SUPPORT

- Margin, padding, `auto` margins: universal
- Logical versions ( `margin-inline` , `padding-block` ): Baseline 2021; see sheet 31

## CODE EXPLAINED

→ Missing values copy their opposite side

→ `margin-inline: auto` centers a block with a set width

→ `margin-left: auto` in flex eats all free space before `.login`

→ Adjacent vertical margins collapse to the larger one

→ `flow-root` creates a formatting context and stops collapse

## SHORTHANDS AND COLLAPSING

```
.a { margin: 1rem; }                  /* all four */
.b { margin: 1rem 2rem; }             /* T/B  L/R */
.c { margin: 1rem 2rem 3rem; }        /* T  L/R  B */
.d { margin: 1rem 2rem 3rem 4rem; }   /* T R B L */

/* center a block */
.page { max-width: 60rem; margin-inline: auto; }

/* push the last flex item right */
.nav { display: flex; }
.nav .login { margin-left: auto; }

/* collapsing: gap is 32px, not 52px */
h2 { margin-bottom: 32px; }
p  { margin-top: 20px; }

/* stop parent-child collapse */
.card { padding-top: 1px; }           /* or */
.card { display: flow-root; }

/* negative margin: overlap an avatar */
.avatar { margin-top: -2rem; }

/* intrinsic ratio via % padding (old trick) */
.ratio { padding-top: 56.25%; }       /* 16:9 */
/* modern: aspect-ratio: 16 / 9; */
```

## SHORTHAND VALUES

| COUNT | APPLIES TO |
|---|---|
| 1 | All sides |
| 2 | Vertical, horizontal |
| 3 | Top, horizontal, bottom |
| 4 | Top, right, bottom, left |

## MARGIN COLLAPSE

| SITUATION | COLLAPSES? |
|---|---|
| Adjacent siblings (vertical) | Yes |
| Parent and first/last child | Yes, without padding/border |
| Empty block's own top/bottom | Yes |
| Horizontal margins | Never |
| Flex or grid items | Never |
| Floats, absolute, inline-block | Never |

## ⚠ COMMON MISTAKES

- Expecting `margin-top: 10%` to use the parent's height
- Being surprised a child's top margin moves the parent down (collapse through the parent)
- Using `margin: auto` vertically in normal flow (only works in flex/grid or absolute positioning)

## ✓ TIPS & BEST PRACTICES

- Space siblings with `gap` in flex/grid instead of margins
- Pick one direction for flow margins (e.g. only `margin-block-start` ) to avoid collapse puzzles
- Use `aspect-ratio` instead of the padding-percentage trick

## QUICK SUMMARY

- 1–4 values, clockwise from top
- `auto` absorbs free space
- Vertical margins collapse in normal flow
- `%` padding/margin = parent width

# Sizing

width · height · min-/max- · min-content · max-content · fit-content · aspect-ratio

## HOW BIG IS A BOX?

Block boxes take the full available width and are as tall as their content. `width` and `height` set sizes explicitly; **`min-*`** and **`max-*`** set limits that always win (`min` beats `max` if they conflict). **Intrinsic keywords** size from content: `min-content` is the narrowest a box can get without overflow, `max-content` is its widest unwrapped size, and `fit-content` is `max-content` capped at the available space. `aspect-ratio` sets one dimension from the other.

## NOTES

- Percentage heights only work if the parent's height is definite
- `max-width: 100%` keeps images and media from overflowing
- `aspect-ratio` is ignored if both width and height are set
- `height: auto` means "fit content"; it can't be transitioned without newer `interpolate-size`

## ♿ ACCESSIBILITY

- Avoid fixed `height` on text containers; zoomed or translated text overflows (WCAG 1.4.10 reflow)
- Use `min-height` instead so boxes grow with content

## BROWSER SUPPORT

- width/height/min/max and intrinsic keywords: universal
- `aspect-ratio` : Baseline 2021

## CODE EXPLAINED

- → `max-width` + `margin-inline: auto` = centered, shrinking column
- → `height: auto` with `max-width: 100%` keeps image ratios
- → `aspect-ratio: 1` : square from width alone
- → `fit-content` shrink-wraps a block like inline-block would
- → `min-width` wins when it conflicts with `max-width`

## EXPLICIT AND INTRINSIC SIZES

```
.container {
  width: 100%;
  max-width: 70rem;          /* cap */
  margin-inline: auto;
}

.panel {
  min-height: 12rem;         /* grows with content */
}

img, video {
  max-width: 100%;
  height: auto;              /* keep proportions */
}

.video  { aspect-ratio: 16 / 9; width: 100%; }
.avatar { aspect-ratio: 1; width: 3rem; object-fit: cover; }

/* intrinsic sizes */
.tag      { width: fit-content; }   /* shrink-wrap */
.caption  { width: min-content; }   /* widest word */
.no-wrap  { width: max-content; }   /* one line */

/* min wins over max */
.box { max-width: 200px; min-width: 300px; }  /* 300px */

/* full height app */
html, body { height: 100%; }
.app { min-height: 100dvh; }
```

## SIZE KEYWORDS

| VALUE | SIZE |
|---|---|
| `auto` | Fill (block width) or fit content (height) |
| `min-content` | Smallest without overflow |
| `max-content` | Content on one line |
| `fit-content` | max-content, but ≤ available |
| `fit-content(20rem)` | Same, capped at 20rem (grid) |
| `stretch` | Fill available space (newer) |

## PRECEDENCE

| RULE | RESULT |
|---|---|
| `min-width > max-width` | min-width wins |
| `max-width < width` | max-width wins |
| `width and height and aspect-ratio` | ratio ignored |
| `% height, parent auto` | Treated as auto |

## ⚠ COMMON MISTAKES

- Setting `height` on text boxes and clipping content when text grows
- `height: 100%` inside a parent with no set height: does nothing
- Forgetting `height: auto` on images with HTML `width` / `height` attributes

## ✓ TIPS & BEST PRACTICES

- Prefer `min-height` and `max-width` over fixed sizes
- Keep HTML `width` / `height` on images (for layout shift) and add `height: auto` in CSS
- Reach for `fit-content` before `display: inline-block`

## QUICK SUMMARY

- `min-*` / `max-*` limits win
- `min-content` · `max-content` · `fit-content`
- `aspect-ratio: 16 / 9`
- `%` height needs a definite parent

# Logical Properties

inline vs block axis · margin-inline · padding-block · inset-inline · inline-size · writing-mode · direction

## FLOW-RELATIVE DIRECTIONS

**Logical properties** describe sides by text flow instead of screen direction. The **inline** axis runs along a line of text (left→right in English, right→left in Arabic), and the **block** axis is the direction lines stack (top→bottom). `margin-inline-start` is the left margin in English and the right margin in Hebrew, with no extra CSS. They also adapt to vertical writing modes.

## NOTES

- `inline-size` / `block-size` replace `width` / `height`
- Shorthands: `margin-inline: 1rem 2rem` = start, end
- `inset-inline-start` replaces `left` for positioning
- Logical values exist too: `text-align: start`, `float: inline-start`

## ♿ ACCESSIBILITY

- Logical properties keep layouts correct for right-to-left readers without a separate stylesheet
- Set `dir` in the HTML ( `<html dir="rtl">` ); CSS `direction` alone doesn't change semantics

## BROWSER SUPPORT

- Logical margin/padding/inset/size/border: Baseline 2021
- Logical `border-radius` : Baseline 2021; `float: inline-start` : Baseline 2023

## CODE EXPLAINED

- → `inline-start` = left in LTR, right in RTL
- → `text-align: start` follows `dir` automatically
- → `margin-block: a b` = block-start, block-end
- → `inline-size` = width in horizontal writing modes
- → In `vertical-rl`, inline runs top to bottom and blocks stack right to left

## PHYSICAL VS LOGICAL

```
/* physical: breaks in RTL */
.card {
  margin-left: 1rem;
  padding-right: 2rem;
  border-left: 4px solid;
  text-align: left;
}

/* logical: works in any direction */
.card {
  margin-inline-start: 1rem;
  padding-inline-end: 2rem;
  border-inline-start: 4px solid;
  text-align: start;
}

/* shorthands */
.box {
  margin-block: 2rem 1rem;      /* top, bottom */
  padding-inline: 1.5rem;       /* left + right */
  inline-size: min(100%, 40rem);
  max-block-size: 80vh;
  inset-inline-end: 0;          /* right in LTR */
  border-start-end-radius: 8px; /* top-right in LTR */
}

/* vertical text */
.vertical {
  writing-mode: vertical-rl;    /* inline = top→bottom */
}
```

## PHYSICAL → LOGICAL

| PHYSICAL | LOGICAL (HORIZONTAL LTR) |
|---|---|
| `width / height` | `inline-size / block-size` |
| `margin-left / right` | `margin-inline-start / -end` |
| `margin-top / bottom` | `margin-block-start / -end` |
| `padding-left + right` | `padding-inline` |
| `left / right` | `inset-inline-start / -end` |
| `top / bottom` | `inset-block-start / -end` |
| `border-top-left-radius` | `border-start-start-radius` |
| `text-align: left` | `text-align: start` |

## WRITING MODES

| VALUE | LINES |
|---|---|
| `horizontal-tb` | Horizontal, stack top to bottom |
| `vertical-rl` | Vertical, stack right to left (CJK) |
| `vertical-lr` | Vertical, stack left to right |
| `sideways-rl` | Rotated Latin text (newer) |

## ⚠ COMMON MISTAKES

- Mixing physical and logical for the same side: both apply and order decides
- Using `direction: rtl` in CSS instead of the `dir` attribute
- Assuming `inline-size` is always width (it's height in vertical modes)

## ✓ TIPS & BEST PRACTICES

- Write new CSS logically by default; it costs nothing in LTR-only sites
- Use `margin-inline: auto` to center
- Test RTL by setting `dir="rtl"` on `<html>` in DevTools

## QUICK SUMMARY

- inline = along text; block = across lines
- `*-inline-start` / `*-block-end`
- `inline-size` / `block-size`
- Direction comes from `dir`

# Borders & Outlines



## EDGES AND RINGS

A **border** sits between padding and margin and takes space. Its shorthand `border: 2px solid red` sets width, style and color; nothing shows unless the **style** is set. **`border-radius`** rounds corners and also clips backgrounds. An **outline** is drawn outside the border, takes **no space**, and is what browsers use for focus rings; `outline-offset` moves it away from the edge.

## NOTES

- Default border color is `currentColor` ; default width is `medium` (3px)
- `border-radius: 50%` on a square makes a circle; `999px` makes a pill
- Elliptical corners: `border-radius: 50% / 20%`
- Outlines now follow `border-radius` in all major browsers

## ♿ ACCESSIBILITY

- Never `outline: none` without a visible replacement on `:focus-visible`
- If a focus style uses `box-shadow` , add `outline: 2px solid transparent` so forced-colors mode still shows a ring

## BROWSER SUPPORT

- Borders, radius, outline: universal
- Outline following border-radius: Baseline 2023

## CODE EXPLAINED

→ Shorthand order doesn't matter, but style is required

→ Radius values go clockwise from top-left

→ `999px` radius = fully rounded ends at any height

→ `outline-offset` adds a gap without shifting layout

→ `border-image … 1` slices the gradient as a border (no radius support)

## BORDERS, RADIUS, OUTLINES

```
.card {
  border: 1px solid #ddd;          /* width style color */
  border-radius: 12px;
}

.alert {
  border-left: 4px solid #f57c00;  /* one side */
  border-radius: 0 8px 8px 0;      /* TL TR BR BL */
}

.pill   { border-radius: 999px; }
.circle { border-radius: 50%; aspect-ratio: 1; }
.blob   { border-radius: 40% 60% 70% 30% /
                         40% 50% 60% 50%; }

/* styles */
.dashed { border: 2px dashed currentColor; }
.double { border: 4px double #555; }

/* focus ring that takes no space */
:focus-visible {
  outline: 3px solid #ff9800;
  outline-offset: 3px;
}

/* gradient border */
.fancy {
  border: 4px solid;
  border-image: linear-gradient(90deg, #5b4bc4, #00bcd4) 1;
}
```

## BORDER-STYLE

| VALUE | LOOK |
|---|---|
| `solid` | Single line |
| `dashed / dotted` | Dashes / dots |
| `double` | Two lines (needs ≥3px) |
| `groove / ridge` | 3D carved / raised |
| `inset / outset` | 3D pressed / raised box |
| `none / hidden` | No border |

## BORDER VS OUTLINE

| FEATURE | BORDER | OUTLINE |
|---|---|---|
| `Takes space` | Yes | No |
| `Per side` | Yes | No |
| `Follows radius` | Yes | Yes (now) |
| `Offset` | No | `outline-offset` |
| `Used for focus` | Sometimes | By default |

## ⚠ COMMON MISTAKES

- `border: 2px red` with no style: nothing renders
- Adding a border on hover and shifting layout; use outline, box-shadow or a transparent border by default
- Expecting `border-image` to respect `border-radius`

## ✓ TIPS & BEST PRACTICES

- Reserve space with `border: 2px solid transparent` , then color it on hover
- Use `outline` for focus, never remove it without a replacement
- Combine `border-radius` with `overflow: clip` to round child media

## QUICK SUMMARY

- `border: width style color`
- `border-radius` clockwise from TL
- `outline` takes no space
- `outline-offset` for focus rings

# Shadows

box-shadow · inset · spread · layered shadows · text-shadow · drop-shadow() · elevation scales

## DEPTH WITHOUT IMAGES

**box-shadow** draws one or more shadows around an element's border box: `x y blur spread color`. **inset** draws it inside. Shadows follow `border-radius` and take no layout space. **text-shadow** applies to glyphs (no spread or inset). The `filter: drop-shadow()` function follows the **alpha shape** of an element, so it shadows transparent PNGs and SVG outlines, not just rectangles.

## NOTES

- Comma-separate several shadows; the first one is drawn on top
- Positive spread grows the shadow; negative shrinks it
- A shadow with 0 blur and spread acts like a solid offset copy
- Animating `box-shadow` repaints each frame; animate the `opacity` of a pseudo-element instead

## ♿ ACCESSIBILITY

- Don't rely on a subtle shadow alone to separate a control from its background; borders survive forced-colors mode
- `text-shadow` can help contrast on images but doesn't replace sufficient contrast

## BROWSER SUPPORT

- `box-shadow`, `text-shadow`, `filter: drop-shadow()` : universal

## CODE EXPLAINED

→ Low alpha black gives natural shadows

→ Three stacked shadows mimic soft real-world light

→ `inset` flips the shadow inside the box

→ `0 0 0 3px` = a crisp ring using only spread

→ `drop-shadow()` traces transparent areas of images and SVG

## SHADOW RECIPES

```
/* soft card */
.card { box-shadow: 0 1px 3px rgb(0 0 0 / .12); }

/* layered = more realistic */
.raised {
  box-shadow:
    0 1px 2px rgb(0 0 0 / .08),
    0 4px 8px rgb(0 0 0 / .08),
    0 12px 24px rgb(0 0 0 / .08);
}

/* inset: pressed or input field */
.pressed { box-shadow: inset 0 2px 4px rgb(0 0 0 / .2); }

/* spread-only ring (focus) */
.ring { box-shadow: 0 0 0 3px #5b4bc4; }

/* glow */
.glow { box-shadow: 0 0 16px 4px rgb(91 75 196 / .5); }

/* readable text on images */
.hero h1 { text-shadow: 0 2px 6px rgb(0 0 0 / .6); }

/* shape-aware shadow */
.logo { filter: drop-shadow(0 4px 6px rgb(0 0 0 / .3)); }

/* elevation tokens */
:root {
  --shadow-1: 0 1px 2px rgb(0 0 0 / .1);
  --shadow-2: 0 4px 12px rgb(0 0 0 / .12);
}
```

## BOX-SHADOW VALUES

| PART | MEANING |
|---|---|
| `inset` | Draw inside (optional, first or last) |
| `x offset` | Right is positive |
| `y offset` | Down is positive |
| `blur` | Softness (≥ 0) |
| `spread` | Grow (+) or shrink (−) |
| `color` | Defaults to `currentColor` |

## WHICH SHADOW?

| NEED | USE |
|---|---|
| `Card elevation` | `box-shadow` |
| `Pressed button, inner well` | `box-shadow: inset` |
| `Icon or PNG with transparency` | `filter: drop-shadow()` |
| `Text on photos` | `text-shadow` |
| `Focus ring` | `outline` (or spread ring) |

## ⚠ COMMON MISTAKES

- Using pure black at high opacity: looks harsh and dirty
- Animating large `box-shadow` s, which causes paint jank
- Expecting `box-shadow` on a transparent PNG to follow the image shape

## ✓ TIPS & BEST PRACTICES

- Define 3–5 elevation tokens and use only those
- Fade a pre-rendered shadow on a `::after` for smooth hover effects
- Tint shadows with the brand hue instead of pure black for warmth

## QUICK SUMMARY

- `box-shadow: x y blur spread color`
- Comma-separate to layer
- `inset` for inner shadows
- `drop-shadow()` follows shape

# Font Properties



## CHOOSING AND SIZING TYPE

`font-family` takes a **stack**: a list of fonts tried in order, ending with a **generic family** (`serif`, `sans-serif`, `monospace`, `system-ui`) as the guaranteed fallback. `font-size` sets the em box, `font-weight` the thickness (100–900), and `font-style` italic or oblique. The `font` shorthand sets them all at once but **resets every font longhand it doesn't mention**, including `line-height`.

## NOTES

- `font` shorthand order: style, variant, weight, size/line-height, family (size and family required)
- Weights without a matching font file are synthesized (faux bold); `font-synthesis` controls this
- `bolder` / `lighter` are relative to the parent's weight
- Keyword sizes: `xx-small` … `xxx-large` ; `smaller` / `larger` are relative

## ♿ ACCESSIBILITY

- Body text of at least 16px ( `1rem` ) and sizes in `rem` respect user settings
- Avoid thin weights (100–300) for body text; low stroke contrast hurts readability

## BROWSER SUPPORT

- All font properties: universal
- `system-ui` : Baseline; `ui-monospace` , `ui-serif` , `ui-rounded` : Safari only, so add fallbacks

## CODE EXPLAINED

→ Each family is tried in order until one is available

→ Family names with spaces need quotes; generics never do

→ `font: … 2rem/1.1 …` : size, slash, line-height

→ A later `font` shorthand resets `line-height` to `normal`

→ `font: inherit` gives controls the page's type

## STACKS, SIZES, SHORTHAND

```
body {
  font-family: system-ui, -apple-system, "Segoe UI",
               Roboto, sans-serif;
  font-size: 1rem;
  font-weight: 400;
  line-height: 1.5;
}

code, pre {
  font-family: ui-monospace, "Cascadia Code",
               Menlo, Consolas, monospace;
  font-size: .9em;            /* relative to parent */
}

h1 { font-weight: 800; }
em { font-style: italic; }
.small-caps { font-variant-caps: small-caps; }

/* shorthand: style weight size/line-height family */
.title { font: italic 700 2rem/1.1 Georgia, serif; }

/* shorthand resets line-height! */
.oops { line-height: 2; font: 1rem sans-serif; }
/* line-height is now normal */

/* don't fake bold or italic */
.brand { font-synthesis: none; }

/* inherit fonts into form controls */
button, input, select, textarea { font: inherit; }
```

## GENERIC FAMILIES

| KEYWORD | TYPICAL RESULT |
|---|---|
| `serif` | Times New Roman, Georgia |
| `sans-serif` | Arial, Helvetica |
| `monospace` | Courier, Consolas |
| `system-ui` | OS UI font (SF, Segoe UI, Roboto) |
| `cursive / fantasy` | Decorative; unpredictable |
| `ui-monospace` | SF Mono (Safari only) |
| `math / emoji` | Math or emoji fonts |

## WEIGHT NAMES

| VALUE | NAME |
|---|---|
| `100` | Thin |
| `300` | Light |
| `400` | Normal / Regular |
| `500` | Medium |
| `600` | Semibold |
| `700` | Bold |
| `900` | Black |

## ⚠ COMMON MISTAKES

- No generic family at the end of a stack: the browser default may be a serif
- Using the `font` shorthand after `line-height` and losing it
- Requesting weight 600 when only 400 and 700 are loaded (it rounds to the nearest)

## ✓ TIPS & BEST PRACTICES

- End every stack with a generic family
- Use `system-ui` for fast, native-looking UI text
- Load only the weights you use

## QUICK SUMMARY

- Stack fonts; end with a generic
- `font-weight` 100–900
- `font` shorthand resets longhands
- Size text in `rem`

# Web Fonts



## LOADING CUSTOM FONTS

**`@font-face`** declares a font: a family name, where to load it (`src`), and which weight and style the file covers. **WOFF2** is the only format you need today. **`font-display`** controls what shows while the file downloads: `swap` shows fallback text immediately and swaps when ready. **Fallback metric overrides** (`size-adjust`, `ascent-override`) make the fallback occupy the same space, so the swap doesn't shift layout.

## NOTES

- Fonts download only when text actually uses them
- One `@font-face` per file; group them with the same `font-family` name
- `unicode-range` lets the browser skip files for scripts the page doesn't use
- `local()` in `src` tries an installed font first

## ♿ ACCESSIBILITY

- `font-display: swap` keeps text readable during slow loads (no invisible text)
- Icon fonts read as random characters to screen readers; use SVG icons

## BROWSER SUPPORT

- `@font-face`, WOFF2, `font-display`, `unicode-range`: universal
- `size-adjust`, `ascent-override`, `descent-override`: Baseline 2023 (Safari 17)

## CODE EXPLAINED

→ Two faces share one family name; `font-weight` picks between them

→ `format("woff2")` lets old browsers skip files they can't use

→ `local("Arial")` + `size-adjust`: a metric-matched fallback

→ `unicode-range`: only download when matching characters appear

→ Preload needs `crossorigin`, even for same-origin fonts

## @FONT-FACE AND FALLBACK TUNING

```
@font-face {
  font-family: "Inter";
  src: url("/fonts/inter-400.woff2") format("woff2");
  font-weight: 400;
  font-style: normal;
  font-display: swap;
}

@font-face {
  font-family: "Inter";
  src: url("/fonts/inter-700.woff2") format("woff2");
  font-weight: 700;
  font-display: swap;
}

/* fallback sized to match Inter */
@font-face {
  font-family: "Inter Fallback";
  src: local("Arial");
  size-adjust: 107%;
  ascent-override: 90%;
}

body { font-family: "Inter", "Inter Fallback", sans-serif; }

/* only Latin glyphs in this file */
@font-face {
  font-family: "Inter";
  src: url("/fonts/inter-latin.woff2") format("woff2");
  unicode-range: U+0000-00FF;
}

/* in <head>: start the download early
<link rel="preload" href="/fonts/inter-400.woff2"
      as="font" type="font/woff2" crossorigin> */
```

## FONT-DISPLAY

| VALUE | BEHAVIOR |
|---|---|
| `auto` | Browser decides (usually block) |
| `block` | Invisible text up to ~3s, then swap |
| `swap` | Fallback now, swap whenever loaded |
| `fallback` | ~100ms block, swap only if fast |
| `optional` | Use only if already cached/fast |

## @FONT-FACE DESCRIPTORS

| DESCRIPTOR | PURPOSE |
|---|---|
| `font-family` | Name you use in CSS |
| `src` | `url()` / `local()` + `format()` |
| `font-weight` | Weight or range (`100 900`) |
| `font-style` | `normal`, `italic` |
| `unicode-range` | Characters this file covers |
| `size-adjust` | Scale glyphs |
| `ascent-override` | Fix line-box height |

## ⚠ COMMON MISTAKES

- Missing `crossorigin` on font preloads: the font downloads twice
- Shipping TTF/OTF or several formats; WOFF2 alone covers current browsers
- Declaring weights you never use, or using weights you never declared (faux bold)

## ✓ TIPS & BEST PRACTICES

- Self-host WOFF2 with `font-display: swap` and preload the one or two critical files
- Subset fonts to the characters you need
- Generate fallback overrides with a tool like Fontaine or Capsize

## QUICK SUMMARY

- `@font-face { font-family; src; }`
- WOFF2 only
- `font-display: swap`
- `size-adjust` stops layout shift

# Variable Fonts & Features

font-variation-settings · wght/wdth axes · font-optical-sizing · font-feature-settings · tabular-nums · ligatures

## ONE FILE, MANY STYLES

A **variable font** packs a continuous range of styles into one file along **axes**: weight (`wght`), width (`wdth`), slant (`slnt`), italic (`ital`), optical size (`opsz`) and custom ones. Use the normal properties (`font-weight: 650`) for registered axes, and `font-variation-settings` only for custom ones. **OpenType features** (tabular numbers, small caps, ligatures) are switched on with high-level `font-variant-*` properties, or `font-feature-settings` as a last resort.

## NOTES

- Declare the range in `@font-face` : `font-weight: 100 900`
- Registered axes are lowercase ( `wght` ); custom axes are uppercase ( `GRAD` )
- `font-variation-settings` and `font-feature-settings` reset all unlisted values when redeclared
- `font-optical-sizing: auto` adjusts detail for small and large sizes

## ♿ ACCESSIBILITY

- Tabular figures keep numbers aligned in tables, which helps scanning
- Grade ( `GRAD` ) changes weight without changing width, useful for dark mode or hover without reflow

## BROWSER SUPPORT

- Variable fonts and `font-variation-settings` : Baseline Widely available
- `font-variant-numeric` , `font-optical-sizing` : universal

## CODE EXPLAINED

- → `font-weight: 300 1000` tells the browser the file covers that range
- → Use `font-stretch` for `wdth` and `font-weight` for `wght`
- → Uppercase tags are font-specific custom axes
- → `tabular-nums` : every digit has the same width
- → `font-feature-settings` for stylistic sets the high-level props don't cover

## AXES AND FEATURES

```
@font-face {
  font-family: "Recursive";
  src: url("recursive.woff2") format("woff2");
  font-weight: 300 1000;          /* range */
  font-display: swap;
}

h1 { font-weight: 850; }          /* any value in range */
.condensed { font-stretch: 75%; }  /* wdth axis */
.mono { font-variation-settings: "MONO" 1; } /* custom */

/* animate weight smoothly */
.link { transition: font-weight .2s; }
.link:hover { font-weight: 700; }

/* numbers */
.price, td.num {
  font-variant-numeric: tabular-nums lining-nums;
}
.fraction { font-variant-numeric: diagonal-fractions; }
.zero     { font-variant-numeric: slashed-zero; }

/* ligatures and caps */
code      { font-variant-ligatures: none; }
.label    { font-variant-caps: all-small-caps; }

/* low-level fallback */
.ss01 { font-feature-settings: "ss01" 1, "cv11" 1; }
```

## REGISTERED AXES

| TAG | CSS PROPERTY |
|---|---|
| `wght` | `font-weight` |
| `wdth` | `font-stretch` (`font-width`) |
| `slnt` | `font-style: oblique 10deg` |
| `ital` | `font-style: italic` |
| `opsz` | `font-optical-sizing` |

## FONT-VARIANT-*

| VALUE | EFFECT |
|---|---|
| `tabular-nums` | Equal-width digits |
| `oldstyle-nums` | Digits with ascenders/descenders |
| `diagonal-fractions` | 1/2 → ½ style |
| `slashed-zero` | 0 with a slash |
| `small-caps` | Lowercase as small capitals |
| `no-common-ligatures` | Turn off fi, fl |
| `ordinal` | 1st with superscript st |

## ⚠ COMMON MISTAKES

- Using `font-variation-settings: "wght" 700` instead of `font-weight` , which breaks inheritance and bolder/lighter
- Redeclaring `font-feature-settings` and silently dropping earlier features
- Forgetting the range in `@font-face` , so the browser synthesizes weights

## ✓ TIPS & BEST PRACTICES

- Prefer high-level properties; use the low-level settings only for custom axes and stylistic sets
- Use `tabular-nums` in tables, prices and timers
- One variable file often beats four static weights in total size

## QUICK SUMMARY

- Axes: `wght` `wdth` `slnt` `ital` `opsz`
- Range in `@font-face`
- `font-variant-numeric: tabular-nums`
- Custom axes via `font-variation-settings`

# Text Styling



## DECORATING AND ALIGNING TEXT

**`text-align`** positions inline content in its line box (use `start`/`end` for direction-aware alignment). **`text-decoration`** is a shorthand for line, style, color and thickness, with `text-underline-offset` to move underlines clear of descenders. **`text-transform`** changes case visually without changing the text. To cut long text, `text-overflow: ellipsis` handles one line and `line-clamp` handles several.

## NOTES

- `text-overflow` needs `overflow: hidden` and `white-space: nowrap` to work
- Multi-line clamp still uses the `-webkit-` prefixed trio for broad support
- `text-align: justify` + `hyphens: auto` avoids rivers of space
- `text-decoration-skip-ink: auto` (default) breaks underlines around descenders

## ♿ ACCESSIBILITY

- Avoid `justify` for body text; uneven spacing slows dyslexic readers
- Capitals typed into the HTML can be read letter by letter; keep source text in normal case and use `text-transform`
- Truncated text should be available in full somewhere (title, tooltip, detail view)

## BROWSER SUPPORT

- All decoration and alignment properties: universal
- `-webkit-line-clamp` : universal (prefixed); unprefixed `line-clamp` : limited

## CODE EXPLAINED

- → `text-indent` indents the first line only
- → `text-decoration` shorthand: line, thickness, color (and style)
- → Underline offset moves the line below descenders
- → Ellipsis needs all three: hidden overflow, no wrapping, `ellipsis`
- → The prefixed `-webkit-box` trio is the cross-browser multi-line clamp

## DECORATION, CASE, TRUNCATION

```
.prose     { text-align: start; text-indent: 2em; }
.centered  { text-align: center; }

/* nicer underlines */
a {
  text-decoration: underline 2px #5b4bc4;
  text-underline-offset: 3px;
  text-decoration-skip-ink: auto;
}
a:hover { text-decoration-thickness: 3px; }
.strike { text-decoration: line-through wavy red; }

/* case (display only) */
.label { text-transform: uppercase; letter-spacing: .06em; }
.name  { text-transform: capitalize; }

/* one line with ellipsis */
.truncate {
  overflow: hidden;
  white-space: nowrap;
  text-overflow: ellipsis;
}

/* clamp to 3 lines */
.excerpt {
  display: -webkit-box;
  -webkit-box-orient: vertical;
  -webkit-line-clamp: 3;
  overflow: hidden;
}
```

## TEXT-* PROPERTIES

| PROPERTY | VALUES |
|---|---|
| `text-align` | `start` `end` `center` `justify` |
| `text-decoration-line` | `underline` `overline` `line-through` |
| `text-decoration-style` | `solid` `double` `dotted` `dashed` `wavy` |
| `text-decoration-thickness` | `auto`, `from-font`, length |
| `text-underline-offset` | Length or `auto` |
| `text-transform` | `uppercase` `lowercase` `capitalize` |
| `text-indent` | Length or %; `hanging` |

## TRUNCATION

| NEED | USE |
|---|---|
| `One line + …` | `text-overflow: ellipsis` trio |
| `N lines + …` | `-webkit-line-clamp: N` trio |
| `Fade out` | `mask-image: linear-gradient(#000 70%, #0000)` |
| `Middle of a URL` | Not possible in CSS; do it in JS |

## ⚠ COMMON MISTAKES

- Setting `text-overflow: ellipsis` without `overflow: hidden` and `nowrap`
- Writing headings in uppercase in the HTML instead of using `text-transform`
- Removing link underlines in body text; color alone fails WCAG 1.4.1

## ✓ TIPS & BEST PRACTICES

- Use `text-underline-offset` and thickness instead of `border-bottom` for links
- Add `letter-spacing` to uppercase labels for legibility
- Use `text-align: start` so RTL works automatically

## QUICK SUMMARY

- `text-align: start | center | end`
- `text-decoration` + `text-underline-offset`
- `text-transform` is visual only
- Ellipsis trio; line-clamp trio

# Spacing & Wrapping

line-height · letter-spacing · word-spacing · white-space · overflow-wrap · word-break · hyphens · text-wrap

## LINE HEIGHT AND LINE BREAKS

**`line-height`** sets the height of each line box; use a **unitless** number (1.4–1.6 for body text) so it scales with font size and inherits as a ratio. `letter-spacing` and `word-spacing` adjust tracking. Wrapping is controlled by `white-space` (whether spaces collapse and lines wrap), `overflow-wrap` (break long words only when they'd overflow), `word-break`, `hyphens`, and **`text-wrap`**, which can **`balance`** headings or make paragraphs **`pretty`**.

## NOTES

- `hyphens: auto` needs a `lang` attribute to pick the dictionary
- `white-space: pre-wrap` keeps spaces and newlines but still wraps
- `text-wrap: balance` is limited to short blocks (a few lines) for performance
- `overflow-wrap: anywhere` also lets min-content shrink, `break-word` doesn't

## ♿ ACCESSIBILITY

- WCAG 1.4.12: layouts must survive line-height 1.5, letter-spacing .12em, word-spacing .16em and paragraph spacing 2em
- Avoid tight `line-height` below 1.2 for body text

## BROWSER SUPPORT

- line-height, spacing, white-space, overflow-wrap, word-break: universal
- `hyphens: auto` : Baseline 2023; `text-wrap: balance` : Baseline 2024
- `text-wrap: pretty` : not yet in every engine; it's safe to add as enhancement

## CODE EXPLAINED

- → Unitless `line-height` inherits as a multiplier
- → `balance` evens out line lengths in short text
- → `pretty` avoids a single word on the last line
- → `overflow-wrap: break-word` breaks only words that would overflow
- → `word-break: break-all` breaks anywhere, even in normal words

## READABLE LINES

```
body {
  line-height: 1.55;          /* unitless */
}
h1, h2, h3 {
  line-height: 1.15;
  text-wrap: balance;         /* even heading lines */
}
p { text-wrap: pretty; }      /* avoid orphan words */

.caps    { letter-spacing: .08em; }
.tight   { letter-spacing: -.01em; } /* large headings */

/* long URLs and IDs */
.content { overflow-wrap: break-word; }
.hash    { word-break: break-all; }

/* hyphenation (needs lang="en" on html) */
article p {
  hyphens: auto;
  text-align: justify;
}

/* whitespace handling */
pre   { white-space: pre; }        /* keep all, no wrap */
.poem { white-space: pre-line; }   /* keep newlines */
.chip { white-space: nowrap; }     /* never wrap */
.log  { white-space: pre-wrap; }   /* keep + wrap */
```

## WHITE-SPACE

| VALUE | SPACES | NEWLINES | WRAPS |
|---|---|---|---|
| `normal` | Collapse | Collapse | Yes |
| `nowrap` | Collapse | Collapse | No |
| `pre` | Keep | Keep | No |
| `pre-wrap` | Keep | Keep | Yes |
| `pre-line` | Collapse | Keep | Yes |
| `break-spaces` | Keep (can wrap) | Keep | Yes |

## BREAKING WORDS

| PROPERTY | EFFECT |
|---|---|
| `overflow-wrap: break-word` | Break only if a word overflows |
| `overflow-wrap: anywhere` | Same, and affects min-content |
| `word-break: break-all` | Break between any letters |
| `word-break: keep-all` | No breaks inside CJK words |
| `hyphens: auto` | Dictionary hyphenation |
| `<wbr> / ` | Manual break / soft hyphen in HTML |

## ⚠ COMMON MISTAKES

- `line-height: 24px` on the body: headings with bigger text overlap
- Using `word-break: break-all` for URLs and breaking ordinary words mid-letter
- `hyphens: auto` without a `lang` attribute: nothing happens

## ✓ TIPS & BEST PRACTICES

- Set `line-height` unitless on `body` ; tighten it for large headings
- Add `overflow-wrap: break-word` to user-generated content
- Use `text-wrap: balance` on headings and `pretty` on paragraphs

## QUICK SUMMARY

- Unitless `line-height` ~1.5
- `white-space` controls spaces + wrapping
- `overflow-wrap: break-word` for long words
- `text-wrap: balance | pretty`

# Lists & Counters

list-style-type · list-style-position · ::marker · @counter-style · counter-reset · counter-increment · counters()

## BULLETS AND NUMBERING

**list-style-type** picks the bullet or numbering system (`disc`, `decimal`, `lower-roman`, or any string). **`::marker`** styles the bullet itself: color, size and content. **CSS counters** number anything: `counter-reset` creates a counter, `counter-increment` steps it, and `counter()` or `counters()` prints it in generated content, which is how numbered headings and nested "1.2.3" outlines work. **`@counter-style`** defines custom numbering systems.

## NOTES

- `list-style-position: inside` puts the marker inside the content box
- `list-style: none` removes markers but Safari also drops list semantics
- `<ol>` items use the built-in `list-item` counter
- `counters(name, ".")` joins nested counter values

## ♿ ACCESSIBILITY

- Add `role="list"` to unstyled lists if the item count matters to screen-reader users
- Numbers created with counters in `::before` may not be read as list numbers; keep real `<ol>` where order matters

## BROWSER SUPPORT

- `list-style-*` , counters: universal
- `::marker` : Baseline (Safari supports a subset of properties)
- `@counter-style` : Baseline 2023

## CODE EXPLAINED

→ A string value becomes the literal marker

→ `::marker` colors bullets without extra spans

→ `counter-reset` on the parent, `counter-increment` on each item

→ `counters(item, ".")` prints the full nested path

→ `@counter-style` with `system: cyclic` repeats its symbols

## MARKERS AND COUNTERS

```
ul.checks { list-style-type: "✓ "; }
ol.roman  { list-style-type: upper-roman; }
ul.inside { list-style-position: inside; }

li::marker {
  color: #5b4bc4;
  font-weight: 700;
}

/* unstyled list for navigation */
.nav-list { list-style: none; padding: 0; }

/* numbered headings */
article        { counter-reset: section; }
article h2     { counter-increment: section; }
article h2::before {
  content: counter(section) ". ";
}

/* nested outline numbering: 1, 1.1, 1.1.1 */
ol.outline      { counter-reset: item; list-style: none; }
ol.outline > li    { counter-increment: item; }
ol.outline > li::before {
  content: counters(item, ".") " ";
}

/* custom system */
@counter-style thumbs {
  system: cyclic;
  symbols: "👍";
  suffix: " ";
}
ul.fun { list-style: thumbs; }
```

## LIST-STYLE-TYPE

| VALUE | EXAMPLE |
|---|---|
| `disc / circle / square` | • ○ ▪ |
| `decimal` | 1, 2, 3 |
| `decimal-leading-zero` | 01, 02 |
| `lower-alpha / upper-alpha` | a, b / A, B |
| `lower-roman / upper-roman` | i, ii / I, II |
| `"→ "` | Any string |
| `none` | No marker |

## COUNTER PROPERTIES

| SYNTAX | DOES |
|---|---|
| `counter-reset: n` | Create n at 0 |
| `counter-reset: n 5` | Start at 5 |
| `counter-increment: n` | Add 1 |
| `counter-increment: n 2` | Add 2 |
| `counter-set: n 10` | Set to 10 |
| `counter(n, upper-roman)` | Print with a style |
| `counters(n, ".")` | Print nested path |

## ⚠ COMMON MISTAKES

- Incrementing the counter on the parent instead of each item
- Resetting the counter on every item, so all show 1
- Styling `::marker` with `padding` or `display` (only a few properties apply)

## ✓ TIPS & BEST PRACTICES

- Use string `list-style-type` for simple custom bullets
- Use counters for numbered headings, figures and steps
- Keep `<ol>` for real sequences; the browser and screen readers count for you

## QUICK SUMMARY

- `list-style-type` / `::marker`
- `counter-reset` → `counter-increment` → `counter()`
- `counters()` for 1.2.3
- `@counter-style` custom systems

# Backgrounds

background-image · background-size · cover/contain · background-position · background-repeat · background-clip · multiple backgrounds

## PAINTING BEHIND CONTENT

An element's background is a **color** plus any number of **image layers** (images or gradients). Each layer has its own size, position, repeat, origin and clip, listed comma-separated with the **first layer on top**. `background-size: cover` fills the box and crops; `contain` fits the whole image and may leave gaps. `background-clip` decides where painting stops (border, padding, content or text).

## NOTES

- `background-color` always paints under every image layer
- In the shorthand, size follows position after a slash: `center / cover`
- `background-attachment: fixed` is ignored on most mobile browsers
- `background-clip: text` + transparent text fills letters with the background

## ♿ ACCESSIBILITY

- Background images can't have alt text; put meaningful images in `<img>`
- Set a `background-color` close to the image's average so text stays readable while it loads or if it fails

## BROWSER SUPPORT

- Multiple backgrounds, size, origin, clip: universal
- `background-clip: text` : works everywhere, but keep the `-webkit-` version too for older Chrome/Safari

## CODE EXPLAINED

→ Background color first so text is readable before the image loads

→ `center 30%` : horizontal then vertical position

→ `/ cover` after the position sets the size in the shorthand

→ The scrim gradient sits on top of the photo

→ `background-clip: text` shows the background only through the glyphs

## LAYERS AND SIZING

```css
.hero {
  background-color: #1d1b26;                /* fallback */
  background-image: url("mountains.webp");
  background-size: cover;
  background-position: center 30%;
  background-repeat: no-repeat;
}

/* same thing as a shorthand */
.hero {
  background: #1d1b26 url("mountains.webp")
              center 30% / cover no-repeat;
}

/* multiple layers: first is on top */
.banner {
  background:
    linear-gradient(#0000, #000a),          /* scrim */
    url("photo.jpg") center / cover;
}

/* pattern tile */
.dots {
  background: radial-gradient(#ccc 1px, transparent 1px)
              0 0 / 16px 16px;
}

/* gradient text */
.gradient-text {
  background: linear-gradient(90deg, #5b4bc4, #00bcd4);
  -webkit-background-clip: text;
  background-clip: text;
  color: transparent;
}
```

## BACKGROUND-* LONGHANDS

| PROPERTY | COMMON VALUES |
|---|---|
| `background-image` | `url()`, gradients, `none` |
| `background-size` | `cover` `contain` `auto` `100%` `50%` |
| `background-position` | `center` `top` `right` `50%` `30%` |
| `background-repeat` | `no-repeat` `repeat-x` `space` `round` |
| `background-attachment` | `scroll` `fixed` `local` |
| `background-origin` | `padding-box` `border-box` `content-box` |
| `background-clip` | `border-box` `padding-box` `content-box` `text` |

## COVER VS CONTAIN

| VALUE | FILLS BOX | CROPS | GAPS |
|---|---|---|---|
| `cover` | Yes | Yes | No |
| `contain` | No | No | Possible |
| `100% 100%` | Yes | No | No (distorts) |
| `auto` | Natural size | Maybe | Maybe |

## ⚠ COMMON MISTAKES

- Putting content images (products, people) in CSS backgrounds: no alt text, no lazy-loading control
- Setting `background` shorthand after `background-size` and resetting it
- Forgetting `color: transparent` with `background-clip: text`

## ✓ TIPS & BEST PRACTICES

- Add a gradient scrim layer under text on photos
- Use `image-set()` for resolution-aware background images
- Prefer `<img>` + `object-fit` for important or LCP images

## QUICK SUMMARY

- Layers: first = top
- `center / cover no-repeat`
- `cover` crops, `contain` fits
- `background-clip: text` for gradient text

# Linear & Radial Gradients



## GENERATED IMAGES

Gradients are **images** generated by CSS, so they go anywhere `url()` does: `background-image`, `border-image`, `mask-image`, `list-style-image`. **`linear-gradient()`** blends along a line set by an angle or `to <side>`; **`radial-gradient()`** blends outward from a center as a circle or ellipse. **Color stops** place colors along the line; two stops at the same position make a **hard edge** instead of a blend.

## NOTES

- `0deg` points up and angles go clockwise; the default is `to bottom` (180deg)
- A stop with two positions ( `red 20% 40%` ) holds a solid band
- A lone percentage between colors is a **hint** that moves the midpoint
- Radial sizes: `closest-side` , `farthest-corner` (default) and others

## ♿ ACCESSIBILITY

- Text over a gradient needs contrast at *every* point it covers; test the lightest spot

## BROWSER SUPPORT

- Linear and radial gradients: universal
- Interpolation in other spaces ( `in oklch` ): Baseline 2024

## CODE EXPLAINED

- → No direction means top to bottom
- → `to right` or an angle sets the line
- → `70%` alone is a hint: moves where yellow and green meet
- → `#002395 0 33.3%` : solid band; next color starting at `0` makes a hard edge
- → `circle at top left … transparent 60%` : a corner glow

## LINEAR AND RADIAL

```
.a { background: linear-gradient(#5b4bc4, #00bcd4); }
.b { background: linear-gradient(to right, #5b4bc4, #0bd); }
.c { background: linear-gradient(135deg, #f093fb, #f55); }

/* multiple stops and a hint */
.d {
  background: linear-gradient(90deg,
    red, orange 25%, yellow 50%, 70%, green);
}

/* hard stops: flag stripes */
.flag {
  background: linear-gradient(
    #002395 0 33.3%, #fff 0 66.6%, #ed2939 0);
}

/* fade text out at the bottom */
.fade::after {
  content: "";
  position: absolute; inset: auto 0 0;
  height: 4rem;
  background: linear-gradient(transparent, var(--surface));
}

/* radial */
.e { background: radial-gradient(#fff, #5b4bc4); }
.f { background: radial-gradient(circle at top left,
       #ffe082, transparent 60%); }
.spot {
  background: radial-gradient(circle closest-side,
    #5b4bc4 0 98%, transparent);
}
```

## LINEAR DIRECTIONS

| VALUE | LINE POINTS |
|---|---|
| `to bottom (default)` | Down (180deg) |
| `to top` | Up (0deg) |
| `to right` | Right (90deg) |
| `to bottom right` | Toward that corner |
| `45deg` | Up-right |
| `0.25turn` | Right |

## RADIAL OPTIONS

| PART | VALUES |
|---|---|
| Shape | `circle`, `ellipse` (default) |
| Size | `closest-side` `closest-corner` `farthest-side` `farthest-corner` |
| Explicit size | `circle 100px`, `ellipse 50% 30%` |
| Position | `at center`, `at 30% 70%`, `at top left` |

## ⚠ COMMON MISTAKES

- Writing `linear-gradient(right, …)` : needs `to right`
- Gradient to `transparent` going gray in old browsers; use the color at zero alpha
- Expecting gradients on `background-color` : they're images

## ✓ TIPS & BEST PRACTICES

- Use hard stops for stripes, flags and progress fills
- Layer gradients for complex effects instead of images
- Add `in oklch` to keep mid-colors vivid: `linear-gradient(in oklch, blue, yellow)`

## QUICK SUMMARY

- `linear-gradient(angle, stops)`
- `radial-gradient(shape at pos, stops)`
- Same position twice = hard edge
- Gradients are images

# Conic & Repeating Gradients

conic-gradient() · from · at · pie charts · repeating-linear-gradient() · stripes · in oklch interpolation

## AROUND A POINT, OVER AND OVER

**`conic-gradient()`** sweeps colors **around** a center point instead of outward, which makes pie charts, color wheels and spinners. `from` sets the starting angle and `at` the center. The **repeating** variants (`repeating-linear-gradient()`, `repeating-radial-gradient()`, `repeating-conic-gradient()`) tile their stop pattern endlessly, which is ideal for stripes and checkerboards. A color-interpolation space (`in oklch`) changes how the middle colors blend.

## NOTES

- Conic stops use angles or percentages of the full turn
- Hard stops in a conic gradient make pie slices
- Repeating gradients repeat the distance between the first and last stop
- `longer hue` in `in oklch longer hue` goes the long way around the wheel

## ♿ ACCESSIBILITY

- A CSS pie chart is decorative to screen readers; provide the numbers as text or a table

## BROWSER SUPPORT

- Conic and repeating gradients: Baseline Widely available
- Interpolation color spaces ( `in oklch` , `longer hue` ): Baseline 2024

## CODE EXPLAINED

- → Each slice starts where the last ended ( `0` = same as previous stop)
- → A radial mask turns the pie into a donut
- → `longer hue` from red to red sweeps the whole hue wheel
- → `repeating-conic` tiled at 20px makes a checkerboard
- → Stripes repeat every 20px along the 45° line

## CONIC, REPEATING, INTERPOLATION

```
/* pie chart: 40 / 35 / 25 */
.pie {
  border-radius: 50%;
  background: conic-gradient(
    #5b4bc4 0 40%,
    #00bcd4 0 75%,
    #ffb74d 0);
}

/* donut: mask out the middle */
.donut {
  mask: radial-gradient(circle, transparent 55%, #000 56%);
}

/* color wheel */
.wheel {
  border-radius: 50%;
  background: conic-gradient(in oklch longer hue,
    red, red);
}

/* checkerboard */
.checker {
  background: repeating-conic-gradient(
    #ddd 0 25%, #fff 0 50%) 0 0 / 20px 20px;
}

/* diagonal stripes */
.stripes {
  background: repeating-linear-gradient(45deg,
    #5b4bc4 0 10px, #7e6fd6 10px 20px);
}

/* rings */
.rings {
  background: repeating-radial-gradient(circle,
    #5b4bc4 0 4px, transparent 4px 12px);
}
```

## CONIC SYNTAX

| PART | EXAMPLE |
|---|---|
| `Start angle` | `from 90deg` |
| `Center` | `at 25% 50%` |
| `Stops` | `red 0 25%, blue 90deg` |
| `Space` | `in oklch` |
| `Full` | `conic-gradient(from 0deg at center, …)` |

## INTERPOLATION

| SPACE | MIDPOINTS |
|---|---|
| `srgb (default)` | Can turn gray or muddy |
| `oklab` | Even, perceptual |
| `oklch` | Vivid; hue travels around the wheel |
| `shorter hue` | Default hue path |
| `longer hue` | The other way around |

## ⚠ COMMON MISTAKES

- Building data charts in CSS only: values aren't readable or accessible
- Forgetting `background-size` with repeating patterns made from conic tiles
- Using hsl interpolation and getting uneven brightness across the wheel

## ✓ TIPS & BEST PRACTICES

- Feed pie values from custom properties: `var(--a)` stops
- Use `in oklch` for gradients between saturated colors
- Patterns from gradients cost no requests and scale perfectly

## QUICK SUMMARY

- `conic-gradient(from a at x y, stops)`
- Hard stops = slices
- `repeating-*-gradient()` tiles stops
- `in oklch` for nicer blends

# Filters & Blend Modes

filter · blur() · brightness() · grayscale() · backdrop-filter · mix-blend-mode · background-blend-mode · isolation

## IMAGE EFFECTS IN CSS

**`filter`** applies image effects to an element and its children: `blur()`, `brightness()`, `contrast()`, `grayscale()`, `saturate()`, `hue-rotate()`, `drop-shadow()`, applied left to right. **`backdrop-filter`** applies them to what's *behind* the element, the basis of frosted-glass panels. **Blend modes** mix colors like layers in an image editor: `mix-blend-mode` blends an element with what's behind it, `background-blend-mode` blends an element's own background layers.

## NOTES

- Any `filter` other than `none` creates a stacking context and a containing block for fixed children
- `backdrop-filter` needs a partly transparent background to show the effect
- `isolation: isolate` limits blending to a group
- Filters are GPU-friendly but large blurs on scroll can still be expensive

## ♿ ACCESSIBILITY

- Check text contrast on frosted panels with busy content behind them
- Avoid `filter: invert()` dark modes; they invert images and break brand colors

## BROWSER SUPPORT

- `filter`, blend modes, `isolation`: universal
- `backdrop-filter` unprefixed: Baseline 2024 (keep `-webkit-` for older Safari)

## CODE EXPLAINED

→ Filters chain left to right

→ `opacity()` in a filter can be GPU-accelerated with other effects

→ Backdrop blur only works if the panel background is see-through

→ `difference` with white text inverts whatever is behind it

→ `background-blend-mode: multiply` tints the photo with the color layer

## FILTERS AND BLENDING

```
img.muted   { filter: grayscale(1) contrast(1.1); }
img.muted:hover { filter: none; }

.disabled   { filter: saturate(0) opacity(.6); }
.night      { filter: brightness(.8) sepia(.2); }
.blur-bg    { filter: blur(8px); }

/* frosted glass */
.glass {
  background: rgb(255 255 255 / .2);
  -webkit-backdrop-filter: blur(12px) saturate(1.5);
  backdrop-filter: blur(12px) saturate(1.5);
}

/* text that inverts over any background */
.cursor-label { mix-blend-mode: difference; color: white; }

/* tint a photo with a color */
.duotone {
  background: url("photo.jpg") center / cover, #5b4bc4;
  background-blend-mode: multiply;
}

/* keep blending inside the card */
.card { isolation: isolate; }
.card .overlay { mix-blend-mode: screen; }
```

## FILTER FUNCTIONS

| FUNCTION | EXAMPLE |
|---|---|
| `blur()` | `blur(4px)` |
| `brightness()` | `brightness(1.2)` |
| `contrast()` | `contrast(150%)` |
| `grayscale()` | `grayscale(1)` |
| `saturate()` | `saturate(2)` |
| `hue-rotate()` | `hue-rotate(90deg)` |
| `invert()` / `sepia()` | `invert(1)`, `sepia(.5)` |
| `drop-shadow()` | `drop-shadow(0 2px 4px #0004)` |

## BLEND MODES

| MODE | EFFECT |
|---|---|
| `multiply` | Darkens; white disappears |
| `screen` | Lightens; black disappears |
| `overlay` | Contrast boost |
| `difference` | Inverts where light |
| `color` / `luminosity` | Keep hue or lightness only |
| `soft-light` | Gentle overlay |

## ⚠ COMMON MISTAKES

- Applying `filter: blur()` to a container and blurring its text too
- Using `backdrop-filter` on an opaque background and seeing nothing
- Not isolating, so a blend mode mixes with the whole page

## ✓ TIPS & BEST PRACTICES

- Use `filter` on hover for image galleries (grayscale → color)
- Pair `backdrop-filter` with a semi-transparent background and a border
- Keep `-webkit-backdrop-filter` alongside the standard property

## QUICK SUMMARY

- `filter: fn() fn()` left to right
- `backdrop-filter` = behind
- `mix-blend-mode` with what's below
- `isolation: isolate` contains blending

# Clipping, Masking & Images

clip-path · circle()/polygon()/inset() · mask-image · mask-size · object-fit · object-position · image-rendering

## SHAPING WHAT SHOWS

**`clip-path`** cuts an element to a shape: everything outside is invisible and doesn't receive clicks. Shapes include `inset()`, `circle()`, `ellipse()`, `polygon()` and `path()`. **Masks** are softer: `mask-image` uses an image's alpha (often a gradient) so parts fade rather than cut. For replaced elements like `<img>` and `<video>`, **`object-fit`** controls how content fills its box and **`object-position`** where it's anchored.

## NOTES

- Clip-path shapes with the same number of points can be animated
- `mask` is a shorthand like `background`, with size, position and repeat
- `object-fit: cover` is the `<img>` version of `background-size: cover`
- `image-rendering: pixelated` keeps pixel art crisp when scaled

## ♿ ACCESSIBILITY

- Clipped content is still read by screen readers; don't use clip-path to hide information
- Make sure clipped buttons keep a large enough click area

## BROWSER SUPPORT

- `clip-path` basic shapes, `object-fit`, `object-position`: universal
- Unprefixed `mask-*`: Baseline 2023; `clip-path: shape()`: newer

## CODE EXPLAINED

- → `circle(50%)`: radius half the box
- → `polygon()` takes x y pairs, clockwise
- → Animating `inset()` from full to zero reveals left to right
- → Mask: black (opaque) shows, transparent hides
- → `currentColor` background + SVG mask = a single-color icon you can recolor

## CLIP, MASK, FIT

```
.avatar { clip-path: circle(50%); }
.diag   { clip-path: polygon(0 0, 100% 0,
                             100% 85%, 0 100%); }
.inset  { clip-path: inset(10px round 12px); }
.tri    { clip-path: polygon(50% 0, 100% 100%, 0 100%); }

/* reveal animation */
.reveal {
  clip-path: inset(0 100% 0 0);
  transition: clip-path .6s ease;
}
.reveal.is-visible { clip-path: inset(0 0 0 0); }

/* fade edges with a mask */
.scroller {
  mask-image: linear-gradient(90deg,
    transparent, #000 10%, #000 90%, transparent);
}

/* icon colored by CSS */
.icon {
  background: currentColor;
  mask: url("icon.svg") center / contain no-repeat;
}

/* images that fill a fixed box */
.thumb {
  width: 100%;
  aspect-ratio: 4 / 3;
  object-fit: cover;
  object-position: 50% 25%;   /* keep faces */
}
.pixel-art { image-rendering: pixelated; }
```

## SHAPE FUNCTIONS

| FUNCTION | EXAMPLE |
|---|---|
| `inset()` | `inset(10% 20% round 8px)` |
| `circle()` | `circle(40% at 50% 50%)` |
| `ellipse()` | `ellipse(50% 30%)` |
| `polygon()` | `polygon(0 0, 100% 0, 50% 100%)` |
| `path()` | `path('M0 0 L100 0 …')` |
| `url()` | Reference an SVG `<clipPath>` |

## OBJECT-FIT

| VALUE | RESULT |
|---|---|
| `fill (default)` | Stretch to box (distorts) |
| `contain` | Whole image, letterboxed |
| `cover` | Fill box, crop overflow |
| `none` | Natural size, cropped |
| `scale-down` | Smaller of none / contain |

## ⚠ COMMON MISTAKES

- Hiding text with clip-path and assuming it's hidden from screen readers
- Forgetting a fixed size or `aspect-ratio` on `object-fit` images
- Animating between polygons with different point counts (it jumps)

## ✓ TIPS & BEST PRACTICES

- Use `object-fit: cover` for thumbnails and hero images
- Fade scroll edges with a gradient `mask-image`
- Use mask icons to recolor SVGs with `currentColor`

## QUICK SUMMARY

- `clip-path` hard cut
- `mask-image` soft fade
- `object-fit: cover` for `<img>`
- `object-position` sets the crop anchor

# Display & Formatting Contexts

display · block · inline · inline-block · none · contents · flow-root · two-value syntax

## HOW A BOX PARTICIPATES

`display` sets two things: the **outer** type (how the box sits among siblings: `block` or `inline`) and the **inner** type (how its children are laid out: `flow`, `flex`, `grid`, `table`). The two-value syntax makes this explicit: `display: inline flex` is the same as `inline-flex`. Some values start a new **block formatting context** (BFC), an isolated layout region that contains floats and stops margin collapse.

## NOTES

- `display: none` removes the box and its children from layout and the accessibility tree
- `display: contents` removes the box but keeps its children, which become children of the grandparent
- `flow-root` is the clean way to make a BFC (clearfix replacement)
- Every element has a UA default: `div` is block, `span` inline, `li` list-item

## ♿ ACCESSIBILITY

- `display: contents` has historically stripped semantics from buttons, tables and lists in some browsers; test with a screen reader
- `display: none` hides from everyone; use a visually-hidden class to hide only visually

## BROWSER SUPPORT

- Single-keyword values: universal
- Two-value syntax ( `block flex` ): Baseline 2023; `flow-root` , `contents` : Baseline

## CODE EXPLAINED

- → `inline-block` : sits in a line but accepts width, height and vertical margin
- → `inline-flex` : flex container that flows inline with text
- → `flow-root` contains floats inside without hacks
- → `contents` : the wrapper disappears and its children become grid items
- → `list-item` generates a marker box

## OUTER AND INNER DISPLAY

```
.block   { display: block; }          /* block flow */
.inline  { display: inline; }         /* inline flow */
.ib      { display: inline-block; }   /* inline flow-root */
.hidden  { display: none; }

/* inner layout types */
.row     { display: flex; }           /* block flex */
.chip    { display: inline-flex; }    /* inline flex */
.layout  { display: grid; }           /* block grid */
.badge   { display: inline grid; }    /* two-value */

/* contain floats, stop margin collapse */
.media   { display: flow-root; }

/* make children join the parent grid */
.grid > .wrapper { display: contents; }

/* list-item gets a ::marker */
.faux-li { display: list-item; }

/* table layout without table markup */
.t   { display: table; width: 100%; }
.tr  { display: table-row; }
.td  { display: table-cell; vertical-align: middle; }
```

## DISPLAY VALUES

| VALUE | TWO-VALUE FORM |
|---|---|
| `block` | `block flow` |
| `inline` | `inline flow` |
| `inline-block` | `inline flow-root` |
| `flex / inline-flex` | `block flex / inline flex` |
| `grid / inline-grid` | `block grid / inline grid` |
| `flow-root` | `block flow-root` |
| `list-item` | `block flow list-item` |
| `table` | `block table` |

## NEW BFC CREATED BY

| TRIGGER | NOTES |
|---|---|
| `display: flow-root` | Cleanest option |
| `overflow other than visible/clip` | Side effect: clips |
| `float, position: absolute/fixed` | Also leaves flow |
| `display: inline-block` | Inline-level |
| `flex/grid items` | Each item is its own context |
| `contain: layout/paint` | Also isolates |

## ⚠ COMMON MISTAKES

- Using `display: none` to hide skip links or labels screen readers need
- Applying `width` to inline elements and wondering why nothing changes
- Using `overflow: hidden` as a clearfix and clipping focus rings or shadows

## ✓ TIPS & BEST PRACTICES

- Use `flow-root` instead of clearfix hacks
- Use `contents` sparingly; test semantics
- Read `display` as outer + inner to predict behavior

## QUICK SUMMARY

- outer (block/inline) + inner (flow/flex/grid)
- `none` removes from layout and a11y tree
- `flow-root` = new BFC
- `contents` = drop the wrapper box

# Normal Flow & Floats



## THE DEFAULT LAYOUT

Without flex, grid or positioning, elements lay out in **normal flow**: block boxes stack vertically and fill the width, inline boxes run along lines and wrap like text. A **float** is pulled to the left or right edge and inline content wraps around it. Floats were once used for whole page layouts; today they're for what they were designed for: wrapping text around images and pull quotes, optionally along a shape with `shape-outside`.

## NOTES

- Floats are removed from block flow but still push inline content aside
- A parent containing only floats collapses to zero height unless it's a BFC
- `clear: both` moves an element below earlier floats
- `float: inline-start` follows text direction

## ♿ ACCESSIBILITY

- Floated content still reads in source order; place the float in the HTML near the text it relates to

## BROWSER SUPPORT

- `float` , `clear` : universal; `shape-outside` : Baseline Widely available
- `float: inline-start` / `inline-end` : Baseline 2023

## CODE EXPLAINED

→ Text and inline content flow around the floated image

→ `shape-outside` lets text follow the circle, not the box

→ `flow-root` makes the parent wrap its floats

→ The clearfix pseudo-element clears floats inside old layouts

→ `clear: both` pushes the footer under every float

## FLOATING AND CLEARING

```css
/* wrap text around an image */
.article img.left {
  float: left;
  width: 40%;
  margin: 0 1rem 1rem 0;
}

/* wrap along a circle */
.portrait {
  float: right;
  width: 12rem;
  aspect-ratio: 1;
  border-radius: 50%;
  shape-outside: circle(50%);
  shape-margin: 1rem;
}

/* contain floats in the parent */
.media { display: flow-root; }

/* legacy clearfix (old codebases) */
.clearfix::after {
  content: "";
  display: table;
  clear: both;
}

/* start below any floats */
.article footer { clear: both; }

/* drop cap */
.drop::first-letter {
  float: left;
  font-size: 3.2em;
  line-height: .8;
  margin: .1em .1em 0 0;
}
```

## FLOW BASICS

| BOX | BEHAVIOR |
|---|---|
| `Block` | Stacks vertically, full width |
| `Inline` | Flows in lines, wraps |
| `Float` | Moved to side; text wraps |
| `Absolute` | Removed from flow entirely |
| `Flex/grid item` | Laid out by the container |

## FLOAT & CLEAR

| VALUE | EFFECT |
|---|---|
| `float: left / right` | Push to that side |
| `float: inline-start / end` | Direction-aware |
| `float: none` | Default |
| `clear: left / right` | Below floats on that side |
| `clear: both` | Below all floats |
| `shape-outside` | Wrap text along a shape |

## ⚠ COMMON MISTAKES

- Building columns with floats instead of flex or grid
- Forgetting to contain floats, so the parent's background disappears
- Floating flex or grid items: `float` has no effect on them

## ✓ TIPS & BEST PRACTICES

- Use floats only for text wrap; use flex/grid for layout
- Add `shape-margin` so text doesn't touch the shape
- Use `display: flow-root` instead of clearfix

## QUICK SUMMARY

- Normal flow: blocks stack, inlines wrap
- `float` for text wrap only
- `clear: both` drops below floats
- `flow-root` contains floats

# Positioning



## TAKING BOXES OUT OF PLACE

`position` changes how a box is placed. **`relative`** shifts it visually without moving its space. **`absolute`** removes it from flow and places it relative to its **containing block**, the nearest positioned ancestor. **`fixed`** is relative to the viewport. **`sticky`** acts like relative until scrolling reaches its offset, then sticks within its parent. Offsets use `top`, `right`, `bottom`, `left` or the `inset` shorthand.

## NOTES

- "Positioned" means any value except `static`
- `transform`, `filter` or `contain` on an ancestor become the containing block for `fixed` children
- Absolute boxes shrink-wrap; give `inset: 0` or a width to fill
- Sticky needs an offset ( `top: 0` ) and fails inside an ancestor with `overflow: hidden`

## ♿ ACCESSIBILITY

- Fixed headers can cover focused elements; add `scroll-padding-top` to the scroller (WCAG 2.4.11)
- Positioning changes visual order only; tab and reading order follow the HTML

## BROWSER SUPPORT

- All `position` values: universal; `inset` : Baseline 2021

## CODE EXPLAINED

→ `position: relative` on the card makes it the containing block

→ `inset: 0` stretches an absolute box to all four edges

→ `inset: 0` + fixed size + `margin: auto` centers it

→ `inset: auto 1rem 1rem auto` = bottom-right corner

→ Sticky headers need a background or content shows through

## POSITION VALUES

```css
.card { position: relative; }      /* anchor for children */

.card .badge {
  position: absolute;
  top: .5rem;
  right: .5rem;
}

/* fill the parent */
.overlay {
  position: absolute;
  inset: 0;                        /* all four edges 0 */
}

/* center absolutely */
.center {
  position: absolute;
  inset: 0;
  margin: auto;
  width: 10rem; height: 4rem;
}

/* stays on screen */
.toast { position: fixed; inset: auto 1rem 1rem auto; }

/* sticks while its section scrolls */
.section-head {
  position: sticky;
  top: 0;
  background: var(--surface);      /* hide content below */
}

/* account for a fixed header */
html { scroll-padding-top: 4rem; }
```

## POSITION VALUES

| VALUE | IN FLOW? | RELATIVE TO |
|---|---|---|
| `static` | Yes | Not positioned (default) |
| `relative` | Yes (space kept) | Its own normal spot |
| `absolute` | No | Nearest positioned ancestor |
| `fixed` | No | Viewport |
| `sticky` | Yes | Nearest scroll container |

## STICKY NOT STICKING?

| CAUSE | FIX |
|---|---|
| No `top`/`bottom` set | Add `top: 0` |
| Ancestor has `overflow: hidden` | Use `overflow: clip` |
| Parent is as short as the sticky box | It has nowhere to move |
| Flex/grid item stretches | `align-self: start` |

## ⚠ COMMON MISTAKES

- Forgetting `position: relative` on the parent, so the absolute child aligns to the page
- A `transform` on an ancestor breaking `position: fixed`
- Sticky inside an `overflow: hidden` wrapper

## ✓ TIPS & BEST PRACTICES

- Use `inset` instead of four offsets
- Use `overflow: clip` when you need clipping without breaking sticky
- Prefer grid or flex for layout; position for overlays and badges

## QUICK SUMMARY

- `relative` = shift, keep space
- `absolute` = out of flow, nearest positioned ancestor
- `fixed` = viewport; `sticky` = scroll-dependent
- `inset: 0` fills

# Z-Index & Stacking Contexts

z-index · stacking context triggers · isolation: isolate · opacity & transform contexts · top layer

## WHAT SITS ON TOP

`z-index` orders positioned elements (and flex/grid items) along the z-axis, but only **within their stacking context**. A stacking context is a self-contained group: its children can never rise above or sink below things outside it, no matter how big their z-index. Many properties create one: `opacity` below 1, `transform`, `filter`, `isolation: isolate`, and positioned elements with a z-index. The **top layer** (dialogs, popovers, fullscreen) sits above everything.

## NOTES

- `z-index` has no effect on `position: static` elements (except flex/grid items)
- Within one context, later siblings paint over earlier ones at equal z-index
- Negative z-index puts a child behind its parent's content but not behind the context's background
- `isolation: isolate` makes a context with no other side effects

## ACCESSIBILITY

- Modals in the top layer ( `<dialog>` via `showModal()` ) also make the rest of the page inert for keyboard and screen readers

## BROWSER SUPPORT

- `z-index` , stacking contexts: universal; `isolation` : Baseline
- Top layer via `<dialog>` and popover: Baseline 2022 / 2024

## CODE EXPLAINED

- → `.header` forms a context at level 2
- → `.main` 's transform makes its own context at level 0
- → The dropdown's 9999 is compared only inside `.main`
- → `isolation: isolate` lets `z-index: -1` stay inside the card
- → Dialogs opened with `showModal()` are above all z-indexes

## WHY 9999 DOESN'T WIN

```
/* context A */
.header {
  position: relative;
  z-index: 2;
}

/* context B: transform creates a context */
.main {
  transform: translateZ(0);       /* context, z = auto→0 */
}
.main .dropdown {
  position: absolute;
  z-index: 9999;                  /* only inside .main */
}                                 /* still under .header */

/* fix: isolate components, keep z small */
.card { isolation: isolate; }
.card .bg   { position: absolute; inset: 0; z-index: -1; }
.card .text { position: relative; }

/* layering tokens */
:root {
  --z-dropdown: 10;
  --z-sticky: 20;
  --z-overlay: 30;
  --z-toast: 40;
}

/* top layer: no z-index needed */
dialog::backdrop { background: #0008; }
```

## CREATES A STACKING CONTEXT

| PROPERTY | CONDITION |
|---|---|
| `position + z-index` | absolute/relative with z-index ≠ auto |
| `position` | fixed or sticky (always) |
| `opacity` | < 1 |
| `transform, filter, perspective` | ≠ none |
| `isolation` | isolate |
| `mix-blend-mode` | ≠ normal |
| `contain` | layout, paint, strict, content |
| `will-change` | One of the above properties |
| `flex/grid item` | z-index ≠ auto |

## PAINT ORDER INSIDE

| ORDER | LAYER |
|---|---|
| 1 | Context background and borders |
| 2 | Negative z-index children |
| 3 | Block boxes in flow |
| 4 | Floats |
| 5 | Inline content |
| 6 | z-index: 0 / auto positioned |
| 7 | Positive z-index children |

## ⚠ COMMON MISTAKES

- Escalating z-index to 9999 when the real issue is the parent's context
- Adding `opacity: .99` or a `transform` and unexpectedly trapping a dropdown
- Setting `z-index` on a static element and seeing no effect

## ✓ TIPS & BEST PRACTICES

- Use small z-index scales stored as tokens
- Inspect contexts with the DevTools Layers panel or 3D view
- Use `<dialog>` and `popover` so overlays live in the top layer

## QUICK SUMMARY

- z-index works within a stacking context
- Many properties create contexts
- `isolation: isolate` = clean context
- Top layer beats everything

# Overflow & Visibility

overflow · overflow-x/y · hidden vs clip · scrollbar-gutter · visibility · display:none vs opacity: 0 · content-visibility

## TOO MUCH CONTENT, OR NONE SHOWN

When content is bigger than its box, **`overflow`** decides what happens: `visible` (spill out, the default), `hidden` (clip, but still scrollable by script), `clip` (clip with no scrolling at all), `scroll` (always show scrollbars) or `auto` (scrollbars when needed). Separately, there are several ways to **hide** an element, and they differ in whether it keeps its space, gets focus, and is announced by screen readers.

## NOTES

- Setting one axis to non-visible forces the other off `visible` too (except with `clip`)
- `overflow: hidden` creates a scroll container; this is what breaks `position: sticky`
- `scrollbar-gutter: stable` reserves space so layout doesn't jump when a scrollbar appears
- `content-visibility: auto` skips rendering off-screen sections until needed

## ♿ ACCESSIBILITY

- `visibility: hidden` and `display: none` hide from screen readers; `opacity: 0` does not
- A visually-hidden utility keeps text for screen readers only (sheet 75)
- Scrollable regions need to be keyboard reachable; add `tabindex="0"` and a label

## BROWSER SUPPORT

- `overflow`, `visibility`: universal; `overflow: clip`: Baseline 2022
- `scrollbar-gutter`, `content-visibility`: Baseline 2024

## CODE EXPLAINED

- → `overflow: auto` scrolls only when content overflows
- → `scrollbar-gutter: stable` keeps width constant
- → `overflow: clip` doesn't create a scroll container
- → `opacity: 0` alone still takes clicks; add `pointer-events: none`
- → `contain-intrinsic-size` gives skipped sections a placeholder height

## OVERFLOW AND HIDING

```
.scroll-box {
  max-height: 20rem;
  overflow: auto;               /* bars only when needed */
  scrollbar-gutter: stable;     /* no layout jump */
}

.table-wrap { overflow-x: auto; }  /* wide tables */

.rounded-media {
  border-radius: 12px;
  overflow: clip;               /* sticky still works */
}

/* hide entirely: no space, no a11y */
.is-hidden    { display: none; }

/* invisible but keeps space; not focusable */
.placeholder  { visibility: hidden; }

/* invisible but still there for a11y and clicks */
.fade-out     { opacity: 0; pointer-events: none; }

/* lazy-render long pages */
.article-section {
  content-visibility: auto;
  contain-intrinsic-size: auto 600px;
}

/* tables collapse rows */
tr.collapsed  { visibility: collapse; }
```

## OVERFLOW VALUES

| VALUE | CLIPS | SCROLLABLE |
|---|---|---|
| `visible` | No | No |
| `hidden` | Yes | By script only |
| `clip` | Yes | No |
| `scroll` | Yes | Yes, bars always |
| `auto` | Yes | Yes, bars when needed |

## WAYS TO HIDE

| METHOD | SPACE | FOCUS | SCREEN READER |
|---|---|---|---|
| `display: none` | No | No | Hidden |
| `visibility: hidden` | Yes | No | Hidden |
| `opacity: 0` | Yes | Yes | Read |
| `hidden attribute` | No | No | Hidden |
| `.visually-hidden` | No | Yes | Read |
| `inert attribute` | Yes | No | Hidden |

## ⚠ COMMON MISTAKES

- `overflow-x: hidden` on `body` to fix horizontal scroll instead of finding the wide element
- Using `opacity: 0` to hide menus that can still be tabbed into
- `overflow: hidden` on a sticky element's ancestor

## ✓ TIPS & BEST PRACTICES

- Find overflow culprits with `* { outline: 1px solid red }`
- Use `overflow: clip` for rounded corners on media
- Wrap wide tables in a focusable `overflow-x: auto` container

## QUICK SUMMARY

- `auto` scrolls when needed
- `clip` = clip without scrolling
- `display:none` vs `visibility` vs `opacity`
- `scrollbar-gutter: stable`

# Flexbox: Container



## ONE-DIMENSIONAL LAYOUT

**Flexbox** lays out items along one **main axis** (a row or a column), with the **cross axis** perpendicular to it. `display: flex` makes the container; its direct children become **flex items**. `justify-content` distributes items along the main axis, `align-items` aligns them on the cross axis, and `gap` adds space between them. With `flex-wrap: wrap`, items flow onto new lines and `align-content` spaces those lines.

## NOTES

- Default: `flex-direction: row`, `flex-wrap: nowrap`, `align-items: stretch`
- `row-reverse` and `column-reverse` flip visual order only
- Text nodes directly inside the container become anonymous flex items
- `align-content` only matters when there are multiple lines

## ♿ ACCESSIBILITY

- `*-reverse` makes visual order differ from tab and reading order (WCAG 1.3.2, 2.4.3); reorder the HTML instead

## BROWSER SUPPORT

- Flexbox: universal; `gap` in flexbox: Baseline 2021

## CODE EXPLAINED

- → `justify-content` works on the main axis, `align-items` on the cross axis
- → Two-value `gap`: row gap, then column gap
- → In a column, `justify-content` becomes vertical
- → `justify-content` + `align-items: center` = centered both ways
- → `inline-flex` is a flex container that flows like text

## CONTAINER PROPERTIES

```
.row {
  display: flex;
  flex-direction: row;          /* main axis → */
  flex-wrap: wrap;
  justify-content: space-between; /* main axis */
  align-items: center;          /* cross axis */
  gap: 1rem 2rem;               /* row-gap column-gap */
}

/* shorthand for direction + wrap */
.col { display: flex; flex-flow: column nowrap; }

/* perfect centering */
.center {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100dvh;
}

/* spread a toolbar */
.toolbar {
  display: flex;
  align-items: center;
  gap: .5rem;
}

/* wrapping tags with even line spacing */
.tags {
  display: flex;
  flex-wrap: wrap;
  align-content: flex-start;
  gap: .5rem;
}

/* inline flex: sits in text */
.icon-label {
  display: inline-flex; align-items: center; gap: .25em;
}
```

## JUSTIFY-CONTENT

| VALUE | FREE SPACE |
|---|---|
| `flex-start / start` | Packed at start |
| `flex-end / end` | Packed at end |
| `center` | Packed in the middle |
| `space-between` | Between items, none at edges |
| `space-around` | Half-size at edges |
| `space-evenly` | Equal everywhere |

## ALIGN-ITEMS

| VALUE | CROSS-AXIS |
|---|---|
| `stretch (default)` | Fill the line's height |
| `flex-start / start` | Top (in a row) |
| `flex-end / end` | Bottom |
| `center` | Middle |
| `baseline` | Align text baselines |

## ⚠ COMMON MISTAKES

- Expecting `justify-content` to center vertically in a row (that's `align-items`)
- Using margins between items instead of `gap`
- Using `row-reverse` to reorder content and breaking keyboard order

## ✓ TIPS & BEST PRACTICES

- Think "main axis" and "cross axis", not horizontal and vertical
- Use `gap` for spacing; it never adds space at the edges
- Use flex for one-dimensional rows and columns; grid for two

## QUICK SUMMARY

- `display: flex` → children are items
- `justify-content` = main axis
- `align-items` = cross axis
- `gap` between items

# Flexbox: Items

flex-grow · flex-shrink · flex-basis · flex shorthand · align-self · order · min-width: 0

## HOW ITEMS SHARE SPACE

Each flex item starts at its **`flex-basis`** (its ideal size). If there's space left, items **grow** in proportion to `flex-grow`; if there's too little, they **shrink** in proportion to `flex-shrink` (weighted by basis). The **`flex`** shorthand sets all three; use it rather than the longhands. Items can't shrink below their **min-content** size by default, which is why long words and code blocks overflow until you add `min-width: 0`.

## NOTES

- `flex: 1` = `1 1 0%`: share all space equally regardless of content
- `flex: auto` = `1 1 auto`: grow from content size
- `flex: none` = `0 0 auto`: rigid
- `order` changes visual position only (default 0)

## ♿ ACCESSIBILITY

- `order` does not change tab or reading order; reorder the HTML when the sequence matters

## BROWSER SUPPORT

- All flex item properties: universal

## CODE EXPLAINED

- → `flex: 1` → `1 1 0%`: equal widths
- → `flex: 0 0 16rem` keeps the sidebar exactly 16rem
- → `min-width: 0` lets `.main` shrink below long content
- → `flex: 1 1 15rem`: wrap when items would be narrower than 15rem
- → `margin-inline-start: auto` absorbs the free space

## GROW, SHRINK, BASIS

```
/* equal columns */
.cols > * { flex: 1; }

/* sidebar fixed, main fills */
.layout { display: flex; gap: 2rem; }
.sidebar { flex: 0 0 16rem; }   /* no grow, no shrink */
.main    { flex: 1; min-width: 0; } /* allow shrinking */

/* 2:1 split */
.a { flex: 2; }
.b { flex: 1; }

/* wrap items at a minimum width */
.cards { display: flex; flex-wrap: wrap; gap: 1rem; }
.cards > * { flex: 1 1 15rem; }  /* basis 15rem */

/* one item aligned differently */
.row { display: flex; align-items: flex-start; }
.row .tall { align-self: stretch; }

/* push the last item to the end */
.nav .spacer { margin-inline-start: auto; }

/* visual reorder (use rarely) */
.featured { order: -1; }
```

## FLEX SHORTHAND

| VALUE | MEANS |
|---|---|
| `flex: initial` | `0 1 auto` (default) |
| `flex: 1` | `1 1 0%` |
| `flex: auto` | `1 1 auto` |
| `flex: none` | `0 0 auto` |
| `flex: 0 0 200px` | Fixed 200px |
| `flex: 2 1 10rem` | Grow ×2 from 10rem |

## ITEM PROPERTIES

| PROPERTY | DEFAULT |
|---|---|
| `flex-grow` | 0 |
| `flex-shrink` | 1 |
| `flex-basis` | auto |
| `align-self` | auto (uses align-items) |
| `order` | 0 |
| `min-width` | auto (= min-content) |

## ⚠ COMMON MISTAKES

- Text or `<pre>` overflowing a flex item: add `min-width: 0` (or `overflow: hidden`)
- Using `flex-grow: 1` and expecting equal widths (needs basis 0: `flex: 1`)
- Setting `width` on items and having `flex-basis: auto` silently use it

## ✓ TIPS & BEST PRACTICES

- Always use the `flex` shorthand
- `flex: 1 1 <min>` + `flex-wrap: wrap` makes responsive card rows
- Use `align-self` for one-off cross-axis alignment

## QUICK SUMMARY

- basis → grow or shrink
- `flex: 1` equal share
- `flex: none` rigid
- `min-width: 0` fixes overflow

# Flexbox Patterns

centering · nav bar · media object · sticky footer · equal-height cards · margin-left: auto · wrapping toolbars

## RECIPES

Most everyday components are a handful of flex declarations. These patterns cover the common ones: a **navigation bar** with a pushed-right item, the **media object** (image beside text), a **sticky footer** that stays at the bottom of short pages, **equal-height cards** with buttons aligned to the bottom, and a **wrapping toolbar**. Each relies on `gap`, auto margins and the `flex` shorthand from the last two sheets.

## NOTES

- Auto margins in flex absorb free space in that direction
- Flex items stretch to the tallest item in a line by default
- Nested flex containers are normal; each solves one axis
- If a pattern needs rows *and* columns to line up, use grid

## ♿ ACCESSIBILITY

- Keep nav links in a `<ul>` inside `<nav>`; flex on the `ul` doesn't change semantics
- Make sure wrapped toolbars keep a logical tab order left to right

## BROWSER SUPPORT

- Every pattern here: Baseline Widely available

## CODE EXPLAINED

→ `margin-inline-start: auto` pushes login to the far end

→ `flex: none` stops the image shrinking

→ `main { flex: 1 }` fills leftover height, pushing the footer down

→ `margin-top: auto` sends the button to the bottom of each card

→ The search box grows but wraps below 12rem

## FIVE PATTERNS

```
/* 1. nav: logo left, links, login right */
.nav { display: flex; align-items: center; gap: 1rem; }
.nav .login { margin-inline-start: auto; }

/* 2. media object */
.media { display: flex; gap: 1rem; align-items: start; }
.media img { flex: none; width: 4rem; }
.media .body { flex: 1; min-width: 0; }

/* 3. sticky footer */
body { display: flex; flex-direction: column;
       min-height: 100dvh; }
main { flex: 1; }

/* 4. equal-height cards, buttons at bottom */
.cards { display: flex; gap: 1rem; flex-wrap: wrap; }
.card  { flex: 1 1 16rem; display: flex;
         flex-direction: column; }
.card .btn { margin-top: auto; }

/* 5. toolbar that wraps */
.toolbar {
  display: flex; flex-wrap: wrap;
  gap: .5rem; align-items: center;
}
.toolbar .search { flex: 1 1 12rem; }
```

## PATTERN CHEATS

| GOAL | KEY DECLARATION |
|---|---|
| Center anything | `justify-content` + `align-items: center` |
| Push one item away | `margin-inline-start: auto` |
| Fixed + fluid | `flex: none` + `flex: 1` |
| Bottom-aligned button | column flex + `margin-top: auto` |
| Responsive row | `flex-wrap: wrap` + `flex: 1 1 Xrem` |
| Sticky footer | column body + `main { flex: 1 }` |

## FLEX OR GRID?

| NEED | USE |
|---|---|
| Items in one row or column | Flex |
| Content decides sizes | Flex |
| Rows and columns line up | Grid |
| Layout decides sizes | Grid |
| Overlapping items | Grid |
| Unknown number of wrapping items | Either (grid aligns columns) |

## ⚠ COMMON MISTAKES

- Building a full page grid from nested flex rows; columns won't line up
- Forgetting `min-width: 0` on the text side of a media object
- Last row of wrapped cards stretching wider than the rest (use grid for even columns)

## ✓ TIPS & BEST PRACTICES

- Reach for auto margins before extra wrapper divs
- Use grid when the last row must match earlier columns
- Name patterns as utility classes ( `.cluster`, `.stack`, `.sidebar` )

## QUICK SUMMARY

- Auto margin = push
- Column flex + `margin-top: auto` = bottom
- `main { flex: 1 }` = sticky footer
- Wrap + basis = responsive row

# Grid: Tracks & Templates

display: grid · grid-template-columns · grid-template-rows · fr · repeat() · minmax() · auto-fill vs auto-fit · gap

## TWO-DIMENSIONAL LAYOUT

**CSS Grid** lays items out in rows *and* columns at once. Define **tracks** with `grid-template-columns` and `grid-template-rows`; children are placed into the cells automatically. The **`fr`** unit shares leftover space, **`repeat()`** avoids repetition, and **`minmax(min, max)`** gives a track a flexible range. `repeat(auto-fit, minmax(15rem, 1fr))` makes a fully responsive grid with no media queries.

## NOTES

- `fr` divides space after fixed tracks and gaps are subtracted
- `1fr` is really `minmax(auto, 1fr)`; use `minmax(0, 1fr)` to let it shrink below content
- Rows you don't define are created automatically (implicit tracks)
- `gap` replaces the old `grid-gap`

## ♿ ACCESSIBILITY

- Grid placement can make visual order differ from source order; keep the HTML in a sensible reading order

## BROWSER SUPPORT

- CSS Grid: Baseline since 2017; `gap`: universal
- Subgrid: Baseline 2023 (sheet 56)

## CODE EXPLAINED

- → `200px 1fr 2fr`: fixed column, then the rest split 1:2
- → `auto 1fr auto`: header, stretching body, footer
- → `auto-fit` + `minmax()`: as many 15rem+ columns as fit
- → `min(15rem, 100%)` stops overflow on very narrow screens
- → `minmax(0, 1fr)` lets columns shrink below their content

## DEFINING TRACKS

```css
.grid {
  display: grid;
  grid-template-columns: 200px 1fr 2fr;  /* fixed + flex */
  grid-template-rows: auto 1fr auto;
  gap: 1rem;
  min-height: 100dvh;
}

/* 12-column system */
.twelve { display: grid;
          grid-template-columns: repeat(12, 1fr); }

/* responsive cards, no media queries */
.cards {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(min(15rem, 100%), 1fr));
  gap: 1.5rem;
}

/* allow columns to shrink below long content */
.code-cols {
  grid-template-columns: repeat(2, minmax(0, 1fr));
}

/* sidebar clamps between 12rem and 20rem */
.layout { grid-template-columns: minmax(12rem, 20rem) 1fr; }

/* track fits content up to a max */
.form { grid-template-columns: fit-content(14rem) 1fr; }
```

## TRACK SIZES

| VALUE | SIZE |
|---|---|
| `200px / 10rem` | Fixed |
| `1fr` | Share of free space |
| `auto` | Content-based (stretches) |
| `min-content / max-content` | Intrinsic |
| `minmax(a, b)` | Between a and b |
| `fit-content(x)` | Content, capped at x |
| `repeat(n, …)` | Repeat a pattern |

## AUTO-FILL VS AUTO-FIT

| KEYWORD | SPARE ROOM |
|---|---|
| `auto-fill` | Keeps empty tracks; items stay min size |
| `auto-fit` | Collapses empty tracks; items stretch |
| `Few items, wide screen` | Difference is visible |
| `Many items` | Same result |

## ⚠ COMMON MISTAKES

- Long words or `<pre>` blowing out `1fr` columns: use `minmax(0, 1fr)`
- Applying grid properties to grandchildren: only direct children are items
- Using `auto-fill` when you wanted items to stretch (`auto-fit`)

## ✓ TIPS & BEST PRACTICES

- Learn `repeat(auto-fit, minmax(min(X, 100%), 1fr))` by heart
- Turn on the DevTools grid overlay to see lines and tracks
- Use `fr` instead of percentages; gaps are subtracted automatically

## QUICK SUMMARY

- `grid-template-columns` / `-rows`
- `fr` = free-space share
- `repeat()`, `minmax()`
- auto-fit + minmax = responsive

# Grid: Line-Based Placement



## PLACING ITEMS ON LINES

Grid **lines** are numbered from 1 at the start edge; negative numbers count from the end, so `-1` is always the last explicit line. Place an item with `grid-column: start / end` and `grid-row`, or with **`span n`** to cover n tracks. Lines can also be **named** in the template (`[content-start]`) and used instead of numbers. Items you don't place are auto-placed into the next free cells.

## NOTES

- A grid with 3 columns has 4 column lines
- `grid-column: 1 / -1` spans the full width of the explicit grid
- Items placed in the same cells overlap; later ones paint on top (use `z-index`)
- Lines named `x-start` and `x-end` create an implicit area called `x`

## ♿ ACCESSIBILITY

- Overlapping or reordered items still read in DOM order; check that order makes sense

## BROWSER SUPPORT

- All placement properties: Baseline Widely available

## CODE EXPLAINED

- → `1 / -1`: first line to last line
- → `span 2`: cover two tracks from wherever it's placed
- → `content-start` / `content-end` define an area named `content`
- → `grid-area: 1 / 1` stacks every child in the same cell
- → `grid-area` with four values: row start, column start, row end, column end

## LINES, SPANS, NAMES

```css
.grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 8rem;
  gap: 1rem;
}

.hero    { grid-column: 1 / -1; }     /* full width */
.feature { grid-column: span 2; grid-row: span 2; }
.side    { grid-column: 4; grid-row: 2 / 4; }
.last    { grid-column: -2 / -1; }    /* last column */

/* named lines: full-bleed article layout */
.article {
  display: grid;
  grid-template-columns:
    [full-start] 1fr
    [content-start] min(65ch, 100% - 2rem) [content-end]
    1fr [full-end];
}
.article > *       { grid-column: content; }
.article > .bleed  { grid-column: full; }

/* overlap: text on image, same cell */
.banner > * { grid-area: 1 / 1; }

/* grid-area: row-start / col-start / row-end / col-end */
.box { grid-area: 2 / 1 / 4 / 3; }
```

## PLACEMENT SYNTAX

| VALUE | MEANING |
|---|---|
| `grid-column: 2` | Start at line 2, span 1 |
| `grid-column: 2 / 4` | Lines 2 to 4 |
| `grid-column: 2 / span 3` | Line 2, cover 3 |
| `grid-column: span 2` | Auto start, cover 2 |
| `grid-column: 1 / -1` | Full explicit width |
| `grid-column: content` | Named area or lines |

## LONGHANDS

| PROPERTY | SETS |
|---|---|
| `grid-column-start` | Start line |
| `grid-column-end` | End line |
| `grid-row-start / -end` | Row lines |
| `grid-column` | start / end |
| `grid-row` | start / end |
| `grid-area` | rs / cs / re / ce |

## ⚠ COMMON MISTAKES

- Counting tracks instead of lines (3 columns = 4 lines)
- Expecting `-1` to mean the last implicit row: it's the last *explicit* line
- Placing many items manually when auto-placement would work

## ✓ TIPS & BEST PRACTICES

- Use named lines for full-bleed article layouts
- Stack layers with `grid-area: 1 / 1` instead of absolute positioning
- Show line numbers with the DevTools grid overlay

## QUICK SUMMARY

- Lines start at 1; `-1` = last
- `start / end` or `span n`
- Named lines → areas
- Same cell = overlap

# Grid: Areas & Auto-Placement



## NAMED AREAS AND THE IMPLICIT GRID

**`grid-template-areas`** draws a layout as ASCII art: each string is a row, each word a cell, and repeating a name spans it. Items join an area with `grid-area: name`. Items without a position are **auto-placed**, filling rows by default. When they run past the defined tracks, the grid adds **implicit** rows (or columns) sized by `grid-auto-rows` / `grid-auto-columns`, and `grid-auto-flow: dense` backfills gaps left by large items.

## NOTES

- Each area must be a rectangle; L-shapes are invalid
- A `.` marks an empty cell
- All rows in the template must have the same number of cells
- Redefining the template in a media query rearranges the whole page

## ♿ ACCESSIBILITY

- `dense` packing can move items out of reading order visually; avoid it for content where sequence matters

## BROWSER SUPPORT

- Template areas, auto-flow, implicit tracks: Baseline Widely available

## CODE EXPLAINED

- → Each string is a row; repeated names span cells
- → `grid-area: header` places the element in that area
- → One media query rewrites the whole layout
- → `grid-auto-rows: 10rem` sizes rows created automatically
- → `dense` fills holes left by wide items

## TEMPLATE AREAS

```
.page {
  display: grid;
  grid-template-columns: 14rem 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header"
    "nav    main"
    "footer footer";
  min-height: 100dvh;
}
.page > header { grid-area: header; }
.page > nav    { grid-area: nav; }
.page > main   { grid-area: main; }
.page > footer { grid-area: footer; }

/* rearrange on small screens */
@media (width < 48rem) {
  .page {
    grid-template-columns: 1fr;
    grid-template-areas: "header" "main" "nav" "footer";
  }
}

/* implicit rows and dense packing */
.gallery {
  display: grid;
  grid-template-columns:
    repeat(auto-fill, minmax(10rem, 1fr));
  grid-auto-rows: 10rem;
  grid-auto-flow: dense;
}
.gallery .wide { grid-column: span 2; }
```

## AUTO-PLACEMENT

| PROPERTY | VALUES |
|---|---|
| `grid-auto-flow` | `row` (default), `column`, `dense` |
| `grid-auto-rows` | Size of implicit rows |
| `grid-auto-columns` | Size of implicit columns |
| `grid-template` | Shorthand: rows / columns + areas |
| `grid` | Shorthand incl. auto-flow |

## AREAS RULES

| RULE | EXAMPLE |
|---|---|
| `Rows are strings` | `"a a" "b c"` |
| `Same name spans` | `"h h h"` |
| `Empty cell` | `". main"` |
| `Must be rectangular` | No L-shapes |
| `Equal cell counts` | Every row same length |

## ⚠ COMMON MISTAKES

- Uneven row lengths or L-shaped areas: the whole template is invalid
- Forgetting `grid-area` on children, so they auto-place instead
- Using `grid-auto-flow: dense` on content lists and scrambling reading order

## ✓ TIPS & BEST PRACTICES

- Use template areas for page layouts; they're readable and easy to rearrange
- Set `grid-auto-rows` for galleries so rows stay even
- Use `minmax(auto, …)` implicit rows for content of varying height

## QUICK SUMMARY

- `grid-template-areas` ASCII art
- `grid-area: name`
- Implicit tracks: `grid-auto-rows`
- `dense` backfills holes

# Grid: Alignment & Subgrid



## ALIGNING ITEMS AND SHARING TRACKS

Grid alignment has two axes: **`justify-*`** works along the inline axis (row direction) and **`align-*`** along the block axis. `*-items` aligns every item inside its cell, `*-self` overrides one item, and `*-content` moves the whole grid within the container when the tracks are smaller than it. **`subgrid`** lets a nested grid reuse its parent's tracks, so headings, text and buttons line up across sibling cards.

## NOTES

- `place-items: center` = `align-items: center; justify-items: center`
- Default `stretch` fills the cell (unless the item has a fixed size or aspect-ratio)
- A subgrid item must span the parent tracks it inherits ( `grid-row: span 3` )
- Subgrid inherits the parent's gaps unless it sets its own

## ♿ ACCESSIBILITY

- Subgrid aligns visually without changing DOM order, so card content stays readable in sequence

## BROWSER SUPPORT

- Grid alignment properties: Baseline Widely available
- `subgrid` : Baseline 2023

## CODE EXPLAINED

- → `justify-content` centers the 30rem grid inside a wider container
- → `align-items: start` stops items stretching to the row height
- → `place-items: center` is the shortest centering
- → `grid-row: span 3` + `subgrid` makes card sections align across siblings
- → Column subgrid aligns every label/input pair

## ALIGNMENT AND SUBGRID

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 10rem);
  justify-content: center;    /* whole grid in container */
  align-items: start;         /* items at top of cells */
  justify-items: center;      /* items centered in cells */
}
.grid .special { align-self: end; justify-self: stretch; }

/* center one thing, the shortest way */
.center { display: grid; place-items: center; }

/* cards whose parts line up across a row */
.cards {
  display: grid;
  grid-template-columns:
    repeat(auto-fill, minmax(16rem, 1fr));
  gap: 1rem;
}
.card {
  display: grid;
  grid-row: span 3;                /* title, body, button */
  grid-template-rows: subgrid;     /* share parent rows */
  gap: .5rem;
}

/* form: labels and inputs aligned across rows */
.form { display: grid; grid-template-columns: auto 1fr; }
.form .field {
  grid-column: span 2;
  display: grid;
  grid-template-columns: subgrid;
}
```

## WHICH PROPERTY?

| PROPERTY | MOVES |
|---|---|
| `justify-items` | All items, inline axis |
| `align-items` | All items, block axis |
| `place-items` | Both (align / justify) |
| `justify-self / align-self` | One item |
| `justify-content` | Whole grid, inline axis |
| `align-content` | Whole grid, block axis |
| `place-content` | Both |

## ALIGNMENT VALUES

| VALUE | RESULT |
|---|---|
| `stretch` | Fill the cell (default) |
| `start / end` | Cell start / end |
| `center` | Centered in cell |
| `baseline` | Align text baselines |
| `space-between / -around / -evenly` | `*-content` only |

## ⚠ COMMON MISTAKES

- Using `justify-content` to center items in their cells (that's `justify-items` )
- Forgetting the `span` on a subgrid item, so it inherits only one track
- Expecting stretch on images with fixed width

## ✓ TIPS & BEST PRACTICES

- `display: grid; place-items: center` for any centering job
- Use subgrid for card grids with aligned titles and buttons
- Remember: `*-items` inside cells, `*-content` moves the tracks

## QUICK SUMMARY

- justify = inline, align = block
- items / self / content
- `place-items: center`
- `grid-template-rows: subgrid`

# Grid Patterns



## RECIPES

A few grid patterns cover most page and component layouts: the **auto-fit card grid** (the "RAM" pattern: Repeat, Auto, Minmax), the **holy grail** page with header, footer and two sidebars, a **full-bleed** article where some elements break out of the text column, **overlapping** layers without absolute positioning, and a **sidebar** layout that stacks on narrow screens. Grid shines whenever rows and columns must line up.

## NOTES

- Use `min()` inside `minmax()` to prevent overflow on tiny screens
- Layering with `grid-area: 1 / 1` keeps items in flow, so the container sizes to the tallest
- Nested grids are fine; use subgrid when they need the parent's lines
- Grid and flex combine: grid for the page, flex inside components

## ♿ ACCESSIBILITY

- The holy grail areas should map to landmarks ( `header` , `nav` , `main` , `aside` , `footer` )

## BROWSER SUPPORT

- Every pattern here: Baseline Widely available

## CODE EXPLAINED

- → RAM: as many 16rem+ columns as fit, stretched evenly
- → `grid-template: rows / columns` shorthand
- → The middle column is the text measure; `.bleed` spans all three
- → Everything in one cell, aligned with `align-self`
- → `fit-content(18rem)` sizes the sidebar to content up to 18rem

## FIVE GRID PATTERNS

```
/* 1. RAM card grid */
.cards {
  display: grid; gap: 1rem;
  grid-template-columns:
    repeat(auto-fit, minmax(min(16rem, 100%), 1fr));
}

/* 2. holy grail */
.page {
  display: grid; min-height: 100dvh;
  grid-template: auto 1fr auto / 12rem 1fr 12rem;
}
.page > header, .page > footer { grid-column: 1 / -1; }

/* 3. full bleed */
.post {
  display: grid;
  grid-template-columns: 1fr min(65ch, 100% - 2rem) 1fr;
}
.post > *      { grid-column: 2; }
.post > .bleed { grid-column: 1 / -1; }

/* 4. overlap text on image */
.hero { display: grid; }
.hero > * { grid-area: 1 / 1; }
.hero h1 { align-self: end; z-index: 1; }

/* 5. sidebar that stacks when narrow */
.with-sidebar {
  display: grid; gap: 2rem;
  grid-template-columns: fit-content(18rem) minmax(0, 1fr);
}
@media (width < 40rem) {
  .with-sidebar { grid-template-columns: 1fr; }
}
```

## PATTERN CHEATS

| PATTERN | KEY LINE |
|---|---|
| Card grid | `repeat(auto-fit, minmax(min(X,100%),1fr))` |
| Holy grail | `grid-template: auto 1fr auto / A 1fr A` |
| Full bleed | `1fr min(65ch, 100% - 2rem) 1fr` |
| Overlap | `grid-area: 1 / 1` |
| Centering | `place-items: center` |
| Pancake stack | `grid-template-rows: auto 1fr auto` |

## GRID VS FLEXBOX

| SITUATION | PICK |
|---|---|
| Columns must align across rows | Grid |
| Items size from content in one line | Flex |
| Page-level layout | Grid |
| Nav bars, button groups | Flex |
| Overlap without positioning | Grid |
| Unknown items, even last row | Grid |

## ⚠ COMMON MISTAKES

- Using a fixed `minmax(16rem, 1fr)` that overflows on 320px screens
- Absolute-positioning overlays that could be grid layers
- Choosing flex for card grids and getting a stretched last row

## ✓ TIPS & BEST PRACTICES

- Keep these as utility classes ( `.grid-auto` , `.bleed` , `.stack` )
- Combine grid layout with container queries for truly modular components
- Inspect with the DevTools grid badge

## QUICK SUMMARY

- RAM = Repeat, Auto, Minmax
- `grid-template` shorthand
- Full bleed via named/side columns
- Grid for 2D, flex for 1D

# Media Queries

@media · media types · range syntax · min-width · orientation · and/or/not · breakpoints · mobile-first

## CONDITIONAL CSS

**`@media`** applies rules only when a condition about the device or viewport is true. Conditions combine a **media type** (`screen`, `print`) with **media features** (`width`, `orientation`, `hover`…). The modern **range syntax** reads like math: `(width >= 48rem)`, `(30rem <= width < 60rem)`. **Mobile-first** CSS writes the small-screen layout as the default and adds `min-width`/`>=` queries to enhance it as space grows.

## NOTES

- `and` requires all conditions; a comma means OR; `not` negates a whole query
- `em` / `rem` in media queries are relative to the browser default, not your `html` font size
- Media queries respond to the viewport; container queries (sheet 60) respond to a component's space
- Requires `<meta name="viewport" content="width=device-width, initial-scale=1">` on mobile

## ♿ ACCESSIBILITY

- Content must reflow at 320 CSS px without horizontal scrolling (WCAG 1.4.10)
- Query in `em` / `rem` so layouts respond when users enlarge text

## BROWSER SUPPORT

- `@media`, `min-width` / `max-width`, `orientation`: universal
- Range syntax ( `width >= 48rem` ): Baseline 2023

## CODE EXPLAINED

→ Base styles need no query; queries only add

→ `>=` queries layer up from small to large

→ Range syntax avoids the `.99` overlap hack

→ A comma lets either condition match

→ Height queries catch short landscape phones

## MOBILE-FIRST QUERIES

```
/* base: small screens */
.layout { display: grid; gap: 1rem; }
.nav    { display: none; }

/* tablet and up */
@media (width >= 48rem) {
  .layout { grid-template-columns: 14rem 1fr; }
  .nav    { display: block; }
}

/* desktop */
@media (width >= 75rem) {
  .layout { grid-template-columns: 16rem 1fr 18rem; }
}

/* a range */
@media (48rem <= width < 75rem) {
  .promo { display: none; }
}

/* older syntax, same as above */
@media (min-width: 48rem) and (max-width: 74.99rem) { }

/* OR with a comma */
@media (orientation: landscape), (width >= 60rem) { }

/* type + feature */
@media screen and (max-height: 30rem) {
  .hero { min-height: auto; }
}
```

## MEDIA FEATURES

| FEATURE | EXAMPLE |
|---|---|
| `width / height` | `(width >= 40rem)` |
| `aspect-ratio` | `(aspect-ratio > 16/9)` |
| `orientation` | `(orientation: portrait)` |
| `resolution` | `(resolution >= 2dppx)` |
| `hover / pointer` | `(hover: hover)`, `(pointer: coarse)` |
| `prefers-*` | Sheet 59 |
| `display-mode` | `(display-mode: standalone)` |

## COMMON BREAKPOINTS

| NAME | WIDTH |
|---|---|
| `Small phone` | < 30rem (480px) |
| `Tablet` | ≥ 48rem (768px) |
| `Laptop` | ≥ 64rem (1024px) |
| `Desktop` | ≥ 75rem (1200px) |
| `Wide` | ≥ 90rem (1440px) |

## ⚠ COMMON MISTAKES

- Desktop-first `max-width` queries that pile up overrides
- Forgetting the viewport meta tag; phones render at ~980px and zoom out
- Choosing breakpoints by device instead of where the content breaks

## ✓ TIPS & BEST PRACTICES

- Set breakpoints where your layout breaks, not at device sizes
- Store breakpoints as comments or custom media (preprocessor) to keep them consistent
- Reach for container queries for components

## QUICK SUMMARY

- `@media (width >= X)` mobile-first
- `and` / `,` / `not`
- Range syntax replaces min/max
- Viewport meta tag required

# Preference & Feature Queries

prefers-color-scheme · prefers-reduced-motion · prefers-contrast · forced-colors · hover · pointer · @supports

## RESPECTING USERS AND BROWSERS

**User preference** media features let CSS follow settings people chose in their OS: dark mode (`prefers-color-scheme`), less motion (`prefers-reduced-motion`), more contrast (`prefers-contrast`), and forced high-contrast palettes (`forced-colors`). **Input** features (`hover`, `pointer`) tell you whether the main input can hover and how precise it is. **`@supports`** tests whether the browser understands a property, value or selector, so new CSS can be layered on safely.

## NOTES

- `any-hover` / `any-pointer` check *any* input, not just the primary one
- In `forced-colors: active`, the browser replaces your colors with a system palette
- `@supports not (…)` and `and` / `or` combinations are allowed
- `@supports selector(:has(a))` tests selector support

## ♿ ACCESSIBILITY

- Reduce or remove non-essential motion under `prefers-reduced-motion: reduce` (WCAG 2.3.3)
- In forced-colors mode, backgrounds and box-shadows vanish; keep borders or outlines on controls

## BROWSER SUPPORT

- `prefers-color-scheme`, `prefers-reduced-motion`, `hover`, `pointer`, `@supports`: universal
- `prefers-contrast`: Baseline 2022; `forced-colors`: check support (forced-colors mode is mainly a Windows feature)
- `prefers-reduced-transparency`: not in all engines yet

## CODE EXPLAINED

→ Dark tokens swap only the variables, not every rule

→ Opt-in motion: transitions only when the user hasn't asked for less

→ `prefers-contrast: more` strengthens borders and muted text

→ `CanvasText` is the system text color in forced-colors mode

→ `@supports not` provides the fallback for older browsers

## PREFERENCES AND SUPPORT

```
/* dark mode tokens */
@media (prefers-color-scheme: dark) {
  :root { --surface: #16141d; --text: #ece9f6; }
}

/* motion only when allowed */
@media (prefers-reduced-motion: no-preference) {
  .card { transition: translate .3s ease; }
  html  { scroll-behavior: smooth; }
}

/* more contrast requested */
@media (prefers-contrast: more) {
  :root { --muted: #333; }
  .btn  { border: 2px solid; }
}

/* Windows high contrast etc. */
@media (forced-colors: active) {
  .icon { forced-color-adjust: auto; }
  .card { border: 1px solid CanvasText; }
}

/* touch vs mouse */
@media (pointer: coarse) { .btn { min-height: 44px; } }
@media (hover: hover) { .row:hover { background: #eee; } }

/* feature queries */
@supports (display: grid) and (gap: 1rem) { }
@supports not (aspect-ratio: 1) {
  .square { height: 0; padding-top: 100%; }
}
@supports selector(:has(a)) { .card:has(img) { } }
```

## PREFERENCE FEATURES

| FEATURE | VALUES |
|---|---|
| `prefers-color-scheme` | `light`, `dark` |
| `prefers-reduced-motion` | `no-preference`, `reduce` |
| `prefers-contrast` | `more`, `less`, `custom`, `no-preference` |
| `forced-colors` | `active`, `none` |
| `prefers-reduced-transparency` | `reduce` |
| `prefers-reduced-data` | `reduce` (limited) |

## INPUT FEATURES

| FEATURE | TYPICAL DEVICE |
|---|---|
| `hover: hover` + `pointer: fine` | Mouse, trackpad |
| `hover: none` + `pointer: coarse` | Phone, tablet |
| `hover: none` + `pointer: fine` | Stylus |
| `any-pointer: coarse` | Laptop with touchscreen |

## ⚠ COMMON MISTAKES

- Writing animations first and only disabling them in `reduce`; opt in with `no-preference` instead
- Using `@media (max-width)` to detect touch devices instead of `pointer: coarse`
- Relying on `box-shadow` focus rings that disappear in forced-colors mode

## ✓ TIPS & BEST PRACTICES

- Test preferences in DevTools Rendering → Emulate CSS media features
- Put dark mode in tokens and switch them with one media query
- Wrap new features in `@supports` rather than browser sniffing

## QUICK SUMMARY

- `prefers-color-scheme` / `-reduced-motion` / `-contrast`
- `forced-colors: active`
- `hover` and `pointer` for input
- `@supports (prop: val)` / `selector()`

# Container Queries

container-type · container-name · container shorthand · @container · size queries · style() queries

## COMPONENTS THAT RESPOND TO THEIR SPACE

**Container queries** style an element based on the size of an **ancestor container** instead of the viewport. Mark a wrapper with `container-type: inline-size`, then write `@container (width > 30rem) { … }` for its descendants. The same card can then be compact in a sidebar and spacious in the main column. **Style queries** (`@container style(--theme: dark)`) test a container's custom property values.

## NOTES

- A query applies to *descendants*; an element can't query itself
- `inline-size` containment means the container's width can't depend on its children
- Without a name, `@container` uses the nearest container ancestor
- Container units ( `cqi` , `cqb` ) size things relative to the container

## ♿ ACCESSIBILITY

- Container queries make components reflow inside narrow slots, which also helps at high zoom

## BROWSER SUPPORT

- Size container queries and `cq*` units: Baseline 2023
- Style queries for custom properties: not yet in all engines
- Scroll-state queries: newer, Chromium only

## CODE EXPLAINED

→ `container-type: inline-size` makes each column a size container

→ `container: name / type` shorthand

→ Base styles are for the narrowest slot

→ `@container (width > 30rem)` checks the nearest container's width

→ `4cqi` = 4% of the container's inline size

## RESPONSIVE COMPONENT

```
/* 1. declare containers */
.sidebar, .main {
  container-type: inline-size;
}
.product-list {
  container: products / inline-size;   /* name / type */
}

/* 2. component base: stacked */
.card {
  display: grid;
  gap: 1rem;
}

/* 3. side-by-side when its container is wide */
@container (width > 30rem) {
  .card {
    grid-template-columns: 10rem 1fr;
  }
}

/* named container */
@container products (width > 50rem) {
  .card h2 { font-size: 1.5rem; }
}

/* fluid inside the container */
.card h2 { font-size: clamp(1rem, 4cqi, 1.75rem); }

/* style query (newer) */
@container style(--variant: featured) {
  .card { border: 2px solid gold; }
}
```

## CONTAINER-TYPE

| VALUE | CAN QUERY |
|---|---|
| `normal` | Style queries only |
| `inline-size` | Width (inline size) |
| `size` | Width and height (needs explicit height) |
| `scroll-state` | Stuck / snapped / scrollable (newer) |

## QUERY FORMS

| QUERY | MATCHES WHEN |
|---|---|
| `@container (width > 30rem)` | Nearest container is wider |
| `@container card (…)` | Container named `card` |
| `(orientation: landscape)` | Container wider than tall |
| `(aspect-ratio > 1)` | Same, as ratio |
| `style(--x: y)` | Custom property equals y |

## ⚠ COMMON MISTAKES

- Querying an element that is itself the container (queries apply to descendants)
- Using `container-type: size` without a set height, so the container collapses
- Making the page `body` a container and then wondering why viewport-sized children shrink

## ✓ TIPS & BEST PRACTICES

- Mark layout slots (columns, sidebars) as containers, not the components
- Write component styles mobile-first with `@container (width > …)` upgrades
- Use `cqi` units for fluid component typography

## QUICK SUMMARY

- `container-type: inline-size`
- `@container (width > X)`
- Name with `container: name / type`
- `cqi` = container-relative units

# Print Styles

@media print · @page · size · page margins · break-before · break-inside · orphans/widows · print-color-adjust

## DESIGNING FOR PAPER

**`@media print`** rules apply when printing or saving as PDF. Typical print CSS hides navigation and buttons, switches to dark text on white, shows link URLs, and controls **page breaks** with `break-before`, `break-after` and `break-inside`. The **`@page`** rule sets the paper `size` and page margins. Browsers drop background colors and images by default to save ink; `print-color-adjust: exact` keeps them where they matter.

## NOTES

- `orphans` / `widows` set the minimum lines left at the bottom / top of a page
- `break-inside: avoid` keeps figures, tables rows and cards on one page
- Absolute units ( `pt` , `cm` , `in` ) behave predictably on paper
- Fixed and sticky elements may repeat on every page; set them to `static`

## ♿ ACCESSIBILITY

- Print styles help people who read on paper; keep headings and link text meaningful when printed
- Show URLs after links so printed references are usable

## BROWSER SUPPORT

- `@media print` , `break-*` , `@page { margin }` : universal
- `@page { size }` : Chromium and Firefox; Safari support is newer
- `print-color-adjust` : write both prefixed and unprefixed; `orphans` / `widows` : not in Firefox

## CODE EXPLAINED

→ `@page` sets paper size and printable margins

→ Hide anything interactive or decorative

→ `attr(href)` prints the link destination

→ `break-after: avoid` keeps a heading with its next paragraph

→ `print-color-adjust: exact` keeps colored backgrounds

## A PRINT STYLESHEET

```
@page {
  size: letter portrait;    /* or A4, 8.5in 11in */
  margin: 0.5in 0.6in;
}

@media print {
  nav, .ads, .btn, video { display: none !important; }

  body {
    font: 11pt/1.4 Georgia, serif;
    color: #000;
    background: #fff;
  }

  a[href^="http"]::after {
    content: " (" attr(href) ")";
    font-size: 9pt;
  }

  h1, h2, h3 { break-after: avoid; }   /* stay with text */
  figure, table, .card { break-inside: avoid; }
  .chapter { break-before: page; }
  p { orphans: 3; widows: 3; }

  .header { position: static; }       /* not on each page */

  .chart, .badge {
    -webkit-print-color-adjust: exact;
    print-color-adjust: exact;        /* keep colors */
  }
}
```

## BREAK PROPERTIES

| VALUE | EFFECT |
|---|---|
| `break-before: page` | Start on a new page |
| `break-after: page` | New page after |
| `break-after: avoid` | Don't break after (headings) |
| `break-inside: avoid` | Keep element together |
| `orphans: 3` | ≥ 3 lines at page bottom |
| `widows: 3` | ≥ 3 lines at page top |

## @PAGE

| SYNTAX | USE |
|---|---|
| `size: letter / A4` | Paper size |
| `size: landscape` | Orientation |
| `size: 8.5in 11in` | Explicit size |
| `margin: 2cm` | Page margins |
| `@page :first { }` | First page only |
| `@page :left / :right` | Facing pages |

## ⚠ COMMON MISTAKES

- Leaving dark-mode backgrounds that the browser strips, leaving light text on white
- Absolute layouts and fixed heights that cut content across pages
- Using `page-break-*` only; prefer the modern `break-*` names (legacy ones still work)

## ✓ TIPS & BEST PRACTICES

- Preview with DevTools → Rendering → Emulate CSS media type: print
- Set print colors explicitly; don't inherit screen themes
- Test with Save as PDF in each browser you care about

## QUICK SUMMARY

- `@media print { … }`
- `@page { size; margin }`
- `break-*` controls pagination
- `print-color-adjust: exact`

# Responsive Patterns

intrinsic layouts · fluid type · responsive media · max-width: 100% · viewport meta · layout shifts

## RESPONSIVE WITHOUT BREAKPOINTS

Modern responsive CSS leans on **intrinsic** techniques that adapt continuously instead of jumping at breakpoints: `clamp()` for fluid type and spacing, `min()` for widths, `auto-fit` grids and wrapping flex rows, and container queries for components. Media need `max-width: 100%` and reserved dimensions so they never overflow or cause **layout shift**. Breakpoints remain for big structural changes.

## NOTES

- The viewport meta tag is required for any of this to work on phones
- Reserve space for images with `width` / `height` attributes or `aspect-ratio`
- A fluid type scale: each step is a `clamp()` between a min and max size
- Every-Layout primitives (stack, cluster, sidebar, switcher) solve layouts without queries

## ♿ ACCESSIBILITY

- Test at 200% and 400% zoom; content should reflow without loss (WCAG 1.4.4, 1.4.10)
- Don't disable zoom with `user-scalable=no` or `maximum-scale=1`

## BROWSER SUPPORT

- Every technique on this sheet: Baseline Widely available (container queries: 2023)

## CODE EXPLAINED

- → Fluid steps scale smoothly between minimum and maximum
- → `min(70rem, 100% - …)` replaces width + max-width + padding
- → Switcher: basis flips from huge negative to huge positive at 40rem
- → Sidebar: content keeps at least 60%, sidebar wraps below otherwise
- → `aspect-ratio` sizes video embeds without padding hacks

## INTRINSIC TOOLKIT

```
/* <meta name="viewport"
         content="width=device-width, initial-scale=1"> */

/* fluid type scale */
:root {
  --step-0: clamp(1rem, .95rem + .25vw, 1.125rem);
  --step-2: clamp(1.5rem, 1.2rem + 1.5vw, 2.25rem);
  --space:  clamp(1rem, 3vw, 2rem);
}

/* media never overflow */
img, video, iframe { max-width: 100%; height: auto; }

/* centered column with gutters */
.wrap { width: min(70rem, 100% - 2 * var(--space));
        margin-inline: auto; }

/* switcher: row until too narrow, then column */
.switcher { display: flex; flex-wrap: wrap; gap: 1rem; }
.switcher > * {
  flex-grow: 1;
  flex-basis: calc((40rem - 100%) * 999);
}

/* sidebar: sidebar wraps under content when tight */
.sidebar-l { display: flex; flex-wrap: wrap; gap: 1rem; }
.sidebar-l > :first-child { flex: 1 1 16rem; }
.sidebar-l > :last-child  { flex-basis: 0; flex-grow: 999;
                            min-inline-size: 60%; }

/* responsive embeds */
.embed { aspect-ratio: 16 / 9; width: 100%; }
```

## TOOLKIT

| PROBLEM | TECHNIQUE |
|---|---|
| Text too big/small | `clamp()` fluid type |
| Fixed widths | `min()`, `max-width` |
| Columns | `auto-fit` + `minmax()` |
| Component in narrow slot | Container queries |
| Images overflow | `max-width: 100%` |
| Layout shift | `width`/`height` attrs, `aspect-ratio` |
| Touch targets | `pointer: coarse` query |

## TESTING CHECKLIST

| CHECK | HOW |
|---|---|
| 320px width | DevTools device toolbar |
| 200% / 400% zoom | Browser zoom |
| Long words, translations | Paste long text |
| Landscape phone | Short height |
| Real devices | iOS Safari + Android Chrome |

## ⚠ COMMON MISTAKES

- Hard-coding widths in px that overflow small screens
- Images without dimensions causing content to jump on load (CLS)
- Disabling pinch-zoom in the viewport meta tag

## ✓ TIPS & BEST PRACTICES

- Start with intrinsic layouts; add breakpoints only where needed
- Generate type and space scales with utopia.fyi
- Keep one fluid `--space` token for gutters

## QUICK SUMMARY

- `clamp()` fluid type and space
- `min()` widths
- Media: `max-width: 100%` + aspect
- Breakpoints for structure only

# 2D Transforms

transform · translate() · rotate() · scale() · skew() · transform-origin · individual transform properties · order of functions

## MOVING WITHOUT REFLOW

**`transform`** moves, rotates, scales or skews an element **visually**, without affecting the layout around it. Functions apply **in order**, and each one moves the coordinate system for the ones after it, so `rotate(45deg) translateX(100px)` differs from the reverse. The **individual properties** `translate`, `rotate` and `scale` can be set and animated separately and always apply in that fixed order. Transforms are GPU-friendly, which makes them the go-to for animation.

## NOTES

- `transform-origin` (default `center`) is the pivot for rotate, scale and skew
- Percentages in `translate` refer to the element's *own* size
- Individual properties apply before the `transform` property: translate → rotate → scale → transform
- A transform creates a stacking context and a containing block for fixed children

## ♿ ACCESSIBILITY

- Large or spinning transform animations can trigger vestibular disorders; respect `prefers-reduced-motion`
- Transforms don't change focus order or hit-testing order for keyboard users

## BROWSER SUPPORT

- `transform` 2D functions: universal
- Individual `translate`, `rotate`, `scale`: Baseline 2022

## CODE EXPLAINED

→ `translateX(-50%)` moves by half the element's own width

→ `scaleX(-1)` mirrors horizontally

→ In `.b`, rotation turns the axes first, so the move is diagonal

→ `transform-origin: bottom` pivots a needle from its base

→ Separate `translate` and `scale` can have different transition timings

## TRANSFORM FUNCTIONS

```
.move    { transform: translate(20px, -10px); }
.center-x { transform: translateX(-50%); }   /* own width */
.spin    { transform: rotate(45deg); }
.grow    { transform: scale(1.1); }
.flip    { transform: scaleX(-1); }          /* mirror */
.slant   { transform: skew(-10deg); }

/* order matters */
.a { transform: translateX(100px) rotate(45deg); }
.b { transform: rotate(45deg) translateX(100px); }
/* .b moves along the rotated (diagonal) axis */

/* pivot point */
.door { transform-origin: left center;
        transform: rotateY(30deg); }
.needle { transform-origin: bottom; rotate: -30deg; }

/* individual properties: animate independently */
.card {
  translate: 0 0;
  scale: 1;
  transition: translate .2s, scale .3s;
}
.card:hover { translate: 0 -4px; scale: 1.02; }

/* absolute centering with transform */
.modal {
  position: fixed; top: 50%; left: 50%;
  translate: -50% -50%;
}
```

## 2D FUNCTIONS

| FUNCTION | EXAMPLE |
|---|---|
| `translate(x, y)` | `translate(10px, 50%)` |
| `translateX/Y()` | `translateY(-2px)` |
| `rotate(a)` | `rotate(.25turn)` |
| `scale(x, y)` | `scale(1.5)`, `scale(1, .5)` |
| `skew(x, y)` | `skew(10deg, 0)` |
| `matrix(a,b,c,d,e,f)` | Combined 2D matrix |

## TRANSFORM-ORIGIN

| VALUE | PIVOT |
|---|---|
| `center (default)` | Middle |
| `top left / 0 0` | Top-left corner |
| `bottom` | Bottom center |
| `50% 100%` | Bottom center |
| `20px 40px` | Exact point |

## ⚠ COMMON MISTAKES

- Overwriting a transform on hover and losing the first one (`transform` replaces all functions)
- Animating `left` / `top` instead of `translate`, which forces layout every frame
- Expecting a transform to push neighboring elements

## ✓ TIPS & BEST PRACTICES

- Use individual `translate` / `rotate` / `scale` properties to avoid overwriting
- Animate only `transform` and `opacity` for smooth 60fps motion
- Center with `translate: -50% -50%` when you can't use grid

## QUICK SUMMARY

- `transform` is visual only
- Order of functions matters
- `transform-origin` = pivot
- `translate` / `rotate` / `scale` properties

# 3D Transforms

perspective · perspective-origin · rotateX/Y · translateZ · transform-style: preserve-3d · backface-visibility · card flip

## ADDING DEPTH

3D transforms add a **Z axis** pointing toward the viewer. Without **perspective**, 3D rotation looks flat. Set `perspective` on a parent (shared vanishing point for all children) or the `perspective()` function in a transform (per element); smaller values mean stronger distortion. **`transform-style: preserve-3d`** lets children live in the same 3D space, and **`backface-visibility: hidden`** hides the back of a flipped element, which is how card flips work.

## NOTES

- Typical perspective: 600px–1200px
- `rotateX` tilts forward/back, `rotateY` turns left/right, `rotateZ` = 2D rotate
- `overflow` other than visible on a preserve-3d element flattens it
- `perspective-origin` moves the vanishing point

## ♿ ACCESSIBILITY

- 3D flips and parallax are common motion triggers; disable them under `prefers-reduced-motion: reduce`
- Back-face content is still in the DOM; hide it from screen readers when not shown

## BROWSER SUPPORT

- 3D transforms, `perspective`, `preserve-3d`, `backface-visibility`: Baseline Widely available

## CODE EXPLAINED

- → `perspective` on the scene gives depth to its children
- → `preserve-3d` keeps both faces in one 3D space
- → `rotate: y 180deg` is the individual-property form of `rotateY`
- → `backface-visibility: hidden` shows only the face toward the viewer
- → `perspective()` as a function affects only that element

## CARD FLIP

```
.scene {
  perspective: 800px;              /* depth for children */
}

.card {
  position: relative;
  aspect-ratio: 3 / 4;
  transform-style: preserve-3d;
  transition: rotate .6s;
}

.scene:hover .card,
.card.is-flipped {
  rotate: y 180deg;                /* = rotateY(180deg) */
}

.face {
  position: absolute;
  inset: 0;
  backface-visibility: hidden;
}

.face.back {
  rotate: y 180deg;                /* starts facing away */
}

/* tilt toward the viewer */
.tilt {
  transform: perspective(600px) rotateX(10deg)
             translateZ(20px);
}

@media (prefers-reduced-motion: reduce) {
  .card { transition: none; }
}
```

## 3D FUNCTIONS

| FUNCTION | AXIS |
|---|---|
| `rotateX(a)` | Tilt around horizontal axis |
| `rotateY(a)` | Turn around vertical axis |
| `rotate3d(x, y, z, a)` | Custom axis |
| `translateZ(z)` | Toward / away from viewer |
| `translate3d(x, y, z)` | All three |
| `scaleZ(s)` | Depth scale |
| `perspective(d)` | Per-element depth |

## 3D PROPERTIES

| PROPERTY | PURPOSE |
|---|---|
| `perspective` | Depth for children |
| `perspective-origin` | Vanishing point |
| `transform-style` | `flat` or `preserve-3d` |
| `backface-visibility` | `visible` or `hidden` |
| `transform-origin` | 3D pivot (x y z) |

## ⚠ COMMON MISTAKES

- No perspective: rotations look like squashing, not turning
- Putting `perspective` on the element being rotated instead of its parent
- Adding `overflow: hidden` to a preserve-3d container and flattening it

## ✓ TIPS & BEST PRACTICES

- Use one `perspective` on the parent for consistent scenes
- Keep 3D for small flourishes; it's heavy for large areas
- Hide the hidden face with `aria-hidden` or `inert` when it isn't visible

## QUICK SUMMARY

- Z axis points at the viewer
- `perspective` on the parent
- `preserve-3d` for nested 3D
- `backface-visibility: hidden` for flips

# Transitions

transition-property · duration · timing functions · delay · cubic-bezier() · steps() · transition-behavior · @starting-style

## SMOOTH STATE CHANGES

A **transition** animates a property between its old and new value whenever it changes, for example on hover or when a class is added. Specify which property, how long, the **timing function** (easing) and an optional delay. Only **interpolable** values transition (lengths, colors, numbers, transforms). Newer features let discrete properties like `display` take part (`transition-behavior: allow-discrete`) and define entry styles with **`@starting-style`**.

## NOTES

- Put the transition on the base state so it runs both ways
- List several: `transition: opacity .2s, translate .3s ease-out`
- `transition: all` is easy but animates things you didn't mean to
- `height: auto` can't transition without `interpolate-size` or grid tricks

## ♿ ACCESSIBILITY

- Keep UI transitions short (100–300ms) and wrap larger motion in `prefers-reduced-motion: no-preference`

## BROWSER SUPPORT

- Transitions, `cubic-bezier()`, `steps()`: universal; `linear()` easing: Baseline 2023
- `@starting-style`, `transition-behavior`: Baseline 2024

## CODE EXPLAINED

→ Transition defined on `.btn`, so it plays on hover and un-hover

→ Shorter duration on `:active` feels snappier

→ `cubic-bezier()` defines a custom ease curve

→ `allow-discrete` lets `display` flip at the right moment

→ `@starting-style` sets the "from" values for an element's first render

## TRANSITION RECIPES

```
.btn {
  background: #5b4bc4;
  transition: background-color .2s ease,
              translate .2s ease-out;
}
.btn:hover  { background: #4a3bb0; translate: 0 -2px; }
.btn:active { translate: 0; transition-duration: .05s; }

/* custom easing */
.drawer {
  translate: -100% 0;
  transition: translate .35s cubic-bezier(.2, .8, .2, 1);
}
.drawer.open { translate: 0 0; }

/* fade in a popover, including display */
[popover] {
  opacity: 0;
  transition: opacity .2s, display .2s allow-discrete,
              overlay .2s allow-discrete;
}
[popover]:popover-open { opacity: 1; }
@starting-style {
  [popover]:popover-open { opacity: 0; }
}

/* stepped */
.typing { transition: width 1s steps(20); }

@media (prefers-reduced-motion: reduce) {
  * { transition-duration: 0.01ms !important; }
}
```

## TIMING FUNCTIONS

| VALUE | FEEL |
|---|---|
| `ease (default)` | Quick start, slow end |
| `linear` | Constant speed |
| `ease-in` | Slow start (exits) |
| `ease-out` | Slow end (entrances) |
| `ease-in-out` | Slow both ends |
| `cubic-bezier(x1,y1,x2,y2)` | Custom curve |
| `steps(n, jump-end)` | Discrete jumps |
| `linear(0, .5 50%, 1)` | Piecewise; springs and bounces |

## LONGHANDS

| PROPERTY | EXAMPLE |
|---|---|
| `transition-property` | `opacity, translate` |
| `transition-duration` | `200ms` |
| `transition-timing-function` | `ease-out` |
| `transition-delay` | `100ms` |
| `transition-behavior` | `allow-discrete` |

## ⚠ COMMON MISTAKES

- Putting `transition` only on `:hover`: it animates in but snaps back
- Transitioning `width`, `top` or `margin`, causing layout on every frame
- Expecting `display: none` → block to fade without `allow-discrete` and `@starting-style`

## ✓ TIPS & BEST PRACTICES

- Transition `transform` and `opacity` for the smoothest results
- Use `ease-out` for things entering and `ease-in` for things leaving
- Use DevTools' Animations panel to slow down and inspect transitions

## QUICK SUMMARY

- `transition: prop duration easing delay`
- Set it on the base state
- `cubic-bezier()` · `steps()` · `linear()`
- `@starting-style` for entry

# Keyframe Animations

@keyframes · animation-name · iteration-count · direction · fill-mode · play-state · animation shorthand · multiple animations

## MULTI-STEP MOTION

Where transitions go from A to B on a change, **animations** run a sequence defined by **`@keyframes`**: any number of steps at percentages (`from` = 0%, `to` = 100%). Attach with `animation-name` and a duration; control repetition (`iteration-count`), `direction` (alternate back and forth), and **`fill-mode`**, which decides whether the first or last keyframe applies before and after the animation runs. They start automatically when the style applies.

## NOTES

- Omit a property in a keyframe and it interpolates from the element's own value
- `animation-fill-mode: both` holds the start state during the delay and the end state after
- Several animations: comma-separate them in `animation`
- `animation-play-state: paused` freezes an animation in place

## ♿ ACCESSIBILITY

- Nothing should flash more than 3 times per second (WCAG 2.3.1)
- Infinite animations need a way to stop; wrap them in `prefers-reduced-motion: no-preference` (WCAG 2.2.2)

## BROWSER SUPPORT

- `@keyframes` and `animation-*` : universal
- `animation-composition` : Baseline 2023

## CODE EXPLAINED

- → `from` / `to` are 0% and 100%
- → Shorthand: name, duration, easing, fill-mode
- → Grouped selectors `0%, 100%` share one keyframe
- → `--i` set inline ( `style="--i: 2"` ) staggers the delay
- → In the shorthand the first time is duration, the second is delay

## KEYFRAMES AND SHORTHAND

```
@keyframes fade-up {
  from { opacity: 0; translate: 0 1rem; }
  to   { opacity: 1; translate: 0 0; }
}

.toast {
  animation: fade-up .4s ease-out both;
}

@keyframes pulse {
  0%, 100% { scale: 1; }
  50%      { scale: 1.08; }
}
.badge { animation: pulse 1.5s ease-in-out infinite; }

/* spinner */
@keyframes spin { to { rotate: 1turn; } }
.spinner { animation: spin 1s linear infinite; }

/* staggered list */
.list > li {
  animation: fade-up .4s ease-out both;
  animation-delay: calc(var(--i) * 60ms);
}

/* pause on hover */
.marquee:hover { animation-play-state: paused; }

/* opt in to motion */
@media (prefers-reduced-motion: no-preference) {
  .hero h1 { animation: fade-up .8s .2s ease-out both; }
}
```

## ANIMATION PROPERTIES

| PROPERTY | VALUES |
|---|---|
| `animation-name` | `fade-up`, `none` |
| `animation-duration` | `400ms` |
| `animation-timing-function` | `ease-out`, `steps(8)` |
| `animation-delay` | `200ms` (negative = start partway) |
| `animation-iteration-count` | `3`, `infinite` |
| `animation-direction` | `normal` `reverse` `alternate` |
| `animation-fill-mode` | `none` `forwards` `backwards` `both` |
| `animation-play-state` | `running`, `paused` |

## FILL-MODE

| VALUE | BEFORE | AFTER |
|---|---|---|
| `none` | Element's style | Element's style |
| `forwards` | Element's style | Last keyframe |
| `backwards` | First keyframe (during delay) | Element's style |
| `both` | First keyframe | Last keyframe |

## ⚠ COMMON MISTAKES

- Element jumps back after the animation ends: add `forwards` or `both`
- Flash of final state before a delayed entrance: use `backwards` or `both`
- Animating layout properties like `height` or `left` instead of transforms

## ✓ TIPS & BEST PRACTICES

- Name keyframes by effect ( `fade-up` ), not by element
- Use negative delays to start loaders mid-cycle
- Prefer transitions for simple A→B changes

## QUICK SUMMARY

- `@keyframes name { from {} to {} }`
- `animation: name dur ease delay count dir fill`
- `fill-mode: both` keeps start/end
- Opt in under `no-preference`

# Scroll-Driven Animations

animation-timeline · scroll() · view() · animation-range · scroll-timeline · view-timeline · reduced-motion fallback

## ANIMATION TIED TO SCROLLING

**Scroll-driven animations** replace time with scroll position. Set **`animation-timeline: scroll()`** to drive a normal `@keyframes` animation by how far a scroller has scrolled (reading-progress bars), or **`view()`** to drive it by an element's visibility as it moves through the scrollport (reveal on scroll). **`animation-range`** narrows when it plays, like `entry 0% cover 40%`. No JavaScript, and they run off the main thread.

## NOTES

- Time-based duration doesn't apply on a scroll timeline; the scroll range sets progress
- Declare `animation-timeline` *after* the `animation` shorthand, which resets it
- `scroll(root block)` : the document scroller on the block axis
- Named timelines ( `scroll-timeline` , `view-timeline` ) let other elements follow a scroller

## ♿ ACCESSIBILITY

- Parallax and scroll-linked motion can cause nausea; disable under `prefers-reduced-motion: reduce`
- Never hide content until it scrolls into view without a fallback; unsupported browsers must still show it

## BROWSER SUPPORT

- Scroll-driven animations: not yet in every engine (Chromium first, then Safari); Firefox status: check
- Wrap in `@supports (animation-timeline: view())` so other browsers get static content

## CODE EXPLAINED

- → `scroll(root block)` : progress of the page's vertical scroll
- → `scale: 0 1 → 1 1` grows the bar horizontally from the left
- → `view()` : progress of the card crossing the viewport
- → `entry 0% cover 30%` : run from first appearance until 30% through
- → `scroll-timeline: --gallery inline` names a horizontal scroller's timeline

## PROGRESS BAR AND REVEAL

```
/* reading progress bar */
@keyframes grow { from { scale: 0 1; } to { scale: 1 1; } }

.progress {
  position: fixed; inset: 0 0 auto;
  height: 4px; background: #5b4bc4;
  transform-origin: left;
  animation: grow linear both;
  animation-timeline: scroll(root block);
}

/* reveal as each card enters the viewport */
@keyframes reveal {
  from { opacity: 0; translate: 0 2rem; }
  to   { opacity: 1; translate: 0 0; }
}

@supports (animation-timeline: view()) {
  @media (prefers-reduced-motion: no-preference) {
    .card {
      animation: reveal linear both;
      animation-timeline: view();
      animation-range: entry 0% cover 30%;
    }
  }
}

/* named timeline used by another element */
.gallery { scroll-timeline: --gallery inline; }
.gallery-dots { animation: grow linear both;
                animation-timeline: --gallery; }
```

## TIMELINES

| VALUE | DRIVEN BY |
|---|---|
| `scroll()` | Nearest ancestor scroller |
| `scroll(root)` | Document scroll |
| `scroll(self x)` | Own horizontal scroll |
| `view()` | Element's visibility in its scroller |
| `view(inline)` | Same, horizontal |
| `--name` | A named scroll or view timeline |

## ANIMATION-RANGE

| RANGE | PHASE |
|---|---|
| `cover` | First pixel in → last pixel out |
| `contain` | Fully inside the scrollport |
| `entry` | Crossing the start edge |
| `exit` | Crossing the end edge |
| `entry 0% cover 50%` | From appearing to halfway |

## ⚠ COMMON MISTAKES

- Setting `animation-timeline` before the `animation` shorthand, which resets it to `auto`
- Starting content at `opacity: 0` without `@supports` , so it stays invisible in other browsers
- Giving the animation an easing other than `linear` and getting odd scroll feel

## ✓ TIPS & BEST PRACTICES

- Use `linear` timing so motion tracks the scroll directly
- Keep keyframes to `transform` and `opacity` for smoothness
- Use the Scroll-Driven Animations DevTools extension to debug ranges

## QUICK SUMMARY

- `animation-timeline: scroll()` / `view()`
- `animation-range` limits the phase
- Declare timeline after the shorthand
- `@supports` + reduced-motion guard

# View Transitions

@view-transition · view-transition-name · ::view-transition-old · ::view-transition-new · ::view-transition-group · startViewTransition()

## ANIMATING BETWEEN STATES

The **View Transition API** animates between two states of a page: the browser snapshots the old state, you update the DOM, and it crossfades to the new state. Elements given a **`view-transition-name`** morph individually (a thumbnail growing into a hero image). **Same-document** transitions start with `document.startViewTransition(update)` in JS; **cross-document** transitions between pages of one site only need `@view-transition { navigation: auto; }` in CSS on both pages.

## NOTES

- Each `view-transition-name` must be unique on the page at that moment
- The default is a crossfade; customize it through the `::view-transition-*` pseudo-elements
- `view-transition-class` styles several named transitions at once
- The update still happens instantly where unsupported, so it's safe progressive enhancement

## ♿ ACCESSIBILITY

- Turn off or simplify view transitions under `prefers-reduced-motion: reduce`
- Move focus to the new content after a same-document transition

## BROWSER SUPPORT

- Same-document view transitions: Chromium and Safari 18+, newer elsewhere; check before relying on them
- Cross-document ( `@view-transition` ): Chromium and Safari, newer; ignored elsewhere

## CODE EXPLAINED

- → `navigation: auto` enables transitions for same-origin page loads
- → Matching names on both pages make the image morph
- → `root` is the whole page's snapshot
- → `::view-transition-group()` controls size and position animation
- → Feature-check `startViewTransition` and fall back to a plain update

## SAME- AND CROSS-DOCUMENT

```
/* both pages: opt in to cross-page transitions */
@view-transition { navigation: auto; }

/* morph the product image between pages */
.product-thumb, .product-hero {
  view-transition-name: product-photo;
}

/* customize the default crossfade */
::view-transition-old(root) {
  animation: .25s ease-in both fade-out;
}
::view-transition-new(root) {
  animation: .35s ease-out both slide-in;
}

/* shape morph timing */
::view-transition-group(product-photo) {
  animation-duration: .4s;
}

@media (prefers-reduced-motion: reduce) {
  ::view-transition-group(*),
  ::view-transition-old(*),
  ::view-transition-new(*) { animation: none !important; }
}

/* same-document (JS):
   if (!document.startViewTransition) update();
   else document.startViewTransition(() => update()); */
```

## PSEUDO-ELEMENT TREE

| PSEUDO | REPRESENTS |
|---|---|
| `::view-transition` | Overlay root |
| `::view-transition-group(name)` | Animates size/position |
| `::view-transition-image-pair(name)` | Holds old + new |
| `::view-transition-old(name)` | Snapshot before |
| `::view-transition-new(name)` | Live view after |

## KEY PIECES

| ITEM | PURPOSE |
|---|---|
| `view-transition-name` | Tag an element to morph |
| `view-transition-class` | Group names for styling |
| `@view-transition` | Cross-document opt-in |
| `startViewTransition()` | Same-document trigger |
| `:active-view-transition` | Style during a transition |

## ⚠ COMMON MISTAKES

- Two visible elements with the same `view-transition-name` : the transition is skipped
- Forgetting the opt-in on the *destination* page for cross-document transitions
- Long or bouncy page transitions that slow navigation down

## ✓ TIPS & BEST PRACTICES

- Start with the default crossfade, then name one or two hero elements
- Keep transitions under ~400ms
- Assign names dynamically (inline style) for list items so they stay unique

## QUICK SUMMARY

- `@view-transition { navigation: auto }`
- `view-transition-name` morphs elements
- `::view-transition-old/new/group`
- Progressive enhancement by default

# Cursors & Pointer Interaction

cursor · pointer-events · user-select · touch-action · caret-color · resize · appearance

## HOW ELEMENTS RESPOND TO INPUT

A handful of properties shape interaction rather than looks. **`cursor`** shows what a click will do. **`pointer-events: none`** lets clicks pass through an element to what's beneath. **`user-select`** controls whether text can be selected, **`touch-action`** tells the browser which touch gestures to handle itself (pan, zoom) so your JS gets the rest, **`resize`** adds a drag handle, and **`appearance: none`** strips native control styling so you can restyle it.

## NOTES

- `cursor: pointer` is a convention for links and buttons only
- `pointer-events: none` doesn't stop keyboard focus or activation
- `resize` only works when `overflow` is not `visible`
- `touch-action: manipulation` removes double-tap-to-zoom delay on buttons

## ♿ ACCESSIBILITY

- Never use `pointer-events: none` as a way to disable controls; use `disabled` or `inert` so keyboards are blocked too
- `user-select: none` on content frustrates people who copy text to translate or read aloud

## BROWSER SUPPORT

- `cursor`, `pointer-events`, `resize`, `caret-color`, `touch-action`, `appearance`: universal
- `user-select`: Safari still needs `-webkit-user-select`

## CODE EXPLAINED

→ `not-allowed` signals a disabled control

→ Custom cursor: image, hotspot x y, then a keyword fallback

→ Children can re-enable `pointer-events`

→ `user-select: all` selects the whole snippet in one click

→ `touch-action: pan-y` lets the page scroll while JS handles horizontal swipes

## INTERACTION PROPERTIES

```
a, button, [role="button"], label[for] { cursor: pointer; }
[disabled], [aria-disabled="true"] { cursor: not-allowed; }
.draggable        { cursor: grab; }
.draggable:active { cursor: grabbing; }
.help             { cursor: help; }

/* custom cursor with fallback */
.paint { cursor: url("brush.svg") 4 28, crosshair; }

/* overlay that doesn't block clicks */
.gradient-overlay { pointer-events: none; }
.gradient-overlay a { pointer-events: auto; }

/* no text selection on UI chrome */
.toolbar {
  -webkit-user-select: none;
  user-select: none;
}
.code-snippet { user-select: all; } /* select on click */

/* gestures */
.map      { touch-action: none; }       /* JS handles all */
.carousel { touch-action: pan-y; }      /* page scrolls */
button    { touch-action: manipulation; }

textarea { resize: vertical; }
input    { caret-color: #5b4bc4; }
select   { appearance: none; }          /* restyle */
```

## COMMON CURSORS

| VALUE | MEANING |
|---|---|
| `default` | Arrow |
| `pointer` | Link / clickable |
| `text` | Text can be selected |
| `not-allowed` | Action unavailable |
| `grab / grabbing` | Draggable / dragging |
| `move` | Move the item |
| `wait / progress` | Busy / busy but usable |
| `zoom-in / zoom-out` | Zoomable |
| `ew-resize / ns-resize` | Resize handles |

## TOUCH-ACTION

| VALUE | BROWSER HANDLES |
|---|---|
| `auto` | All gestures |
| `none` | Nothing; JS gets all |
| `pan-x / pan-y` | Only that scroll direction |
| `pinch-zoom` | Zoom only |
| `manipulation` | Pan + zoom, no double-tap delay |

## ⚠ COMMON MISTAKES

- Disabling with `pointer-events: none` while the control is still keyboard-operable
- `user-select: none` on articles and documentation
- Removing `appearance` from checkboxes without adding visible checked and focus states

## ✓ TIPS & BEST PRACTICES

- Pair custom cursors with a keyword fallback
- Use `touch-action` instead of `preventDefault()` in touch handlers
- Limit `resize` to `vertical` for textareas in layouts

## QUICK SUMMARY

- `cursor` signals the action
- `pointer-events: none` = click-through
- `user-select` / `touch-action`
- `appearance: none` to restyle

# Styling Form Controls

appearance: none · accent-color · ::placeholder · ::file-selector-button · field-sizing · custom checkboxes · :user-invalid styling

## MAKING NATIVE CONTROLS FIT

Form controls are rendered partly by the OS, so styling them takes a few specific tools. **`accent-color`** recolors checkboxes, radios, range sliders and progress bars in one line. For full control, **`appearance: none`** removes native styling from the *real* input, which you then rebuild with borders, backgrounds and `:checked` states, keeping it accessible. **`field-sizing: content`** lets inputs and textareas grow with what's typed.

## NOTES

- Always style the native element; hidden inputs + fake divs lose keyboard and screen-reader support
- Set `font: inherit` on controls (they don't inherit by default)
- `::file-selector-button` styles the button inside `<input type="file">`
- Customizable `<select>` ( `appearance: base-select` ) is arriving; plain `select` styling is limited

## ♿ ACCESSIBILITY

- Every control needs a visible label, a 3:1 border contrast against the background (WCAG 1.4.11) and a focus style
- Show errors with text and an icon, not only a red border

## BROWSER SUPPORT

- `accent-color` , `appearance` , `::placeholder` , `::file-selector-button` : Baseline
- `field-sizing` : Chromium only so far; `appearance: base-select` : newer, Chromium first

## CODE EXPLAINED

- → `accent-color` themes native checkboxes and radios
- → `#767676` is the lightest gray with 4.5:1 on white
- → `field-sizing: content` grows the textarea as text is typed
- → `appearance: none` keeps the real checkbox, focusable and announced
- → `::before` draws the check mark; `:checked` reveals it

## CONTROLS STYLED ACCESSIBLY

```
input, select, textarea, button { font: inherit; }

form { accent-color: #5b4bc4; }        /* quick theming */

.input {
  border: 1px solid #767676;           /* 4.5:1 on white */
  border-radius: 6px;
  padding: .5rem .75rem;
}
.input:focus-visible { outline: 3px solid #5b4bc4; }
.input:user-invalid  { border-color: #c62828; }
.input::placeholder  { color: #6b6b6b; }

textarea { field-sizing: content; min-block-size: 3lh; }

/* custom checkbox on the real input */
.check {
  appearance: none;
  width: 1.25rem; height: 1.25rem;
  border: 2px solid currentColor;
  border-radius: 4px;
  display: inline-grid; place-content: center;
}
.check::before {
  content: ""; width: .7rem; height: .7rem;
  clip-path: polygon(14% 44%, 0 65%, 50% 100%,
                     100% 16%, 80% 0, 43% 62%);
  background: currentColor; scale: 0;
}
.check:checked::before { scale: 1; }
.check:focus-visible   { outline: 2px solid #5b4bc4;
                         outline-offset: 2px; }

input[type=file]::file-selector-button {
  border: 0; padding: .5rem 1rem; border-radius: 6px;
}
```

## HOOKS

| SELECTOR / PROPERTY | STYLES |
|---|---|
| `accent-color` | Checkbox, radio, range, progress |
| `appearance: none` | Removes native look |
| `::placeholder` | Placeholder text |
| `::file-selector-button` | File input button |
| `::-webkit-slider-thumb` | Range thumb (prefixed) |
| `caret-color` | Text cursor |
| `:autofill` | Browser-filled fields |

## CHECKLIST

| REQUIREMENT | HOW |
|---|---|
| `Label` | `<label for>` always |
| `Border contrast` | 3:1 against surroundings |
| `Focus` | `:focus-visible` outline |
| `Error` | `:user-invalid` + message text |
| `Disabled` | Visible but clearly muted |
| `Touch size` | ≥ 24px, ideally 44px |

## ⚠ COMMON MISTAKES

- Hiding the real checkbox and styling a `<span>` : not focusable or announced
- Light-gray borders (#ddd) that fail 3:1 non-text contrast
- Placeholders used as labels; they disappear on typing

## ✓ TIPS & BEST PRACTICES

- Start with `accent-color` ; go custom only when needed
- Keep custom controls as real inputs with `appearance: none`
- Test controls in forced-colors mode

## QUICK SUMMARY

- `accent-color` quick theme
- `appearance: none` on the real input
- `font: inherit`
- Labels, contrast, focus, errors

# Scrolling & Scroll Snap

scroll-behavior · scroll-margin · scroll-padding · scroll-snap-type · scroll-snap-align · overscroll-behavior · scrollbar-color

## CONTROLLING SCROLL

CSS can shape scrolling without JS. **`scroll-behavior: smooth`** animates jumps to anchors. **`scroll-padding`** (on the scroller) and **`scroll-margin`** (on targets) offset where things land, for example below a sticky header. **Scroll snap** makes a scroller stop at defined points: `scroll-snap-type` on the container, `scroll-snap-align` on children; it's the basis of CSS-only carousels. **`overscroll-behavior`** stops scroll chaining into the page behind a modal or sidebar.

## NOTES

- `mandatory` always snaps; `proximity` snaps only near a snap point
- `scroll-snap-stop: always` prevents fast swipes skipping items
- `scrollbar-width: thin` and `scrollbar-color: thumb track` style scrollbars
- `overscroll-behavior: contain` also disables pull-to-refresh inside that element

## ♿ ACCESSIBILITY

- Only enable smooth scrolling under `prefers-reduced-motion: no-preference`
- Carousels need visible controls and keyboard access; snap alone isn't enough
- Don't hide scrollbars on content that needs scrolling to be discovered

## BROWSER SUPPORT

- `scroll-behavior`, `scroll-margin/padding`, scroll snap: Baseline Widely available
- `overscroll-behavior`: Baseline 2022; `scrollbar-width`: Baseline 2024; `scrollbar-color`: not in Safari

## CODE EXPLAINED

- → Smooth scrolling only when motion is welcome
- → `scroll-padding-top` on the scroller offsets every anchor target
- → `x mandatory` + `align: start` snaps each slide to the left edge
- → `scroll-snap-stop: always` stops one slide at a time
- → `overscroll-behavior: contain` stops scroll chaining to the page

## SCROLL RECIPES

```
/* smooth anchor jumps, opt-in */
@media (prefers-reduced-motion: no-preference) {
  html { scroll-behavior: smooth; }
}

/* anchors land below a 4rem sticky header */
html { scroll-padding-top: 4rem; }
h2   { scroll-margin-top: 1rem; }      /* extra space */

/* horizontal carousel */
.carousel {
  display: flex;
  gap: 1rem;
  overflow-x: auto;
  scroll-snap-type: x mandatory;
  scroll-padding-inline: 1rem;
  overscroll-behavior-x: contain;
}
.carousel > * {
  flex: 0 0 80%;
  scroll-snap-align: start;
  scroll-snap-stop: always;
}

/* vertical full-screen sections */
.slides { height: 100dvh; overflow-y: auto;
          scroll-snap-type: y proximity; }
.slides > section { height: 100dvh;
                    scroll-snap-align: start; }

/* modal panel doesn't scroll the page */
.drawer { overflow-y: auto; overscroll-behavior: contain; }

/* thin themed scrollbars */
.panel { scrollbar-width: thin;
         scrollbar-color: #5b4bc4 transparent; }
```

## SNAP PROPERTIES

| PROPERTY | VALUES |
|---|---|
| `scroll-snap-type` | `x` `y` `both` + `mandatory` / `proximity` |
| `scroll-snap-align` | `start` `center` `end` |
| `scroll-snap-stop` | `normal` `always` |
| `scroll-padding` | Inset of the snap area (container) |
| `scroll-margin` | Outset of a target (child) |

## OTHER SCROLL PROPS

| PROPERTY | USE |
|---|---|
| `scroll-behavior` | `smooth` anchor/JS scrolls |
| `overscroll-behavior` | `contain`, `none` |
| `scrollbar-gutter` | Reserve scrollbar space |
| `scrollbar-width` | `auto` `thin` `none` |
| `scrollbar-color` | Thumb and track colors |
| `-webkit-overflow-scrolling` | Obsolete; remove |

## ⚠ COMMON MISTAKES

- Setting `scroll-padding` on the target instead of the scroll container (use `scroll-margin` there)
- `mandatory` snapping on content taller than the viewport, which traps users
- Hiding scrollbars with `scrollbar-width: none` on scrollers without other affordances

## ✓ TIPS & BEST PRACTICES

- Use `scroll-padding-top` for sticky headers instead of JS offsets
- Use `proximity` for long content, `mandatory` for slides
- Add `overscroll-behavior: contain` to modals and drawers

## QUICK SUMMARY

- `scroll-behavior: smooth` (opt-in)
- `scroll-padding` on scroller, `scroll-margin` on target
- `scroll-snap-type` + `scroll-snap-align`
- `overscroll-behavior: contain`

# Anchor Positioning & Top Layer

anchor-name · position-anchor · anchor() · position-area · position-try · dialog & popover styling · ::backdrop

## TOOLTIPS AND MENUS WITHOUT JS MATH

**Anchor positioning** places an absolutely positioned element relative to another element anywhere in the page. Name the anchor with **`anchor-name`**, point at it with **`position-anchor`**, then place with **`position-area`** (a 3×3 grid around the anchor) or the **`anchor()`** function in inset properties. **`position-try-fallbacks`** flips the element when it would overflow. Combined with **popovers** and **dialogs**, which live in the **top layer**, it covers tooltips, menus and modals.

## NOTES

- Anchor names are dashed idents: `anchor-name: --menu-btn`
- `position-area: bottom span-right` places below, aligned to the anchor's left edge
- Top-layer elements ignore `z-index` and ancestors' `overflow` and transforms
- `::backdrop` styles the layer behind a modal dialog or popover

## ♿ ACCESSIBILITY

- Use `<button popovertarget>` and `<dialog>` : they wire up focus, Escape and ARIA for you
- Tooltips must show on focus as well as hover, and be dismissible (WCAG 1.4.13)

## BROWSER SUPPORT

- `<dialog>` , `::backdrop` : Baseline 2022; Popover API: Baseline 2024
- Anchor positioning: newer, Chromium and Safari 26+; check support and provide a fallback position

## CODE EXPLAINED

- → `popovertarget` toggles the popover; no JS needed
- → `anchor-name` on the button, `position-anchor` on the menu
- → `position-area` positions in the grid around the anchor
- → `flip-block` moves the menu above if there's no room below
- → `anchor(top)` = the anchor's top edge, usable in any inset

## ANCHORED POPOVER MENU

```
/* HTML:
   <button popovertarget="menu" class="menu-btn">
     Options</button>
   <div id="menu" popover>…</div> */

.menu-btn { anchor-name: --menu-btn; }

#menu {
  position-anchor: --menu-btn;
  position-area: bottom span-right; /* below, left edge */
  margin: 0;
  margin-top: .25rem;
  position-try-fallbacks: flip-block, flip-inline;
}

/* with anchor() for exact edges */
.tooltip {
  position: absolute;
  position-anchor: --target;
  bottom: anchor(top);
  left: anchor(center);
  translate: -50% -.5rem;
}

/* fallback for browsers without anchors */
@supports not (anchor-name: --a) {
  #menu { inset: auto; top: 3rem; left: 1rem; }
}

/* modal dialog */
dialog { border: 0; border-radius: 12px; padding: 1.5rem;
         max-width: min(32rem, 100% - 2rem); }
dialog::backdrop { background: rgb(0 0 0 / .5);
                   backdrop-filter: blur(2px); }
```

## ANCHOR PROPERTIES

| PROPERTY | PURPOSE |
|---|---|
| `anchor-name` | Name an element as an anchor |
| `position-anchor` | Default anchor to position against |
| `position-area` | 3×3 grid placement |
| `anchor(side)` | Edge position in insets |
| `anchor-size()` | Anchor's width/height |
| `position-try-fallbacks` | Alternatives on overflow |
| `position-visibility` | Hide when anchor scrolls away |

## TOP-LAYER ELEMENTS

| ELEMENT | OPENED WITH |
|---|---|
| `Modal dialog` | `dialog.showModal()` |
| `Non-modal dialog` | `dialog.show()` (not top layer) |
| `Popover` | `popovertarget` or `showPopover()` |
| `Fullscreen` | `requestFullscreen()` |
| `::backdrop` | Behind modal dialog/popover |

## ⚠ COMMON MISTAKES

- Building tooltips that only appear on hover
- Fighting `z-index` for modals instead of using `<dialog>` + `showModal()`
- No fallback position where anchor positioning isn't supported

## ✓ TIPS & BEST PRACTICES

- Pair popover + anchor positioning for menus and tooltips
- Add `position-try-fallbacks` so menus stay on screen
- Use the Floating UI library where you need anchoring everywhere today

## QUICK SUMMARY

- `anchor-name` + `position-anchor`
- `position-area` or `anchor()`
- `position-try-fallbacks` flips
- Top layer: dialog, popover

# Organizing Stylesheets

file structure · base/layout/components/utilities · @layer order · @import vs bundling · DRY tokens

## STRUCTURE THAT SCALES

Large stylesheets stay manageable when they're split by **job** and ordered from **generic to specific**: tokens and resets, base element styles, layout primitives, components, then utilities. **Cascade layers** encode that order so specificity stops mattering between groups. Methodologies like ITCSS formalize the idea (see the CSS Systems set); plain CSS with layers, custom properties and nesting gets you most of the way.

## NOTES

- One component per file makes ownership and deletion easy
- Tokens (custom properties) remove repeated magic numbers
- A bundler (Vite, Lightning CSS) combines partials; avoid runtime `@import` chains
- Utilities go last so they can override components

## ♿ ACCESSIBILITY

- Keep focus styles, reduced-motion and forced-colors rules in base layers so every component inherits them

## BROWSER SUPPORT

- `@layer` : Baseline 2022; native nesting: Baseline 2023

## CODE EXPLAINED

- → The `@layer` statement fixes the priority order once
- → Each import goes into its layer, so file order matters less
- → Tokens hold every reusable value
- → Layout primitives ( `stack` , `grid` ) stay separate from components
- → Utilities win because their layer is last

## A LAYERED PROJECT

```
/* main.css */
@layer reset, tokens, base, layout, components, utilities;

@import url("reset.css")             layer(reset);
@import url("tokens.css")            layer(tokens);
@import url("base.css")              layer(base);
@import url("layout/stack.css")      layer(layout);
@import url("layout/grid.css")       layer(layout);
@import url("components/card.css")   layer(components);
@import url("components/btn.css")    layer(components);
@import url("utilities.css")         layer(utilities);

/* tokens.css */
:root {
  --space-1: .25rem;  --space-2: .5rem;  --space-3: 1rem;
  --radius: 8px;
  --brand: oklch(58% .18 285);
}

/* layout/stack.css */
.stack > * + * { margin-block-start: var(--space, 1rem); }

/* components/card.css */
.card {
  padding: var(--space-3);
  border-radius: var(--radius);
  & .card__title { margin: 0; }
}

/* utilities.css */
.visually-hidden { /* see sheet 75 */ }
.mt-0 { margin-block-start: 0; }
```

## TYPICAL FOLDERS

| FOLDER | CONTAINS |
|---|---|
| tokens/ | Colors, spacing, type scale |
| base/ | Reset, element defaults, typography |
| layout/ | Stack, cluster, grid, sidebar |
| components/ | One file per component |
| utilities/ | Single-purpose helpers |
| pages/ | Rare page-specific overrides |

## GENERIC → SPECIFIC

| LAYER | EXAMPLE |
|---|---|
| reset | `* { margin: 0 }` |
| tokens | `--brand`, `--space-3` |
| base | `a`, `h1`, `body` |
| layout | `.stack`, `.grid` |
| components | `.card`, `.btn` |
| utilities | `.mt-0`, `.text-center` |

## ⚠ COMMON MISTAKES

- One huge `styles.css` where nobody knows what's safe to delete
- Page-specific overrides scattered through component files
- Shipping many `@import` s unbundled to production

## ✓ TIPS & BEST PRACTICES

- Declare layer order in one place
- Delete a component's CSS file when the component goes
- See CSS Systems (ITCSS, BEM, Sass) for team-scale methods

## QUICK SUMMARY

- Generic → specific
- `@layer` encodes the order
- Tokens for repeated values
- Bundle partials for production

# Naming Conventions

BEM · block__element--modifier · utility classes · state classes (.is-) · js-hooks · data-attribute state

## NAMES THAT EXPLAIN THEMSELVES

Class names are the API between HTML and CSS. **BEM** (Block, Element, Modifier) makes relationships explicit: `.card` is a block, `.card__title` an element inside it, `.card--featured` a variation. **Utility classes** do one thing each (`.flex`, `.mt-2`). **State** is best expressed with `.is-*` classes or, better, ARIA and `data-*` attributes that also carry meaning. Keep **`js-*`** hooks separate from styling so refactors don't break scripts.

## NOTES

- BEM keeps every selector a single class, so specificity stays flat
- Name by purpose ( `.btn--danger` ), not appearance ( `.btn--red` )
- Never nest BEM elements in names: `.card__body__title` → `.card__title`
- Utility-first frameworks are covered in the CSS Systems set

## ♿ ACCESSIBILITY

- Style from ARIA state ( `[aria-expanded="true"]` ) so the visual state can't disagree with what screen readers announce

## BROWSER SUPPORT

- Naming is a convention; nothing here depends on browser support

## CODE EXPLAINED

- → `.card__title` : element of the card block
- → `.card--featured` : a variant of the same block
- → `.is-open` describes temporary state
- → ARIA and `data-*` state doubles as accessible meaning
- → `js-` classes are for scripts only, so styling changes can't break JS

## BEM, STATE, HOOKS

```
/* block */
.card { border: 1px solid #ddd; }

/* elements */
.card__media { aspect-ratio: 16 / 9; }
.card__title { font-size: 1.25rem; }
.card__body  { padding: 1rem; }

/* modifiers */
.card--featured { border-color: gold; }
.card--compact .card__body { padding: .5rem; }

/* state classes */
.accordion.is-open   { }
.btn.is-loading      { cursor: progress; }

/* prefer attribute state */
.accordion[data-state="open"] { }
.menu-btn[aria-expanded="true"] .icon { rotate: 180deg; }
.tab[aria-selected="true"] { border-bottom: 2px solid; }

/* utilities */
.flex   { display: flex; }
.gap-2  { gap: .5rem; }
.sr-only { /* visually hidden */ }

/* JS hook: never styled */
/* <button class="btn js-open-cart"> */
```

## BEM SYNTAX

| PART | PATTERN |
|---|---|
| Block | `.block` |
| Element | `.block__element` |
| Modifier | `.block--modifier` |
| Element modifier | `.block__element--modifier` |
| Multi-word | `.site-header__nav-link` |

## NAME TYPES

| PREFIX | MEANING |
|---|---|
| c- / none | Component |
| l- / o- | Layout object |
| u- | Utility |
| is- / has- | State |
| js- | JavaScript hook |
| t- | Theme |

## ⚠ COMMON MISTAKES

- Naming by looks ( `.blue-box` ) and renaming everything after a redesign
- Deeply chained BEM like `.a__b__c__d`
- Styling `js-*` classes, then breaking styles when JS changes

## ✓ TIPS & BEST PRACTICES

- Pick one convention and write it down
- Use ARIA attributes for state when they apply
- Keep class names lowercase with hyphens

## QUICK SUMMARY

- `.block__element--modifier`
- Purpose over appearance
- `.is-*` or ARIA for state
- `js-*` hooks unstyled

# Accessible CSS

focus indicators · .visually-hidden · contrast ratios · prefers-reduced-motion · forced-colors · visual vs DOM order

## CSS THAT INCLUDES EVERYONE

CSS can make an accessible page unusable or a rough one much better. The essentials: **visible focus** on every interactive element, **sufficient contrast**, **no information by color alone**, **respect for user preferences** (reduced motion, contrast, forced colors), content that **reflows** at 320px and 400% zoom, and a **visual order** that matches the DOM. The **visually-hidden** pattern hides text visually while keeping it for screen readers.

## NOTES

- WCAG contrast: 4.5:1 normal text, 3:1 large text (≥24px or ≥18.66px bold) and UI parts
- Focus indicators should be at least 2px and contrast 3:1 (WCAG 2.4.13)
- CSS-generated content may or may not be announced; keep meaning in HTML
- `display: none` and `visibility: hidden` remove content from the accessibility tree

## ♿ ACCESSIBILITY

- This whole sheet is the accessibility checklist; test with a keyboard, zoom and a screen reader
- Check forced-colors mode (Windows contrast themes) and dark mode

## BROWSER SUPPORT

- Everything here: Baseline Widely available; `forced-colors` : check Safari

## CODE EXPLAINED

→ `:focus-visible` gives keyboard users a strong ring

→ The visually-hidden pattern keeps text in the accessibility tree

→ `:not(:focus)` lets a hidden skip link appear when tabbed to

→ Reduced motion shortens animations to near-instant

→ System colors keep borders visible in forced-colors mode

## ACCESSIBILITY TOOLKIT

```css
/* 1. visible focus everywhere */
:focus-visible {
  outline: 3px solid var(--focus, #ff9800);
  outline-offset: 2px;
}

/* 2. hide visually, keep for screen readers */
.visually-hidden:not(:focus):not(:active) {
  position: absolute;
  width: 1px; height: 1px;
  margin: -1px; padding: 0; border: 0;
  overflow: hidden;
  clip-path: inset(50%);
  white-space: nowrap;
}

/* 3. skip link appears on focus */
.skip-link { position: absolute; translate: 0 -200%; }
.skip-link:focus { translate: 0 0; }

/* 4. motion opt-in */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}

/* 5. forced colors: keep boundaries */
@media (forced-colors: active) {
  .btn, .card { border: 1px solid CanvasText; }
}

/* 6. links distinguishable without color */
p a { text-decoration: underline; }
```

## WCAG CHECKS FOR CSS

| CRITERION | REQUIREMENT |
|---|---|
| 1.4.1 Use of Color | Not color alone |
| 1.4.3 Contrast | 4.5:1 text, 3:1 large |
| 1.4.10 Reflow | No 2D scroll at 320px |
| 1.4.11 Non-text Contrast | 3:1 for UI parts |
| 1.4.12 Text Spacing | Survive increased spacing |
| 2.3.3 Animation | Disable motion on request |
| 2.4.7 / 2.4.13 Focus | Visible, strong indicator |
| 1.3.2 Meaningful Sequence | Visual = DOM order |

## HIDING TECHNIQUES

| NEED | USE |
|---|---|
| Hide from everyone | `display: none` / `hidden` |
| Hide visually only | `.visually-hidden` |
| Hide from AT only | `aria-hidden="true"` (HTML) |
| Disable a region | `inert` (HTML) |

## ⚠ COMMON MISTAKES

- `outline: none` without a replacement
- Reordering with `order`, `grid-area` or `flex-direction: *-reverse` so tab order jumps around
- Light gray placeholder or body text that fails contrast

## ✓ TIPS & BEST PRACTICES

- Tab through every page after styling it
- Use DevTools: contrast in the color picker, Rendering → emulate vision deficiencies
- Keep the visually-hidden utility in your base layer

## QUICK SUMMARY

- Visible `:focus-visible` always
- 4.5:1 text, 3:1 UI
- Respect reduced motion + forced colors
- Visual order = DOM order

# Debugging & Performance

DevTools Styles/Computed · layout shift · will-change · contain · content-visibility · expensive properties · Stylelint

## FINDING AND FIXING PROBLEMS

**DevTools** is the main debugging tool: the Styles pane shows every matching rule (losers struck through), Computed shows final values and where they came from, and overlays visualize grid, flex and the box model. For performance, the rendering pipeline is **style → layout → paint → composite**. Animating properties that only need compositing (`transform`, `opacity`) is cheap; properties that trigger layout are expensive. **Containment** tells the browser a subtree is independent.

## NOTES

- Struck-through declaration: overridden; yellow warning icon: invalid or ignored
- Layout shift (CLS) comes from late-loading images, fonts and injected content
- `will-change` reserves resources; add it just before animating, not everywhere
- Selector matching is rarely the bottleneck; layout and paint usually are

## ♿ ACCESSIBILITY

- Use DevTools' accessibility pane and contrast checker while debugging styles
- Emulate `prefers-reduced-motion` and forced colors in the Rendering tab

## BROWSER SUPPORT

- `contain` : Baseline 2022; `content-visibility` : Baseline 2024; `will-change` : universal

## CODE EXPLAINED

- → A translucent outline on everything shows which box overflows
- → `translate` and `opacity` skip layout and paint
- → Animating `top` / `height` re-runs layout each frame
- → `contain: layout paint` limits how far changes ripple
- → HTML `width` / `height` plus `height: auto` reserves image space; `min-height` reserves ad space

## DEBUG AND SPEED UP

```
/* outline every box to find overflow */
* { outline: 1px solid rgb(255 0 0 / .3); }

/* find the element causing horizontal scroll */
/* DevTools console:
   [...document.querySelectorAll('*')]
     .filter(e => e.scrollWidth > innerWidth) */

/* cheap animation: compositor only */
.toast { transition: translate .3s, opacity .3s; }

/* expensive: layout every frame */
.toast-bad { transition: top .3s, height .3s; }

/* hint right before animating, then remove */
.drawer.is-opening { will-change: translate; }

/* isolate a widget's layout and paint */
.widget { contain: layout paint; }

/* skip rendering off-screen sections */
.feed-item {
  content-visibility: auto;
  contain-intrinsic-size: auto 400px;
}

/* stop layout shift: <img width="800" height="450"> */
img { max-width: 100%; height: auto; }  /* keeps ratio */
.ad-slot { min-height: 250px; }   /* reserve space */
```

## PROPERTY COST

| CHANGES | TRIGGERS |
|---|---|
| `transform, opacity, filter` | Composite only |
| `color, background, box-shadow` | Paint |
| `width, height, margin, top` | Layout + paint |
| `font-size, display` | Layout + paint |
| `content-visibility: auto` | Skips off-screen work |

## DEVTOOLS PANELS

| PANEL | USE |
|---|---|
| `Styles` | Matching rules, overrides, edit live |
| `Computed` | Final values + source |
| `Layout` | Grid/flex overlays |
| `Rendering` | Emulate media, paint flashing |
| `Performance` | Record layout/paint costs |
| `CSS Overview` | Colors, fonts, unused declarations |
| `Coverage` | Unused CSS bytes |

## ⚠ COMMON MISTAKES

- `will-change: transform` on hundreds of elements, wasting memory
- Animating `height` or `box-shadow` on large elements
- Guessing at overrides instead of reading the Styles pane

## ✓ TIPS & BEST PRACTICES

- Turn on paint flashing to see what repaints
- Lint with Stylelint to catch invalid values before they ship
- Use the Coverage panel to find unused CSS

## QUICK SUMMARY

- Styles + Computed panes first
- Animate `transform` and `opacity`
- `contain` / `content-visibility`
- Reserve space to prevent CLS

# Quick Reference

selectors · at-rules · units · layout properties · functions · global keywords

## CSS AT A GLANCE

A one-page summary of the syntax covered in this set: the selectors you'll use most, the at-rules, units, layout properties, functions and the global keywords that work on every property. Each line points back to the sheet with the full explanation. When something on this page surprises you, open DevTools and check the cascade (sheet 05) and specificity (sheet 06) first.

## NOTES

- Specificity: (IDs, classes/attributes/pseudo-classes, types/pseudo-elements)
- Cascade: origin & importance → inline → layer → specificity → order
- Inherited: text properties; not inherited: box and layout properties
- Prefer `rem`, `gap`, logical properties and `@layer` in new code

## ♿ ACCESSIBILITY

- Focus: `:focus-visible` ring · Contrast: 4.5:1 / 3:1 · Motion: `prefers-reduced-motion` · Hide visually: `.visually-hidden`

## BROWSER SUPPORT

- Widely available: grid, flex gap, `clamp()`, `aspect-ratio`, `:has()`, nesting, container queries
- Newly available: `@property`, `light-dark()`, relative colors, `@starting-style`, `text-wrap: balance`
- Check first: anchor positioning, scroll-driven animations, `@scope`, view transitions

## CODE EXPLAINED

→ Combinators: space, `>`, `+`, `~`

→ Conditional at-rules: `@media`, `@container`, `@supports`

→ Definition at-rules: `@property`, `@font-face`, `@keyframes`

→ `repeat(auto-fit, minmax())` = responsive grid in one line

→ `clamp()` + `var()` cover most dynamic values

## SYNTAX SUMMARY

```
/* selectors */
*  p  .c  #i  [a="v"]  a:hover  p::first-line
A B  A > B  A + B  A ~ B
:is() :where() :not() :has() :nth-child(2n of .x)

/* at-rules */
@layer a, b;            @import url(x) layer(a);
@media (width >= 48rem) { }
@container (width > 30rem) { }
@supports (display: grid) { }
@property --x { syntax: '<length>'; inherits: false;
                initial-value: 0px; }
@font-face { font-family: X; src: url(x.woff2); }
@keyframes k { from { } to { } }
@scope (.card) to (.content) { }
@starting-style { }  @view-transition { navigation: auto; }

/* layout */
display: flex | grid | inline-flex | flow-root;
grid-template-columns: repeat(auto-fit, minmax(15rem, 1fr));
place-items: center;   gap: 1rem;
position: sticky; top: 0;   inset: 0;

/* values */
calc() min() max() clamp() var(--x, fb)
oklch(60% .15 285) color-mix(in oklch, a, b)
light-dark(#fff, #111)   attr(data-x)
```

## UNITS

| UNIT | RELATIVE TO |
|---|---|
| `px` | Reference pixel |
| `rem / em` | Root / own font size |
| `%` | Varies by property |
| `ch / lh` | 0 width / line height |
| `vw / vh / dvh` | Viewport |
| `cqi` | Container inline size |
| `fr` | Grid free space |

## GLOBAL KEYWORDS

| KEYWORD | RESULT |
|---|---|
| `inherit` | Parent's value |
| `initial` | Spec default |
| `unset` | inherit or initial |
| `revert` | Browser default |
| `revert-layer` | Previous layer's value |

## FIND IT

| TOPIC | SHEET |
|---|---|
| Cascade, layers | 05, 08 |
| Selectors | 11–18 |
| Units, math | 20–22 |
| Color | 25–27 |
| Flex, grid | 50–57 |
| Responsive | 58–62 |
| Motion | 63–68 |

## ⚠ COMMON MISTAKES

- Fixing a cascade problem with `!important` instead of layers or lower specificity
- Fixed `px` sizes everywhere instead of `rem` and intrinsic sizing
- Removing focus outlines

## ✓ TIPS & BEST PRACTICES

- Start every project with a reset, tokens and a layer order
- Reach for grid and flex `gap` before margins
- Check Baseline status before shipping new features without fallbacks

## QUICK SUMMARY

- Selectors → cascade → values
- Flex 1D, Grid 2D
- Tokens + layers + logical props
- Progressive enhancement for new CSS